

PromptToMotion

Full source code

A text-to-animation generator: describe a scene, get a detailed looping animation, then play, scrub and download it as MP4, GIF or Lottie.

Generated 2026-10-02 | Branch: claude/website-app-idea-2pu8ao

Every file is printed in full, with line numbers. Long lines wrap onto the next row and the continuation is marked with an arrow in the left margin.

Contents

index.html	Page: landing page and studio (HTML with Tailwind classes)	286 lines	3
css/app.css	Styles: colour tokens, controls, preview, entrance	200 lines	8
js/tailwind.config.js	Tailwind configuration	23 lines	12
js/fx.js	Site effects: entrance, particles, typing, reveals, loading orb	318 lines	13
js/runtime.js	Player inside the preview: clock, camera, lighting, blur, physics, exporters	589 lines	18
js/scenes.js	Scenes: examples, the offline generator, runner, dragon, castle, UFO	838 lines	27
js/sound.js	Procedural sound design	163 lines	40
js/app.js	The app: controls, generators, player control, exports, history	667 lines	43
build.py	Bundles everything into one HTML file	39 lines	54

9 files, 3123 lines of code in total.

```
1 <title>PromptToMotion</title>
2 <meta name="description" content="Describe an animation in words and get a detailed, looping animation. Export MP4, GIF or Lottie.">
3 <link rel="preconnect" href="https://fonts.googleapis.com">
4 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
5 <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=DM+Sans:wght@400;500;600;700&family=Syne:wght@700;800&display=swap">
6 <link rel="stylesheet" href="css/app.css">
7 <script src="https://cdn.tailwindcss.com/3.4.16"></script>
8 <script src="js/tailwind.config.js"></script>
9 <script>document.documentElement.dataset.intro = "wait";</script>
10
11 <!-- ===== Entrance ===== -->
12 <div id="intro" aria-hidden="true">
13   <canvas id="introCanvas"></canvas>
14   <div class="intro-center">
15     <svg class="intro-logo" viewBox="0 0 120 120" width="112" height="112" fill="none">
16       <circle class="il-ring" cx="60" cy="60" r="52" pathLength="1" />
17       <rect class="il-box" x="24" y="34" width="72" height="52" rx="14" pathLength="1" />
18       <path class="il-play" d="M52 48v24l22-12z" />
19     </svg>
20     <div class="intro-word" aria-hidden="true"><span>Prompt</span><span>To</span><span>Motion</span></div>
21     <div class="intro-bar"><i id="introBar"></i></div>
22     <div class="intro-pct tc" id="introPct">0%</div>
23   </div>
24   <button type="button" id="introSkip" class="intro-skip">Skip</button>
25 </div>
26
27 <!-- ===== Header ===== -->
28 <header id="siteHeader" class="site-header sticky top-0 z-30">
29   <div class="mx-auto flex max-w-[1180px] items-center justify-between gap-4 px-4 py-3">
30     <a href="#top" class="flex items-center gap-2 font-display text-lg font-extrabold tracking-tight">
31       <svg width="26" height="26" viewBox="0 0 26 26" aria-hidden="true"><rect x="1" y="4" width="24" height="18" rx="5" fill="var(--accent)"/><path d="M10.5 9.2v7.6l6.4-3.8z" fill="v
32       ar(--accent-ink)"/></svg>
33       PromptToMotion
34     </a>
35     <a href="#studio" class="btn btn-primary">Open studio</a>
36   </div>
37 </header>
38
39 <main id="top">
40   <!-- ===== Hero ===== -->
41   <section id="hero" class="hero relative overflow-hidden">
42     <canvas id="heroCanvas" aria-hidden="true"></canvas>
43     <div class="relative mx-auto grid max-w-[1180px] items-center gap-12 px-4 pb-20 pt-14 md:grid-cols-[minmax(0,1fr)_minmax(0,1.05fr)] md:pt-20">
44       <div class="min-w-0">
45         <h1 class="font-display text-[clamp(2.2rem,4.8vw,3.9rem)] font-extrabold leading-[1.02] tracking-tight"><span data-split>Describe it.</span><br><span class="grad-text" data-sp
46         lit>Watch it move.</span></h1>
47         <p class="mt-5 max-w-[42ch] text-[17px] text-mute" data-anim="up">Type a scene in plain words and get a detailed animation you can play, tweak and download.</p>
48         <form id="heroForm" class="mt-8" autocomplete="off" data-anim="up">
49           <label for="heroPrompt" class="sr-only">Describe your animation</label>
50           <div class="flex flex-col gap-3 sm:flex-row">
51             <div class="gborder relative flex-1">
52               <input id="heroPrompt" class="field field-bare text-[16px]" type="text" maxlength="400" title="Press the right arrow key in the empty box to use the idea that is typing"
53             >
54               <span id="heroTyper" class="typer" aria-hidden="true"></span>
55             </div>
56             <button class="btn btn-primary px-6 py-3 text-[15px]" type="submit">Generate</button>
57           </div>
58         </form>
59       </div>
60       <div class="min-w-0" data-anim="right">
61         <div class="hero-card tilt" data-tilt="4" id="heroCard">
62           <div class="monitor fill" id="heroMonitor" style="aspect-ratio:16/9">
63             <div class="busy" id="heroBusy" hidden>
64               <canvas id="heroOrb" aria-hidden="true"></canvas>
65               <div class="busy-inner">
66                 <div class="busy-head"><span class="ring-spin" aria-hidden="true"></span><p class="font-display text-base font-bold">Drawing your scene</p></div>
67                 <p class="prompt-echo" id="heroEcho"></p>
68               </div>
69             </div>
70           </div>
71         </div>
72       </div>
73     </div>
74   </section>
75 </main>
```

```
69     <div class="flex items-center justify-between gap-3 px-1 pt-3 text-[13px]">
70       <span class="flex min-w-0 items-center gap-2"><i class="rec-dot" aria-hidden="true"></i><span class="truncate font-semibold" id="heroTitle">Rooftop runner</span></span>
71       <span class="tc shrink-0 text-mute" id="heroTc">00:00.00</span>
72     </div>
73   </div>
74 </div>
75 </div>
76 </section>
77
78 <!-- ===== What you can direct ===== -->
79 <section class="mx-auto max-w-[1180px] px-4 py-20">
80   <h2 class="rv font-display text-3xl font-extrabold tracking-tight md:text-4xl">Direct it like a film</h2>
81   <div class="mt-10 grid gap-10 sm:grid-cols-2 lg:grid-cols-4">
82     <div class="rv feature">
83       <div class="fx-demo fx-cam" aria-hidden="true"><i class="orbit-path"></i><i class="cam-dot"></i><i class="cam-subject"></i></div>
84       <h3>Camera</h3><p>Push, pan, orbit, crane or handheld.</p>
85     </div>
86     <div class="rv feature">
87       <div class="fx-demo fx-light" aria-hidden="true"><i class="light-beam"></i><i class="light-obj"></i></div>
88       <h3>Lighting</h3><p>Sunlight, cinematic, neon, golden hour.</p>
89     </div>
90     <div class="rv feature">
91       <div class="fx-demo fx-blur" aria-hidden="true"><i class="blur-dot"></i><i class="blur-dot b2"></i><i class="blur-dot b3"></i><i class="blur-dot b4"></i></div>
92       <h3>Motion blur</h3><p>Real shutter blur, up to 60 fps.</p>
93     </div>
94     <div class="rv feature">
95       <div class="fx-demo fx-phys" aria-hidden="true"><i class="swing s1"></i><i class="swing s2"></i><i class="swing s3"></i></div>
96       <h3>Physics</h3><p>Cloth, hair and tails that move naturally.</p>
97     </div>
98   </div>
99 </section>
100
101 <!-- ===== Studio ===== -->
102 <section id="studio" class="studio-bg">
103   <div class="mx-auto max-w-[1180px] px-4 py-16">
104     <h2 class="font-display text-3xl font-extrabold tracking-tight md:text-4xl">Make an animation</h2>
105
106     <div class="mt-8 grid grid-cols-1 items-start gap-6 lg:grid-cols-[minmax(0,360px)_minmax(0,1fr)]">
107       <!-- Controls -->
108       <form id="studioForm" class="card grid min-w-0 gap-6" autocomplete="off">
109         <div class="grid gap-2">
110           <div class="flex items-baseline justify-between gap-2">
111             <label class="label" for="prompt">Describe your animation</label>
112             <button type="button" class="link-btn" id="surpriseBtn">Surprise me</button>
113           </div>
114           <div class="gborder relative">
115             <textarea id="prompt" class="field field-bare min-h-[104px] resize-y" maxlength="600" title="Press the right arrow key in the empty box to use the idea that is typing"><
116           /textarea>
117           <span id="promptTyper" class="typer typer-area" aria-hidden="true"></span>
118         </div>
119       </div>
120
121       <div class="grid gap-2">
122         <span class="label" id="styleLabel">Style</span>
123         <div class="seg seg-wrap" id="styleSeg" role="radiogroup" aria-labelledby="styleLabel"></div>
124         <p class="text-xs text-mute" id="styleNote"></p>
125       </div>
126
127       <div class="grid gap-4">
128         <div class="grid gap-1">
129           <div class="flex items-baseline justify-between"><label class="label" for="duration">Length</label><span class="tc text-sm" id="durationOut">6 s</span></div>
130           <input id="duration" type="range" min="2" max="15" step="1" value="6">
131         </div>
132         <div class="grid gap-4 min-[400px]:grid-cols-2">
133           <div class="grid min-w-0 gap-2"><span class="label" id="aspectLabel">Shape</span><div class="seg w-full" id="aspectSeg" role="radiogroup" aria-labelledby="aspectLabel"><
134         </div>
135           <div class="grid min-w-0 gap-2"><span class="label" id="fpsLabel">Frame rate</span><div class="seg w-full" id="fpsSeg" role="radiogroup" aria-labelledby="fpsLabel"></div>
136         </div>
137       </div>
138     </div>
139   </div>
140 </section>
```

```
137     <div class="grid gap-3">
138         <button class="btn btn-primary w-full py-3 text-[16px]" id="generateBtn" type="submit"><span class="spinner" aria-hidden="true"></span><span class="btn-label">Generate ani
139     mation</span></button>
140     <p class="min-h-[20px] text-sm text-mute" id="status" role="status" aria-live="polite"></p>
141 </div>
142 <!-- Everything else lives here, folded away -->
143 <details id="advanced" class="adv">
144     <summary><span>Advanced</span><span class="adv-sum" id="lookSummary"></span></summary>
145     <div class="adv-body">
146         <div class="grid gap-4 min-[400px]:grid-cols-2">
147             <div class="grid min-w-0 gap-1.5"><label class="label" for="cameraSel">Camera</label><select id="cameraSel" class="field"></select></div>
148             <div class="grid min-w-0 gap-1.5"><label class="label" for="lightSel">Lighting</label><select id="lightSel" class="field"></select></div>
149         </div>
150         <div class="grid gap-1">
151             <div class="flex items-baseline justify-between"><label class="label" for="camK">Camera strength</label><span class="tc text-sm" id="camKOut">1.0x</span></div>
152             <input id="camK" type="range" min="0.3" max="2" step="0.1" value="1">
153         </div>
154         <div class="grid gap-2"><span class="label" id="blurLabel">Motion blur</span><div class="seg w-full" id="blurSeg" role="radiogroup" aria-labelledby="blurLabel"></div></d
155     iv>
156     <div class="grid gap-2"><span class="label" id="detailLabel">Detail</span><div class="seg w-full" id="detailSeg" role="radiogroup" aria-labelledby="detailLabel"></div></
157     div>
158     <div class="grid gap-2">
159         <label class="check"><input type="checkbox" id="physics" class="accent-check"> Fluid character physics</label>
160         <label class="check"><input type="checkbox" id="smooth" class="accent-check" checked=""> Smooth preview</label>
161     </div>
162     <div class="grid gap-2 border-t border-line pt-4">
163         <div class="flex flex-wrap items-center justify-between gap-2">
164             <span class="label" id="engineLabel">Generator</span>
165             <div class="seg id="engineSeg" role="radiogroup" aria-labelledby="engineLabel">
166                 <button type="button" role="radio" data-engine="claude" aria-checked="false">Claude AI</button>
167                 <button type="button" role="radio" data-engine="mock" aria-checked="true">Offline</button>
168             </div>
169         </div>
170         <div class="flex flex-wrap items-center justify-between gap-2" id="tierRow">
171             <label class="label" for="tier">Thinking time</label>
172             <select id="tier" class="field w-auto">
173                 <option value="quick">Fast</option>
174                 <option value="default" selected="">Balanced</option>
175                 <option value="complex">Best</option>
176             </select>
177         </div>
178         <p class="text-xs text-mute" id="engineNote"></p>
179     </div>
180     <div class="grid gap-2 border-t border-line pt-4">
181         <div class="flex flex-wrap items-center gap-2">
182             <span class="label mr-auto">Scene code</span>
183             <button type="button" class="link-btn" id="codeBtn" aria-expanded="false">Show</button>
184             <button type="button" class="link-btn" id="copyBtn">Copy</button>
185         </div>
186         <pre class="code" id="code" hidden=""></pre>
187     </div>
188 </details>
189 </form>
190 <!-- Viewer -->
191 <div class="card min-w-0">
192     <div class="mb-4 flex flex-wrap items-baseline justify-between gap-x-4 gap-y-1">
193         <h3 class="min-w-0 truncate font-display text-xl font-bold" id="takeTitle">Rooftop runner</h3>
194         <p class="text-sm text-mute" id="takeMeta"></p>
195     </div>
196     <div class="monitor" id="monitor" style="aspect-ratio:16/9" title="Drag to steer the camera">
197         <div class="busy" id="busy" hidden="">
198             <canvas id="orbCanvas" aria-hidden="true"></canvas>
199             <div class="busy-inner">
200                 <div class="busy-head"><span class="ring-spin" aria-hidden="true"></span><p class="font-display text-lg font-bold" id="busyTitle">Making your animation</p></div>
201                 <p class="prompt-echo" id="promptEcho"></p>
202                 <ul id="busySteps"></ul>
203                 <div class="bar busy-bar" aria-hidden="true"><i id="busyBar"></i></div>
204                 <p class="tc text-xs" style="color:#A9AEC0" id="busyDetail"></p>

```

```
205         <button type="button" class="btn w-fit" id="stopBtn">Stop</button>
206     </div>
207 </div>
208 </div>
209
210 <!-- Transport -->
211 <div class="mt-4 grid gap-2">
212     <input id="scrub" type="range" min="0" max="1000" value="0" aria-label="Timeline">
213     <div class="flex flex-wrap items-center gap-x-1.5 gap-y-2">
214         <button type="button" class="btn btn-icon" id="prevBtn" title="Previous frame (-)" aria-label="Previous frame"><svg width="16" height="16" viewBox="0 0 16 16" fill="currentColor"><path d="M3 3h2v10H3zM13 3v10L6 8z"/></svg></button>
215         <button type="button" class="btn btn-icon btn-primary" id="playBtn" title="Play or pause (Space)" aria-label="Pause"><svg id="playIcon" width="16" height="16" viewBox="0 0 16 16" fill="currentColor"><path d="M3 2h4v12H3zM9 2h4v12H9z"/></svg></button>
216         <button type="button" class="btn btn-icon" id="nextBtn" title="Next frame (->)" aria-label="Next frame"><svg width="16" height="16" viewBox="0 0 16 16" fill="currentColor"><path d="M11 3h2v10h-2zM3 3v10L7 5z"/></svg></button>
217         <span class="tc ml-2 text-sm" id="timecode">00:00.00 / 00:06.00</span>
218         <span id="frameNo" class="hidden">frame 1/180</span>
219         <span class="ml-auto flex items-center gap-1.5">
220             <button type="button" class="btn btn-icon" id="loopBtn" aria-pressed="true" title="Loop (L)" aria-label="Loop"><svg width="16" height="16" viewBox="0 0 16 16" fill="none" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"><path d="M2.5 7V6a2 2 0 0 1 2-2h7l-2-2M13.5 9v1a2 2 0 0 1-2 2h-7l2 2"/></svg></button>
221             <button type="button" class="btn btn-icon" id="soundBtn" aria-pressed="false" title="Sound for the preview (downloads are silent)" aria-label="Sound"><svg width="16" height="16" viewBox="0 0 16 16" fill="none" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"><path d="M2 6v4h3l4 3V3L5 6z"/><path d="M11.5 5.5a3.5 3.5 0 0 1 0 5"/></svg></button>
222             <button type="button" class="btn btn-icon" id="fullBtn" title="Full screen" aria-label="Full screen"><svg width="16" height="16" viewBox="0 0 16 16" fill="none" stroke="currentColor" stroke-width="2"><path d="M2 6V2h4M10 2h4V6M14 2h4V6M14 10h4V2"/></svg></button>
223         </span>
224     </div>
225 </div>
226
227 <!-- Change + download -->
228 <div class="mt-6 grid gap-5 border-t border-line pt-5">
229     <div class="grid gap-2">
230         <label class="label" for="change">Change it</label>
231         <div class="flex flex-col gap-2 sm:flex-row">
232             <input id="change" class="field flex-1" type="text" placeholder="Make it night time, add rain, speed it up..." maxlength="300">
233             <button type="button" class="btn" id="changeBtn">Apply</button>
234         </div>
235     </div>
236     <div class="hidden items-center justify-between gap-2 rounded-xl border border-err px-3 py-2 text-sm text-err" id="errBox">
237         <span id="errText" class="min-w-0"></span>
238         <button type="button" class="btn" id="fixBtn">Fix it</button>
239     </div>
240     <div class="grid gap-2">
241         <span class="label">Download</span>
242         <div class="flex flex-wrap gap-2" id="exportBtns">
243             <button type="button" class="btn" data-format="video" title="Records in real time, up to 1280 px">MP4</button>
244             <button type="button" class="btn" data-format="gif" title="Loops forever, 480 px">GIF</button>
245             <button type="button" class="btn" data-format="lottie" title="JSON for Lottie players">Lottie</button>
246         </div>
247         <div class="hidden gap-2" id="exportProg">
248             <div class="flex items-center justify-between gap-2 text-sm"><span id="exportLabel">Rendering</span><button type="button" class="link-btn" id="exportCancel">Cancel</button></div>
249             <div class="bar"><i id="exportBar"></i></div>
250         </div>
251     </div>
252 </div>
253 </div>
254 </div>
255
256 <!-- History -->
257 <div class="mt-16">
258     <div class="mb-5 flex flex-wrap items-center justify-between gap-3">
259         <h3 class="font-display text-2xl font-extrabold tracking-tight">Your animations</h3>
260         <div class="flex flex-wrap items-center gap-3">
261             <div class="seg id="galleryTabs" role="tablist" aria-label="Gallery">
262                 <button type="button" role="tab" data-tab="mine" aria-pressed="true">Mine</button>
263                 <button type="button" role="tab" data-tab="examples" aria-pressed="false">Examples</button>
264             </div>
265             <button type="button" class="link-btn" id="clearBtn">Clear history</button>
266         </div>
267     </div>
```

```
268         <div class="grid grid-cols-1 items-start gap-4 min-[480px]:grid-cols-2 md:grid-cols-3 xl:grid-cols-4" id="gallery"></div>
269     </div>
270 </div>
271 </section>
272 </main>
273
274 <footer>
275     <div class="mx-auto flex max-w-[1180px] flex-wrap items-center justify-between gap-3 px-4 py-8 text-sm text-mute" style="padding-bottom: calc(2rem + env(safe-area-inset-bottom, 0px))">
276         <span class="font-display font-bold text-ink">PromptToMotion</span>
277         <span>Your animations stay in this browser.</span>
278     </div>
279 </footer>
280
281 <script src="https://cdn.jsdelivr.net/npm/motion@11.18.2/dist/motion.js"></script>
282 <script src="js/fx.js"></script>
283 <script src="js/runtime.js"></script>
284 <script src="js/scenes.js"></script>
285 <script src="js/sound.js"></script>
286 <script src="js/app.js"></script>
```

```

1  /* PromptToMotion styles. Colours live in the tokens below; js/tailwind.config.js maps them into Tailwind. */
2  :root {
3    --bg: #F6F7FB; --panel: #FFFFFF; --panel2: #EEF0F7; --ink: #12141C; --mute: #5A6275; --line: #E0E3EC;
4    --accent: #3B47F5; --accent-ink: #FFFFFF; --accent-soft: rgba(59, 71, 245, 0.09);
5    --hot: #E8481F; --ok: #12805A; --err: #C7362B; --stage: #0A0B10;
6  }
7  @media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) {
8    --bg: #0B0C11; --panel: #14161E; --panel2: #10121A; --ink: #ECEE6; --mute: #9AA1B5; --line: #252938;
9    --accent: #8892FF; --accent-ink: #0B0C11; --accent-soft: rgba(136, 146, 255, 0.14);
10   --hot: #FF7A5C; --ok: #45C28A; --err: #FF6F63; --stage: #000000; color-scheme: dark;
11  } }
12  :root[data-theme="dark"] {
13    --bg: #0B0C11; --panel: #14161E; --panel2: #10121A; --ink: #ECEE6; --mute: #9AA1B5; --line: #252938;
14    --accent: #8892FF; --accent-ink: #0B0C11; --accent-soft: rgba(136, 146, 255, 0.14);
15    --hot: #FF7A5C; --ok: #45C28A; --err: #FF6F63; --stage: #000000; color-scheme: dark;
16  }
17
18  html { scroll-behavior: smooth; scroll-padding-top: 72px; }
19  body { background: var(--bg); color: var(--ink); font-family: "DM Sans", "Segoe UI", system-ui, sans-serif; font-size: 15px; line-height: 1.55; -webkit-font-smoothing: antialiased; }
20  [hidden] { display: none !important; }
21  h1, h2, h3 { text-wrap: balance; }
22  :focus-visible { outline: 2px solid var(--accent); outline-offset: 2px; border-radius: 6px; }
23  .tnum, .tc { font-family: "DM Mono", ui-monospace, Menlo, monospace; font-variant-numeric: tabular-nums; }
24  .label { font-size: 12px; font-weight: 600; color: var(--mute); }
25
26  /* ----- Buttons and inputs ----- */
27  .btn { position: relative; overflow: hidden; isolation: isolate; display: inline-flex; align-items: center; justify-content: center; gap: 8px; font-weight: 600; font-size: 14px; border-radius: 10px; padding: 9px 16px; border: 1px solid var(--line); background: var(--panel); color: var(--ink); cursor: pointer; transition: border-color 0.15s, box-shadow 0.2s, transform 0.12s; }
28  .btn::after { content: ""; position: absolute; inset: 0; z-index: -1; background: radial-gradient(120px circle at var(--mx, 50%) var(--my, 50%), var(--accent-soft), transparent 70%); opacity: 0; transition: opacity 0.2s; }
29  .btn:hover:not(:disabled) { border-color: var(--mute); }
30  .btn:hover:not(:disabled)::after { opacity: 1; }
31  .btn:active:not(:disabled) { transform: scale(0.97); }
32  .btn:disabled { opacity: 0.45; cursor: not-allowed; }
33  .btn-primary { background: var(--accent); color: var(--accent-ink); border-color: var(--accent); }
34  .btn-primary::after { background: radial-gradient(120px circle at var(--mx, 50%) var(--my, 50%), rgba(255, 255, 255, 0.35), transparent 70%); }
35  .btn-primary:hover:not(:disabled) { border-color: var(--accent); box-shadow: 0 8px 26px -8px var(--accent); }
36  .btn-icon { padding: 0; width: 38px; height: 38px; }
37  .btn-icon[aria-pressed="true"] { background: var(--accent-soft); border-color: var(--accent); color: var(--accent); }
38  .btn-icon[aria-pressed="false"] svg { opacity: 0.55; }
39  .link-btn { background: none; border: 0; padding: 2px 0; font: 600 13px "DM Sans", system-ui, sans-serif; color: var(--accent); cursor: pointer; }
40  .link-btn:hover { text-decoration: underline; }
41  .spinner { display: none; width: 16px; height: 16px; border-radius: 50%; border: 2.5px solid rgba(255, 255, 255, 0.35); border-top-color: #fff; animation: spinr 0.7s linear infinite; }
42  #generateBtn[data-busy="1"] .spinner { display: inline-block; }
43
44  .field { width: 100%; font: inherit; color: var(--ink); background: var(--bg); border: 1px solid var(--line); border-radius: 12px; padding: 11px 14px; }
45  .field::placeholder { color: var(--mute); opacity: 0.8; }
46  select.field { padding: 9px 12px; }
47  .field-bare { border: 0; background: transparent; border-radius: 12px; outline: none; }
48  .check { display: flex; align-items: center; gap: 8px; font-size: 14px; cursor: pointer; }
49  .accent-check { accent-color: var(--accent); width: 16px; height: 16px; }
50
51  .seg { display: inline-flex; border: 1px solid var(--line); border-radius: 10px; overflow: hidden; background: var(--panel); }
52  .seg button { flex: 1 1 auto; min-width: 0; white-space: nowrap; padding: 7px 8px; font-size: 13px; font-weight: 600; color: var(--mute); background: transparent; border: 0; cursor: pointer; transition: background 0.15s, color 0.15s; }
53  .seg button + button { border-left: 1px solid var(--line); }
54  .seg button:hover { color: var(--ink); }
55  .seg button[aria-pressed="true"], .seg button[aria-checked="true"] { background: var(--ink); color: var(--bg); }
56  .seg.w-full { display: flex; }
57  .seg-wrap { flex-wrap: wrap; border: 0; background: none; gap: 6px; border-radius: 0; overflow: visible; }
58  .seg-wrap button { flex: 0 0 auto; border: 1px solid var(--line); border-radius: 999px; background: var(--panel); }
59  .seg-wrap button + button { border-left: 1px solid var(--line); }
60  .seg-wrap button[aria-checked="true"] { border-color: var(--ink); }
61
62  input[type="range"] { -webkit-appearance: none; appearance: none; width: 100%; height: 22px; background: transparent; cursor: pointer; }
63  input[type="range"]::-webkit-slider-runnable-track { height: 4px; border-radius: 2px; background: var(--line); }
64  input[type="range"]::-moz-range-track { height: 4px; border-radius: 2px; background: var(--line); }
65  input[type="range"]::-webkit-slider-thumb { -webkit-appearance: none; width: 16px; height: 16px; margin-top: -6px; border-radius: 50%; background: var(--accent); border: 2px solid var(--panel); box-shadow: 0 0 0 1px var(--accent); }

```

```
66 input[type="range"]::-moz-range-thumb { width: 12px; height: 12px; border-radius: 50%; background: var(--accent); border: 2px solid var(--panel); }
67 input[type="range"]:disabled { opacity: 0.4; cursor: not-allowed; }
68 #scrub::-webkit-slider-thumb { background: var(--hot); box-shadow: 0 0 1px var(--hot); width: 14px; height: 14px; margin-top: -5px; }
69 #scrub::-moz-range-thumb { background: var(--hot); }
70
71 /* The prompt boxes: a plain border that turns into a moving gradient only while you type */
72 .gborder { border: 1.5px solid var(--line); border-radius: 14px; background: var(--panel); transition: box-shadow 0.25s; }
73 @property --ang { syntax: "<angle>"; inherits: false; initial-value: 0deg; }
74 .gborder:focus-within { border-color: transparent; background: linear-gradient(var(--panel), var(--panel)) padding-box, conic-gradient(from var(--ang), var(--accent), var(--hot), #22d
> 3ee, var(--accent)) border-box; animation: spinAng 5s linear infinite; box-shadow: 0 0 4px var(--accent-soft); }
75 @keyframes spinAng { to { --ang: 360deg; } }
76 .typer { position: absolute; left: 16px; right: 16px; top: 50%; transform: translateY(-50%); color: var(--mute); white-space: nowrap; overflow: hidden; pointer-events: none; font-size
> : 16px; }
77 .typer::after { content: ""; display: inline-block; width: 2px; height: 1.1em; margin-left: 2px; vertical-align: -0.2em; background: var(--accent); animation: blink 1s steps(1) infini
> te; }
78 .typer.off { display: none; }
79 .typer-area { top: 13px; transform: none; white-space: normal; font-size: 15px; }
80 @keyframes blink { 50% { opacity: 0; } }
81
82 /* ----- Layout pieces ----- */
83 .card { min-width: 0; background: var(--panel); border: 1px solid var(--line); border-radius: 20px; padding: 24px; }
84 @media (max-width: 480px) { .card { padding: 18px; border-radius: 16px; } }
85 .card.grid, .adv-body { grid-template-columns: minmax(0, 1fr); }
86 .studio-bg { background: var(--panel2); border-top: 1px solid var(--line); }
87 .site-header { background: color-mix(in srgb, var(--bg) 82%, transparent); backdrop-filter: blur(12px); -webkit-backdrop-filter: blur(12px); border-bottom: 1px solid transparent; padd
> ing-top: env(safe-area-inset-top, 0px); transition: border-color 0.25s; }
88 .site-header.scrolled { border-bottom-color: var(--line); }
89 .hero { background: radial-gradient(900px 420px at 88% -5%, var(--accent-soft), transparent 70%), var(--bg); isolation: isolate; }
90 #heroCanvas { position: absolute; inset: 0; width: 100%; height: 100%; z-index: -1; -webkit-mask-image: linear-gradient(90deg, transparent 8%, rgba(0, 0, 0, 0.5) 45%, #000 75%); mask-
> image: linear-gradient(90deg, transparent 8%, rgba(0, 0, 0, 0.5) 45%, #000 75%); }
91 .hl-word { display: inline-block; white-space: nowrap; }
92 .hl-ch { display: inline-block; will-change: transform, opacity, filter; }
93 .grad-text .hl-ch { background: linear-gradient(100deg, var(--accent) 20%, var(--hot) 50%, var(--accent) 80%); background-size: 300% 100%; -webkit-background-clip: text; background-cl
> ip: text; color: transparent; -webkit-text-fill-color: transparent; animation: shimmer 7s linear infinite; animation-delay: calc(var(--ci, 0) * -0.14s); }
94 @keyframes shimmer { to { background-position: -200% 0; } }
95 .hero-card { background: var(--panel); border: 1px solid var(--line); border-radius: 22px; padding: 8px 8px 12px; box-shadow: 0 30px 60px -36px rgba(30, 40, 120, 0.45); transition: tr
> ansform 0.18s ease-out; will-change: transform; }
96 .rec-dot { width: 8px; height: 8px; border-radius: 50%; background: var(--hot); flex: none; animation: blinkdot 1.6s ease-in-out infinite; }
97 @keyframes blinkdot { 50% { opacity: 0.3; } }
98
99 /* Details fold */
100 .adv { border-top: 1px solid var(--line); padding-top: 4px; }
101 .adv summary { list-style: none; cursor: pointer; display: flex; align-items: baseline; justify-content: space-between; gap: 10px; padding-block: 12px; font-weight: 600; font-size: 14
> px; }
102 .adv summary::-webkit-details-marker { display: none; }
103 .adv summary::after { content: ""; flex: none; width: 8px; height: 8px; margin-left: 4px; border-right: 2px solid var(--mute); border-bottom: 2px solid var(--mute); transform: rotate(
> 45deg) translateY(-2px); transition: transform 0.2s; align-self: center; }
104 .adv[open] summary::after { transform: rotate(-135deg); }
105 .adv-sum { margin-left: auto; font-weight: 400; font-size: 12px; color: var(--mute); text-align: right; min-width: 0; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
106 .adv[open] .adv-sum { display: none; }
107 .adv-body { display: grid; gap: 18px; padding-block: 8px 4px; }
108
109 /* Features row on the landing page: no boxes, just a small live demo, a name and a line */
110 .feature { display: grid; gap: 4px; align-content: start; }
111 .feature h3 { font: 700 18px/1.2 "Syne", "Arial Black", system-ui, sans-serif; margin: 10px 0 0; }
112 .feature p { color: var(--mute); margin: 0; font-size: 15px; }
113 .fx-demo { height: 84px; border-radius: 14px; position: relative; overflow: hidden; background: var(--panel); border: 1px solid var(--line); }
114 .fx-cam .orbit-path { position: absolute; left: 50%; top: 50%; width: 110px; height: 46px; margin: -23px 0 0 -55px; border: 1.5px dashed var(--line); border-radius: 50%; }
115 .fx-cam .cam-dot { position: absolute; left: 0; top: 0; width: 14px; height: 10px; border-radius: 3px; background: var(--accent); offset-path: ellipse(55px 23px at 50% 50%); offset-ro
> tate: 0deg; animation: along 4s linear infinite; }
116 @keyframes along { from { offset-distance: 0%; } to { offset-distance: 100%; } }
117 .fx-cam .cam-subject { position: absolute; left: 50%; top: 50%; width: 20px; height: 20px; margin: -10px; border-radius: 50%; background: radial-gradient(circle at 35% 35%, #fff, var(
> --hot)); }
118 .fx-light .light-beam { position: absolute; left: -10px; top: -30px; width: 70%; height: 170%; background: linear-gradient(105deg, rgba(255, 210, 130, 0.6), transparent 72%); transfor
> m-origin: top left; animation: sweep 5s ease-in-out infinite alternate; }
119 @keyframes sweep { from { transform: rotate(-4deg); } to { transform: rotate(28deg); } }
120 .fx-light .light-obj { position: absolute; left: 50%; top: 50%; width: 30px; height: 30px; margin: -12px 0 0 -15px; border-radius: 50%; background: radial-gradient(circle at 25% 30%,
> #fff5dc, var(--accent) 45%, #14163a 100%); animation: litpulse 5s ease-in-out infinite alternate; }
121 @keyframes litpulse { from { filter: brightness(0.7); } to { filter: brightness(1.25); } }
122 .fx-blur .blur-dot { position: absolute; top: 35px; left: -20px; width: 14px; height: 14px; border-radius: 50%; background: var(--hot); animation: dash 1.8s cubic-bezier(0.45, 0.05, 0
> .55, 0.95) infinite; }
```

```
123 .fx-blur .b2 { animation-delay: 0.035s; opacity: 0.5; } .fx-blur .b3 { animation-delay: 0.07s; opacity: 0.3; } .fx-blur .b4 { animation-delay: 0.105s; opacity: 0.18; }
124 @keyframes dash { 0% { left: -20px; } 75%, 100% { left: calc(100% + 6px); } }
125 .fx-phys .swing { position: absolute; top: 0; width: 2px; height: 50px; background: var(--mute); transform-origin: top center; animation: sway 2.2s ease-in-out infinite; }
126 .fx-phys .swing::after { content: ""; position: absolute; left: -7px; bottom: -14px; width: 16px; height: 16px; border-radius: 50%; background: var(--accent); }
127 .fx-phys .s1 { left: 30%; } .fx-phys .s2 { left: 50%; animation-delay: -0.2s; } .fx-phys .s3 { left: 70%; animation-delay: -0.4s; }
128 @keyframes sway { 0%, 100% { transform: rotate(32deg); } 50% { transform: rotate(-32deg); } }
129
130 /* ----- The preview ----- */
131 .monitor { --ar: 1.7778; --maxh: min(66vh, 620px); position: relative; width: min(100%, calc(var(--maxh) * var(--ar))); background: var(--stage); border-radius: 14px; overflow: hidden
> ; margin-inline: auto; }
132 .monitor.fill { width: 100%; }
133 .monitor.iframe { position: absolute; inset: 0; width: 100%; height: 100%; border: 0; display: block; }
134 .monitor.fullscreen { width: 100vw; height: 100vh; aspect-ratio: auto !important; border-radius: 0; }
135 .monitor.making { animation: glowPulse 1.8s ease-in-out infinite; }
136 @keyframes glowPulse { 50% { box-shadow: 0 0 2px var(--accent), 0 0 40px -4px var(--accent); } }
137 .busy { position: absolute; inset: 0; background: rgba(6, 7, 14, 0.88); color: #f1f2f7; display: grid; place-items: center; padding: 16px; overflow: hidden; -webkit-backdrop-filter: b
> lur(6px); backdrop-filter: blur(6px); }
138 .busy.canvas { position: absolute; inset: 0; width: 100%; height: 100%; }
139 .busy.inner { position: relative; display: grid; gap: 10px; width: min(430px, 100%); justify-items: center; text-align: center; max-height: 100%; overflow: auto; }
140 .busy.head { display: flex; align-items: center; gap: 10px; }
141 .ring.spin { width: 22px; height: 22px; border-radius: 50%; border: 3px solid rgba(255, 255, 255, 0.18); border-top-color: #ff7a5c; border-right-color: #8892ff; animation: spinr 0.8s
> linear infinite; flex: none; }
142 @keyframes spinr { to { transform: rotate(360deg); } }
143 .prompt-echo { font: 600 15px/1.4 "DM Sans", system-ui, sans-serif; color: #eaeaff; max-width: 38ch; min-height: 2.8em; text-shadow: 0 0 18px rgba(136, 146, 255, 0.9); margin: 0; }
144 .prompt-echo::after { content: ""; display: inline-block; width: 2px; height: 1em; margin-left: 2px; vertical-align: -0.15em; background: #ff7a5c; animation: blink 0.9s steps(1) infin
> ite; }
145 .busy ul { list-style: none; margin: 0; padding: 0; display: grid; gap: 6px; font-size: 14px; text-align: left; }
146 .busy li { display: flex; gap: 10px; align-items: center; color: #A9AEC0; }
147 .busy li[data-s="done"] { color: #F1F2F7; }
148 .busy li[data-s="run"] { color: #FFFFFF; font-weight: 600; }
149 .dot { width: 10px; height: 10px; border-radius: 50%; border: 2px solid #6E7590; flex: none; }
150 li[data-s="done"] .dot { background: #45C28A; border-color: #45C28A; }
151 li[data-s="run"] .dot { border-color: #FF7A5C; animation: pulse 1s infinite; }
152 @keyframes pulse { 50% { transform: scale(1.5); opacity: 0.5; } }
153 .bar { height: 6px; border-radius: 3px; background: var(--line); overflow: hidden; }
154 .bar > i { display: block; height: 100%; width: 0; background: var(--hot); transition: width 0.12s linear; }
155 .busy-bar { width: 100%; height: 4px; background: rgba(255, 255, 255, 0.14); }
156 #busyBar { background: linear-gradient(90deg, #8892ff, #ff7a5c); transition: width 0.5s cubic-bezier(0.22, 1, 0.36, 1); }
157 #heroBusy .busy-inner { gap: 6px; }
158 #heroBusy .prompt-echo { font-size: 13px; min-height: 2.6em; }
159
160 /* ----- Gallery ----- */
161 .take { display: grid; gap: 10px; text-align: left; padding: 8px 8px 12px; border: 1px solid var(--line); border-radius: 16px; background: var(--panel); cursor: pointer; transition: b
> order-color 0.15s, transform 0.15s; }
162 .take:hover { border-color: var(--mute); transform: translateY(-2px); }
163 .take[aria-current="true"] { border-color: var(--accent); box-shadow: 0 0 1px var(--accent); }
164 .take .thumb { width: 100%; border-radius: 10px; background: var(--stage); overflow: hidden; display: grid; place-items: center; }
165 .take .thumb img { width: 100%; height: 100%; object-fit: cover; display: block; }
166 .shimmer { background: linear-gradient(100deg, var(--panel2) 30%, var(--line) 50%, var(--panel2) 70%); background-size: 200% 100%; animation: shimmer 1.4s linear infinite; }
167 .code { max-height: 300px; overflow: auto; background: var(--bg); border: 1px solid var(--line); border-radius: 10px; padding: 12px; font: 12px/1.55 "DM Mono", ui-monospace, Menlo, mo
> nospace; white-space: pre; margin: 0; }
168
169 /* ----- Entrance overlay: always dark on purpose. A CSS-only failsafe removes it after 7 s if scripts fail. ----- */
170 #intro { position: fixed; inset: 0; z-index: 9999; background: #05060d; display: grid; place-items: center; color: #f1f2f7; overflow: hidden; cursor: pointer; animation: introFail 0s
> linear 7s forwards; }
171 @keyframes introFail { to { opacity: 0; visibility: hidden; pointer-events: none; } }
172 #introCanvas { position: absolute; inset: 0; width: 100%; height: 100%; }
173 .intro-center { position: relative; display: grid; justify-items: center; gap: 16px; transform: translateY(-4vh); }
174 .intro-logo { filter: drop-shadow(0 0 20px rgba(136, 146, 255, 0.85)); overflow: visible; }
175 .intro-logo * { fill: none; stroke: #8892ff; stroke-width: 3; stroke-linecap: round; stroke-linejoin: round; stroke-dasharray: 1; stroke-dashoffset: 1; }
176 .intro-logo .il-ring { stroke-width: 1.5; stroke: #ff7a5c; opacity: 0.8; animation: drawIn 1.1s 0.1s cubic-bezier(0.22, 1, 0.36, 1) forwards; }
177 .intro-logo .il-box { animation: drawIn 1s 0.35s cubic-bezier(0.22, 1, 0.36, 1) forwards, boxFill 0.5s 1.05s ease-out forwards; }
178 .intro-logo .il-play { stroke: none; fill: #fff; stroke-dasharray: none; stroke-dashoffset: 0; opacity: 0; transform-box: fill-box; transform-origin: center; animation: playPop 0.55s
> 0.95s cubic-bezier(0.34, 1.56, 0.64, 1) forwards; }
179 @keyframes drawIn { to { stroke-dashoffset: 0; } }
180 @keyframes boxFill { to { fill: rgba(136, 146, 255, 0.28); } }
181 @keyframes playPop { from { opacity: 0; transform: scale(0.2) rotate(-40deg); } to { opacity: 1; transform: scale(1) rotate(0); } }
182 .intro-word { font: 800 clamp(22px, 5vw, 34px) / 1 "Syne", "Arial Black", system-ui, sans-serif; letter-spacing: -0.02em; display: flex; gap: 0.32em; }
183 .intro-word span { display: inline-block; opacity: 0; transform: translateY(20px); filter: blur(10px); animation: wordIn 0.75s cubic-bezier(0.22, 1, 0.36, 1) forwards; }
184 .intro-word span:nth-child(1) { animation-delay: 0.55s; } .intro-word span:nth-child(2) { animation-delay: 0.68s; color: #ff7a5c; } .intro-word span:nth-child(3) { animation-delay: 0.
> 81s; }
```

```
185 @keyframes wordIn { to { opacity: 1; transform: none; filter: blur(0); } }
186 .intro-bar { width: min(240px, 60vw); height: 3px; background: rgba(255, 255, 255, 0.12); border-radius: 2px; overflow: hidden; }
187 .intro-bar i { display: block; height: 100%; width: 0; background: linear-gradient(90deg, #8892ff, #fff7a5c); box-shadow: 0 0 14px #8892ff; }
188 .intro-pct { font-size: 12px; color: #8b90a6; }
189 .intro-skip { position: absolute; right: 16px; bottom: calc(16px + env(safe-area-inset-bottom, 0px)); background: transparent; border: 1px solid rgba(255, 255, 255, 0.2); color: #b9bd
> d0; border-radius: 999px; padding: 6px 14px; font: 600 12px "DM Sans", system-ui, sans-serif; cursor: pointer; }
190 .intro-skip:hover { border-color: #8892ff; color: #fff; }
191 /* Content waits for the entrance, then animates in (fx.js). Failsafes show it after 7.5 s even if scripts fail. */
192 html[data-intro="wait"] [data-anim], html[data-intro="wait"] .hl-ch { opacity: 0; animation: introShow 0s linear 7.5s forwards; }
193 @keyframes introShow { to { opacity: 1; } }
194 html.rv-on .rv:not(.in) { opacity: 0; animation: introShow 0s linear 20s forwards; }
195
196 @media (prefers-reduced-motion: reduce) {
197   html { scroll-behavior: auto; }
198   *, *::before, *::after { animation-duration: 0.001ms !important; animation-iteration-count: 1 !important; animation-delay: 0s !important; transition-duration: 0.001ms !important; }
199   html[data-intro="wait"] [data-anim], html[data-intro="wait"] .hl-ch, html.rv-on .rv:not(.in) { opacity: 1; }
200 }
```

```
1  /*
2  * Tailwind config, shared by the Play CDN (browser) and the Tailwind CLI (node).
3  * All colours point at CSS variables in css/app.css, so light and dark themes are one place to change.
4  */
5  const ptmTailwindConfig = {
6    content: ["/index.html", "/js/app.js", "/js/fx.js"],
7    theme: {
8      extend: {
9        colors: {
10          bg: "var(--bg)", panel: "var(--panel)", panel2: "var(--panel2)", ink: "var(--ink)", mute: "var(--mute)",
11          line: "var(--line)", accent: "var(--accent)", "accent-ink": "var(--accent-ink)", "accent-soft": "var(--accent-soft)",
12          hot: "var(--hot)", ok: "var(--ok)", err: "var(--err)", stage: "var(--stage)"
13        },
14        fontFamily: {
15          display: ["Syne", "Arial Black", "system-ui", "sans-serif"],
16          sans: ["DM Sans", "Segoe UI", "system-ui", "sans-serif"],
17          mono: ["DM Mono", "ui-monospace", "Menlo", "monospace"]
18        }
19      }
20    }
21  };
22  if (typeof module !== "undefined" && module.exports) module.exports = ptmTailwindConfig;
23  else window.tailwind.config = ptmTailwindConfig;
```

```

1  /*
2  * PromptToMotion site effects.
3  *
4  * Everything that makes the page itself move lives here, separate from the app logic:
5  *   - play()       one animation call. Uses Motion (Framer Motion's vanilla engine, window.Motion) when it loaded,
6  *                   and falls back to the Web Animations API, so the page animates either way.
7  *   - runIntro()    entrance: glowing particles converge, logo draws itself, an iris opens onto the hero
8  *   - heroMesh()    interactive particle and wave-mesh background that reacts to the pointer
9  *   - typer()       typing effect for example prompts
10 *   - tilt / glow    pointer-driven 3D tilt on cards and a cursor-following glow on buttons
11 *   - reveal        scroll-in animation with stagger
12 *   - orb()         the generation "loading" orb (also used for the burst when a preview is ready)
13 * Everything respects prefers-reduced-motion.
14 */
15 (() => {
16   "use strict";
17   const $ = (s, r = document) => r.querySelector(s), $$ = (s, r = document) => [...r.querySelectorAll(s)];
18   const root = document.documentElement;
19   const reduce = matchMedia("(prefers-reduced-motion: reduce)").matches;
20   const clamp = (v, a, b) => Math.min(b, Math.max(a, v));
21   const lerp = (a, b, k) => a + (b - a) * k;
22   const easeInOut = x => (x < 0.5 ? 4 * x * x * x : 1 - Math.pow(-2 * x + 2, 3) / 2);
23   const easeOut = x => 1 - Math.pow(1 - x, 3);
24   const cssVar = n => getComputedStyle(root).getPropertyValue(n).trim();
25
26   // ----- animate wrapper
27   const EASE = { out: [0.22, 1, 0.36, 1], inOut: [0.65, 0, 0.35, 1], spring: [0.34, 1.56, 0.64, 1], linear: "linear" };
28   const bez = e => (Array.isArray(e) ? "cubic-bezier(" + e.join(",") + ")" : e);
29   const at = (arr, i) => arr[Math.min(i, arr.length - 1)];
30   function styleAt(kf, i) {
31     const s = {}, tr = [];
32     if (kf.opacity) s.opacity = at(kf.opacity, i);
33     if (kf.x || kf.y) tr.push("translate(" + (kf.x ? at(kf.x, i) : 0) + "px," + (kf.y ? at(kf.y, i) : 0) + "px)");
34     if (kf.scale) tr.push("scale(" + at(kf.scale, i) + ")");
35     if (kf.rotate) tr.push("rotate(" + at(kf.rotate, i) + "deg)");
36     if (tr.length) s.transform = tr.join(" ");
37     if (kf.filter) s.filter = at(kf.filter, i);
38     if (kf.clipPath) s.clipPath = at(kf.clipPath, i);
39     return s;
40   }
41   const setStyle = (el, s) => Object.assign(el.style, s);
42   const clearStyle = (el, kf) => { for (const p of Object.keys(styleAt(kf, 0))) el.style[p] = ""; };
43
44   // play(el, { opacity: [0, 1], y: [24, 0], filter: ["blur(8px)", "blur(0px)"] }, { duration, delay, ease, keep })
45   // Resolves when done. Unless keep is true, inline styles are cleared afterwards so the element returns to its normal look.
46   function play(el, kf, o = {}) {
47     if (!el) return Promise.resolve();
48     const n = Math.max(...Object.values(kf).map(v => v.length)), last = styleAt(kf, n - 1);
49     if (reduce) { setStyle(el, last); if (!o.keep) clearStyle(el, kf); return Promise.resolve(); }
50     const dur = o.duration == null ? 0.6 : o.duration, delay = o.delay || 0, ease = EASE[o.ease] || o.ease || EASE.out;
51     setStyle(el, styleAt(kf, 0)); // hold the first frame, including during the delay
52     let done;
53     try {
54       if (window.Motion && window.Motion.animate) {
55         const a = window.Motion.animate(el, kf, { duration: dur, delay, ease });
56         done = Promise.resolve(a.finished || a);
57       } else throw new Error("no motion");
58     } catch (e) {
59       const frames = Array.from({ length: n }, (_, i) => styleAt(kf, i));
60       const a = el.animate(frames, { duration: dur * 1000, delay: delay * 1000, easing: bez(ease), fill: "both" });
61       done = a.finished.then(() => { if (o.keep) { try { a.commitStyles(); } catch (_) { setStyle(el, last); } } a.cancel(); });
62     }
63     return done.then(() => { if (o.keep) setStyle(el, last); else clearStyle(el, kf); }, () => {});
64   }
65   const stagger = (els, kf, o = {}) => Promise.all(els.map((el, i) => play(el, kf, Object.assign({}, o, { delay: (o.delay || 0) + i * (o.step == null ? 0.07 : o.step) })))));
66
67   // ----- glowing particle sprite
68   const spriteCache = {};
69   function sprite(color) {
70     if (spriteCache[color]) return spriteCache[color];
71     const c = document.createElement("canvas"); c.width = c.height = 64;

```

```

72     const g = c.getContext("2d"), grad = g.createRadialGradient(32, 32, 0, 32, 32, 32);
73     grad.addColorStop(0, color); grad.addColorStop(0.25, color); grad.addColorStop(1, "rgba(0,0,0,0)");
74     g.fillStyle = grad; g.globalAlpha = 1; g.fillRect(0, 0, 64, 64);
75     return (spriteCache[color] = c);
76 }
77 function fitCanvas(cv) {
78     const r = cv.getBoundingClientRect(), d = Math.min(2, window.devicePixelRatio || 1);
79     cv.width = Math.max(1, Math.round(r.width * d)); cv.height = Math.max(1, Math.round(r.height * d));
80     return { w: cv.width, h: cv.height, d };
81 }
82
83 // ----- entrance
84 function heroEntrance() {
85     root.dataset.intro = "play";
86     const kinds = { up: { opacity: [0, 1], y: [30, 0] }, pop: { opacity: [0, 1], scale: [0.9, 1], y: [24, 0] }, fade: { opacity: [0, 1] }, right: { opacity: [0, 1], x: [60, 0], scale:
>     [0.96, 1] } };
87     const chars = $(".hl-ch"), items = $(".[data-anim]");
88     items.forEach((el, i) => play(el, kinds[el.dataset.anim] || kinds.up, { duration: 0.85, delay: 0.05 + i * 0.09 }));
89     stagger(chars, { opacity: [0, 1], y: [46, 0], filter: ["blur(12px)", "blur(0px)"], rotate: [4, 0] }, { duration: 0.8, delay: 0.12, step: 0.032 });
90     $(".[data-count]").forEach(el => countUp(el, +el.dataset.count, 0.9));
91     setTimeout(() => { root.dataset.intro = "done"; }, 2200);
92 }
93 function countUp(el, to, delay) {
94     if (reduce) { el.textContent = to; return; }
95     const t0 = performance.now() + delay * 1000;
96     const tick = now => { const k = clamp((now - t0) / 1200, 0, 1); el.textContent = Math.round(to * easeOut(k)); if (k < 1) requestAnimationFrame(tick); };
97     requestAnimationFrame(tick);
98 }
99
100 function runIntro() {
101     const intro = $("#intro");
102     if (!intro) { heroEntrance(); return Promise.resolve(); }
103     if (reduce) { intro.remove(); root.dataset.intro = "done"; return Promise.resolve(); }
104     return new Promise(resolve => {
105         const cv = $("#introCanvas"), ctx = cv.getContext("2d"), bar = $("#introBar"), pct = $("#introPct");
106         let { w, h, d } = fitCanvas(cv);
107         const N = 170, ps = Array.from({ length: N }, (_, i) => ({ a: Math.random() * 6.283, r0: (0.35 + Math.random() * 0.85) * Math.max(w, h) * 0.55, spin: (Math.random() < 0.5 ? -1 :
> 1) * (0.7 + Math.random() * 1.6), jit: Math.random(), col: i % 5 === 0 ? "#ff7a5c" : i % 3 === 0 ? "#ffffff" : "#8892ff", s: 0.5 + Math.random() * 1.1, delay: Math.random() * 0.3
> 5 }));
108         const TOTAL = 1.85, t0 = performance.now(); let finished = false, last = t0;
109         const onResize = () => ({ w, h, d } = fitCanvas(cv)); addEventListener("resize", onResize);
110         function finish() {
111             if (finished) return; finished = true; removeEventListener("resize", onResize); removeEventListener("keydown", onKey);
112             heroEntrance();
113             play(intro, { clipPath: ["circle(150% at 50% 50%)", "circle(0% at 50% 50%)"], opacity: [1, 1] }, { duration: 0.95, ease: "inOut", keep: true }).then(() => { intro.remove(); re
> solve(); });
114         }
115         function frame(now) {
116             if (finished) return;
117             const t = (now - t0) / 1000, dt = Math.min(0.05, (now - last) / 1000); last = now;
118             const p = clamp(t / TOTAL, 0, 1), cx = w / 2, cy = h * 0.47, ring = Math.min(w, h * 0.75) * 0.33;
119             ctx.globalCompositeOperation = "source-over"; ctx.fillStyle = "rgba(5,6,13,0.22)"; ctx.fillRect(0, 0, w, h);
120             ctx.globalCompositeOperation = "lighter";
121             for (const q of ps) {
122                 const k = easeInOut(clamp((t - q.delay) / 1.25, 0, 1)), burst = clamp((t - 1.5) / 0.4, 0, 1);
123                 q.a += q.spin * dt * (0.6 + k * 2.4);
124                 const r = lerp(q.r0, ring * (0.8 + q.jit * 0.5), k) * (1 + burst * burst * 2.2);
125                 const x = cx + Math.cos(q.a) * r, y = cy + Math.sin(q.a) * r * 0.66, sz = (4 + q.s * 7) * d * (1 - burst * 0.5);
126                 ctx.globalAlpha = (0.1 + 0.36 * k) * (1 - burst);
127                 ctx.drawImage(sprite(q.col), x - sz, y - sz, sz * 2, sz * 2);
128             }
129             ctx.globalAlpha = 1;
130             const shown = Math.round(easeOut(p) * 100); bar.style.width = shown + "%"; pct.textContent = shown + "%";
131             if (t >= TOTAL) finish(); else requestAnimationFrame(frame);
132         }
133         requestAnimationFrame(frame);
134         const skip = () => { if (!finished) finish(); };
135         function onKey(e) { if (e.key === "Escape" || e.key === "Enter" || e.key === " ") skip(); }
136         intro.addEventListener("click", skip); addEventListener("keydown", onKey);
137         const chip = $("#introSkip"); if (chip) chip.addEventListener("click", skip);
138     });

```

```

139 }
140
141 // ----- hero: interactive particle and wave mesh
142 function heroMesh(cv) {
143   if (!cv) return;
144   const ctx = cv.getContext("2d"), host = cv.parentElement;
145   let w = 0, h = 0, d = 1, ps = [], mouse = { x: -9999, y: -9999, on: false }, running = true, visible = true, colors = {}, colT = 0, pulses = [];
146   const readColors = () => { colors = { a: cssVar("--accent") || "#3b47f5", b: cssVar("--hot") || "#e8481f", line: cssVar("--accent") || "#3b47f5" }; };
147   function resize() {
148     ({ w, h, d } = fitCanvas(cv));
149     const n = clamp(Math.round(w * h) / (d * d * 24000)), 16, 52);
150     ps = Array.from({ length: n }, (_, i) => ({ x: Math.random() * w, y: Math.random() * h, vx: (Math.random() - 0.5) * 0.35 * d, vy: (Math.random() - 0.5) * 0.35 * d, s: 0.6 + Math
> .random() * 1.4, hot: i % 7 === 0 }));
151   }
152   resize(); readColors(); addEventListener("resize", resize);
153   host.addEventListener("pointermove", e => { const r = cv.getBoundingClientRect(); mouse.x = (e.clientX - r.left) * d; mouse.y = (e.clientY - r.top) * d; mouse.on = true; });
154   host.addEventListener("pointerleave", () => { mouse.on = false; mouse.x = mouse.y = -9999; });
155   host.addEventListener("pointerdown", e => { const r = cv.getBoundingClientRect(); pulses.push({ x: (e.clientX - r.left) * d, y: (e.clientY - r.top) * d, t: 0 }); });
156   new IntersectionObserver(es => { visible = es[0].isIntersecting; }, { threshold: 0 }).observe(cv);
157   let time = 0, last = performance.now();
158   function draw(now) {
159     requestAnimationFrame(draw);
160     if (!visible || document.hidden) { last = now; return; }
161     const dt = Math.min(0.05, (now - last) / 1000); last = now; time += dt;
162     if (now - colT > 1000) { readColors(); colT = now; }
163     ctx.clearRect(0, 0, w, h);
164     const link = 130 * d, mr = 170 * d;
165     // wave mesh floor: a perspective grid that ripples
166     ctx.lineWidth = d; ctx.strokeStyle = colors.line; const hz = h * 0.62;
167     for (let r = 0; r < 12; r++) {
168       const z = (r + ((time * 0.35) % 1)) / 12, y0 = hz + (h - hz) * z * z; ctx.globalAlpha = 0.03 + 0.09 * z; ctx.beginPath();
169       for (let x = 0; x <= w; x += 24 * d) { const yy = y0 + Math.sin(x * 0.008 + time * 1.4 + r) * 7 * z * d + (mouse.on ? Math.max(0, 1 - Math.hypot(x - mouse.x, y0 - mouse.y) / (
> 260 * d)) * -18 * d : 0); x ? ctx.lineTo(x, yy) : ctx.moveTo(x, yy); }
170     ctx.stroke();
171   }
172   for (let c = -9; c <= 9; c++) { ctx.globalAlpha = 0.05; ctx.beginPath(); ctx.moveTo(w / 2 + c * 26 * d, hz); ctx.lineTo(w / 2 + c * w * 0.11, h); ctx.stroke(); }
173   // particles and links
174   for (const p of ps) {
175     p.x += p.vx; p.y += p.vy;
176     if (mouse.on) { const dx = mouse.x - p.x, dy = mouse.y - p.y, dist = Math.hypot(dx, dy); if (dist < mr) { const f = (1 - dist / mr) * 0.9; p.x -= (dx / dist) * f * 1.6 * d * -
> 0.5; p.y -= (dy / dist) * f * 1.6 * d * -0.5; p.vx += (dx / dist) * 0.004 * d; p.vy += (dy / dist) * 0.004 * d; } }
177     p.vx *= 0.995; p.vy *= 0.995; p.vx += (Math.random() - 0.5) * 0.01 * d; p.vy += (Math.random() - 0.5) * 0.01 * d;
178     if (p.x < -20) p.x = w + 20; if (p.x > w + 20) p.x = -20; if (p.y < -20) p.y = h + 20; if (p.y > h + 20) p.y = -20;
179   }
180   ctx.lineWidth = d;
181   for (let i = 0; i < ps.length; i++) for (let j = i + 1; j < ps.length; j++) {
182     const a = ps[i], b = ps[j], dx = a.x - b.x, dy = a.y - b.y, dd = dx * dx + dy * dy;
183     if (dd < link * link) { ctx.globalAlpha = (1 - Math.sqrt(dd) / link) * 0.16; ctx.strokeStyle = colors.line; ctx.beginPath(); ctx.moveTo(a.x, a.y); ctx.lineTo(b.x, b.y); ctx.st
> roke(); }
184   }
185   if (mouse.on) for (const p of ps) { const dist = Math.hypot(p.x - mouse.x, p.y - mouse.y); if (dist < mr) { ctx.globalAlpha = (1 - dist / mr) * 0.6; ctx.strokeStyle = colors.b;
> ctx.beginPath(); ctx.moveTo(p.x, p.y); ctx.lineTo(mouse.x, mouse.y); ctx.stroke(); } }
186   ctx.globalAlpha = 1; ctx.globalCompositeOperation = "lighter";
187   for (const p of ps) { const s = (5 + p.s * 6) * d, c = p.hot ? colors.b : colors.a; ctx.globalAlpha = 0.32 + 0.22 * Math.sin(time * 2 + p.x); ctx.drawImage(sprite(c), p.x - s, p
> .y - s, s * 2, s * 2); }
188   for (let i = pulses.length - 1; i >= 0; i--) { const q = pulses[i]; q.t += dt; const k = q.t / 1.2; if (k >= 1) { pulses.splice(i, 1); continue; } ctx.globalAlpha = (1 - k) * 0.
> 7; ctx.strokeStyle = colors.b; ctx.lineWidth = 2 * d; ctx.beginPath(); ctx.arc(q.x, q.y, k * 260 * d, 0, 6.283); ctx.stroke(); }
189   ctx.globalCompositeOperation = "source-over"; ctx.globalAlpha = 1;
190 }
191 if (reduce) { visible = true; draw(performance.now()); running = false; } else requestAnimationFrame(draw);
192 }
193
194 // ----- typing effect for example prompts
195 function typer(input, holder, phrases, opts = {}) {
196   if (!input || !holder) return;
197   let i = opts.start !== null ? opts.start : Math.floor(Math.random() * phrases.length), n = 0, dir = 1, hold = 0, timer = 0, current = "";
198   const update = () => { holder.classList.toggle("off", document.activeElement === input || input.value !== ""); };
199   if (reduce) { holder.textContent = phrases[i]; update(); ["focus", "blur", "input"].forEach(ev => input.addEventListener(ev, update)); return; }
200   function step() {
201     current = phrases[i];
202     if (hold > 0) { hold -= 1; timer = setTimeout(step, 60); return; }

```

```

203     n += dir;
204     holder.textContent = current.slice(0, n);
205     if (dir === 1 && n >= current.length) { dir = -1; hold = 26; if (opts.onTyped) opts.onTyped(current); }
206     else if (dir === -1 && n <= 0) { dir = 1; i = (i + 1) % phrases.length; hold = 5; }
207     timer = setTimeout(step, dir === 1 ? 34 + Math.random() * 40 : 14);
208   }
209   ["focus", "blur", "input"].forEach(ev => input.addEventListener(ev, update)); update();
210   input.addEventListener("keydown", e => { if (e.key === "ArrowRight" && input.value === "" && current) { input.value = current; input.dispatchEvent(new Event("input")); e.preventDefault(); } });
211   step();
212   return () => clearTimeout(timer);
213 }
214 // Type text into an element once (used for the prompt shown while an animation is being made).
215 function typeInto(el, text, cps = 48) {
216   if (!el) return () => {};
217   if (reduce) { el.textContent = text; return () => {}; }
218   let k = 0, t = 0; el.textContent = "";
219   const tick = () => { k = Math.min(text.length, k + 1); el.textContent = text.slice(0, k); if (k < text.length) t = setTimeout(tick, 1000 / cps); };
220   tick(); return () => clearTimeout(t);
221 }
222
223 // ----- pointer effects
224 function pointerEffects() {
225   document.addEventListener("pointermove", e => {
226     const b = e.target.closest && e.target.closest(".btn, .glow-el");
227     if (b) { const r = b.getBoundingClientRect(); b.style.setProperty("--mx", e.clientX - r.left + "px"); b.style.setProperty("--my", e.clientY - r.top + "px"); }
228   }, { passive: true });
229   if (reduce) return;
230   for (const el of $$(".tilt")) {
231     const max = parseFloat(el.dataset.tilt || "6");
232     el.addEventListener("pointermove", e => { const r = el.getBoundingClientRect(), x = (e.clientX - r.left) / r.width - 0.5, y = (e.clientY - r.top) / r.height - 0.5; el.style.transform = "perspective(900px) rotateY(" + x * max * 2 + "deg) rotateX(" + -y * max * 2 + "deg) translateZ(0)"; el.style.setProperty("--tx", x * 100 + "%"); el.style.setProperty("--ty", y * 100 + "%"); });
233     el.addEventListener("pointerleave", () => { el.style.transform = ""; });
234   }
235   const hero = $("#hero");
236   if (hero) hero.addEventListener("pointermove", e => {
237     const r = hero.getBoundingClientRect(), x = (e.clientX - r.left) / r.width - 0.5, y = (e.clientY - r.top) / r.height - 0.5;
238     for (const c of $$(".data-depth", hero)) { const dp = parseFloat(c.dataset.depth); c.style.translate = x * dp * -34 + "px " + y * dp * -24 + "px"; }
239   });
240 }
241
242 // ----- scroll reveal and header
243 function setupReveal() {
244   const els = $(".rv");
245   if (reduce || !("IntersectionObserver" in window)) return;
246   els.forEach(el => { const sibs = el.parentElement ? [...el.parentElement.children].filter(c => c.classList.contains("rv")) : []; el.dataset.rvI = Math.max(0, sibs.indexOf(el)); });
247   ;
248   root.classList.add("rv-on");
249   const io = new IntersectionObserver(entries => {
250     for (const en of entries) {
251       if (!en.isIntersecting) continue;
252       const el = en.target; io.unobserve(el);
253       play(el, { opacity: [0, 1], y: [34, 0] }, { duration: 0.75, delay: Math.min(0.4, (+el.dataset.rvI || 0) * 0.09) });
254       el.classList.add("in");
255     }
256   }, { threshold: 0.1, rootMargin: "0px 0px -5% 0px" });
257   els.forEach(el => io.observe(el));
258 }
259 function headerScroll() {
260   const hd = $("#siteHeader"); if (!hd) return;
261   const on = () => hd.classList.toggle("scrolled", scrollY > 8); on(); addEventListener("scroll", on, { passive: true });
262 }
263 function splitHeadline() {
264   for (const el of $$(".data-split")) {
265     const text = el.textContent; el.setAttribute("aria-label", text); el.textContent = "";
266     let ci = 0;
267     text.split(" ").forEach((word, wi, arr) => {
268       const w = document.createElement("span"); w.className = "hl-word"; w.setAttribute("aria-hidden", "true");
269       [...word].forEach(ch => { const s = document.createElement("span"); s.className = "hl-ch"; s.style.setProperty("--ci", ci++); s.textContent = ch; w.append(s); });
270       el.append(w); if (wi < arr.length - 1) el.append(document.createTextNode(" "));
271     });
272   }
273 }

```

```

270     });
271   }
272 }
273
274 // ----- the generation orb (loading state) and burst
275 function orb(cv) {
276   const ctx = cv.getContext("2d"); let w = 0, h = 0, d = 1, run = false, raf = 0, t = 0, burstT = -1, last = 0;
277   const N = 110, ps = Array.from({ length: N }, (_, i) => ({ a: Math.random() * 6.283, rx: 0.16 + Math.random() * 0.34, ry: 0.1 + Math.random() * 0.2, sp: (0.25 + Math.random() * 0.
> 9) * (i % 2 ? 1 : -1), tilt: (Math.random() - 0.5) * 1.6, col: i % 6 === 0 ? "#ff7a5c" : i % 4 === 0 ? "ffffff" : "#8892ff", s: 0.5 + Math.random(), v: Math.random() * 6.283 }));
278   const size = () => ({ w, h, d } = fitCanvas(cv));
279   function frame(now) {
280     if (!run) return; raf = requestAnimationFrame(frame);
281     const dt = Math.min(0.05, (now - last) / 1000); last = now; t += dt;
282     if (burstT >= 0) burstT += dt;
283     ctx.clearRect(0, 0, w, h); ctx.globalCompositeOperation = "lighter";
284     const cx = w / 2, cy = h * 0.42, S = Math.min(w, h * 1.4), pulse = 1 + 0.06 * Math.sin(t * 3);
285     const core = ctx.createRadialGradient(cx, cy, 0, cx, cy, S * 0.3 * pulse); core.addColorStop(0, "rgba(136,146,255,0.55)"); core.addColorStop(0.5, "rgba(255,122,92,0.14)"); core.
> addColorStop(1, "rgba(0,0,0,0)");
286     ctx.globalAlpha = burstT >= 0 ? Math.max(0, 1 - burstT * 1.6) : 1; ctx.fillStyle = core; ctx.fillRect(0, 0, w, h);
287     for (const p of ps) {
288       p.a += p.sp * dt * (burstT >= 0 ? 2 : 1);
289       const k = burstT >= 0 ? 1 + easeOut(clamp(burstT / 0.7, 0, 1)) * 3.2 : 1;
290       const x0 = Math.cos(p.a) * p.rx * S * k, y0 = Math.sin(p.a) * p.ry * S * k, x = cx + x0 * Math.cos(p.tilt) - y0 * Math.sin(p.tilt), y = cy + x0 * Math.sin(p.tilt) + y0 * Math.
> cos(p.tilt);
291       const sz = (5 + p.s * 6) * d * (0.8 + 0.3 * Math.sin(t * 4 + p.v));
292       ctx.globalAlpha = (burstT >= 0 ? Math.max(0, 1 - burstT * 1.3) : 1) * (0.5 + 0.4 * Math.sin(t * 2 + p.v));
293       ctx.drawImage(sprite(p.col), x - sz, y - sz, sz * 2, sz * 2);
294     }
295     ctx.globalAlpha = 1; ctx.globalCompositeOperation = "source-over";
296   }
297   return {
298     start() { size(); t = 0; burstT = -1; if (run) return; run = true; last = performance.now(); if (reduce) { frame(last); run = false; return; } raf = requestAnimationFrame(frame)
> ; },
299     burst() { burstT = 0; },
300     stop() { run = false; cancelAnimationFrame(raf); ctx.clearRect(0, 0, cv.width, cv.height); },
301     resize: size
302   };
303 }
304
305 // ----- boot
306 function boot() {
307   splitHeadline();
308   heroMesh($("#heroCanvas"));
309   pointerEffects();
310   setupReveal();
311   headerScroll();
312   const ideas = window.PTM_IDEAS || [];
313   typer($("#heroPrompt"), $("#heroTyper"), ideas, { start: 1, onTyped: p => window.PTM_onHeroTyped && window.PTM_onHeroTyped(p) });
314   typer($("#prompt"), $("#promptTyper"), ideas);
315   return runIntro();
316 }
317 window.PTM_FX = { play, stagger, orb, typeInto, typer, reduce, boot };
318 }());

```

```

1  /*
2  * PromptToMotion player runtime (v2).
3  *
4  * This function never runs in the main page. app.js turns it into a string with
5  * PTM_runtime.toString() and injects it into the sandboxed preview <iframe>, next to the
6  * scene code (written by Claude, the mock generator, or a built-in example).
7  *
8  * It owns everything that touches the scene:
9  *   - the canvas and a deterministic clock (true FPS: the scene is only sampled on frame boundaries)
10  *   - play / pause / loop / seek / frame step, driven by postMessage from the page
11  *   - CAMERA moves (push, pull, pan, orbit, crane, handheld), applied to every scene
12  *   - LIGHTING looks (natural, cinematic, neon, golden hour, moonlit, studio): colour grade, bloom, vignette, rays
13  *   - MOTION BLUR by temporal supersampling (several sub-frames averaged per output frame)
14  *   - PHYSICS: an optional simulate() step is baked once, so springs, cloth and ropes scrub and export correctly
15  *   - the exporters: MP4 or WebM (MediaRecorder), GIF (own encoder), Lottie (image-sequence JSON)
16  *
17  * Scene contract (what generated code must provide):
18  *   function setup(W, H) { return state }           // optional, runs once
19  *   function simulate(state, dt, t, info) { ... }    // optional, fixed-step physics, mutates state
20  *   function draw(ctx, t, info) { ... }             // required, called once per frame (and per blur sub-frame)
21  *   info = { W, H, duration, progress, frame, frames, fps, state, camera, light }
22  * Helpers available to scenes: TAU, clamp, lerp, hash, noise, ease.*, PHYS.*, and a seeded Math.random.
23  */
24 function PTM_runtime(CONFIG) {
25   const post = (ev, data) => parent.postMessage(Object.assign({ ptm: true, sid: CONFIG.sid, ev }, data || {}), "");
26
27   // ----- helpers exposed to scenes -----
28   const TAU = Math.PI * 2;
29   const clamp = (v, a, b) => Math.min(b, Math.max(a, v));
30   const lerp = (a, b, k) => a + (b - a) * k;
31   const hash = n => { const s = Math.sin(n * 127.1 + 311.7) * 43758.5453; return s - Math.floor(s); };
32   const noise = x => { const i = Math.floor(x), f = x - i, u = f * f * (3 - 2 * f); return lerp(hash(i), hash(i + 1), u); };
33   const ease = {
34     in: x => x * x,
35     out: x => 1 - (1 - x) * (1 - x),
36     inOut: x => (x < 0.5 ? 2 * x * x : 1 - Math.pow(-2 * x + 2, 2)) / 2,
37     sine: x => 0.5 - 0.5 * Math.cos(Math.PI * x),
38     outBack: x => { const c1 = 1.70158, c3 = c1 + 1; return 1 + c3 * Math.pow(x - 1, 3) + c1 * Math.pow(x - 1, 2); },
39     outElastic: x => (x <= 0 ? 0 : x >= 1 ? 1 : Math.pow(2, -10 * x) * Math.sin((x * 10 - 0.75) * (TAU / 3)) + 1)
40   };
41   // Physics helpers for characters, cloth and ropes. Use them inside simulate(); they are plain functions.
42   const PHYS = {
43     // Damped spring. s = { x, v }. Moves s.x toward target; returns s.x.
44     spring(s, target, k, d, dt) { s.v += (-k * (s.x - target) - d * s.v) * dt; s.x += s.v * dt; return s.x; },
45     // Two-bone inverse kinematics (legs, arms). Returns the joint (knee/elbow) and the reachable end point.
46     ik2(ax, ay, tx, ty, l1, l2, dir) {
47       let dx = tx - ax, dy = ty - ay, d = Math.hypot(dx, dy) || 1e-3;
48       const max = l1 + l2 - 1e-3; if (d > max) { dx *= max / d; dy *= max / d; d = max; }
49       const a = (l1 * l1 - l2 * l2 + d * d) / (2 * d), h = Math.sqrt(Math.max(0, l1 * l1 - a * a)), s = dir || 1;
50       return { x: ax + (dx * a) / d - (s * dy * h) / d, y: ay + (dy * a) / d + (s * dx * h) / d, ex: ax + dx, ey: ay + dy };
51     },
52     // How high the hip must be above the ankle line so a foot placed `stride` ahead of or behind the hip is always reachable
53     // (the knee never locks out). bob is how far the hip rises and falls. Put the planted ankle at groundY - sole - 1.
54     hipHeight(l1, l2, stride, bob) { const L = (l1 + l2) * 0.985; return Math.sqrt(Math.max(0, L * L - stride * stride)) - (bob || 0); },
55     // Verlet chain (scarf, hair, tail, rope). Make one with makeChain, then call chain() every simulate() step.
56     makeChain(n, x, y, len) { return Array.from({ length: n }, (_, i) => ({ x: x - i * (len || 10), y: y, px: x - i * (len || 10), py: y, len: len || 10 })); },
57     chain(pts, ax, ay, dt, o) {
58       o = o || {}; const g = o.gravity == null ? 600 : o.gravity, damp = o.damping == null ? 0.985 : o.damping, wind = o.wind || 0;
59       pts[0].x = ax; pts[0].y = ay; pts[0].px = ax; pts[0].py = ay;
60       for (let i = 1; i < pts.length; i++) {
61         const p = pts[i], vx = (p.x - p.px) * damp, vy = (p.y - p.py) * damp;
62         p.px = p.x; p.py = p.y; p.x += vx + wind * dt * dt; p.y += vy + g * dt * dt;
63       }
64       for (let it = 0; it < (o.iters || 5); it++) for (let i = 1; i < pts.length; i++) {
65         const a = pts[i - 1], b = pts[i], dx = b.x - a.x, dy = b.y - a.y, d = Math.hypot(dx, dy) || 1e-3, k = (d - b.len) / d;
66         if (i === 1) { b.x -= dx * k; b.y -= dy * k; } else { a.x += dx * k * 0.5; a.y += dy * k * 0.5; b.x -= dx * k * 0.5; b.y -= dy * k * 0.5; }
67       }
68     }
69   };
70   Object.assign(window, { TAU, clamp, lerp, hash, noise, ease, PHYS });
71 }

```

```

72 // Math.random is reseeded before setup() and before every frame, so the same frame always looks the same.
73 const mulberry = a => () => { a |= 0; a = (a + 0x6d2b79f5) | 0; let t = Math.imul(a ^ (a >>> 15), 1 | a); t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t; return ((t ^ (t >>> 14)) >>
> 0) / 4294967296; };
74 const seed = n => { Math.random = mulberry(n * 9973 + 12345); };
75
76 // ----- state -----
77 const S = {
78   W: CONFIG.W, H: CONFIG.H, fps: CONFIG.fps, duration: CONFIG.duration, pixel: CONFIG.pixel || 1,
79   camMode: CONFIG.camera || "static", camK: CONFIG.camK == null ? 1 : CONFIG.camK, light: CONFIG.light || "none", blur: CONFIG.blur || 0,
80   loop: CONFIG.loop !== false, playing: CONFIG.autoplay !== false, t: 0, lastFrame: -1, dirty: true,
81   ready: false, broken: false, state: null, snaps: null, drawFn: null, setupFn: null, simFn: null, subCap: 99, ema: 0,
82   look: { x: 0, y: 0, tx: 0, ty: 0 }, // the viewer steering the camera by dragging the picture
83   smooth: CONFIG.smooth !== false // preview at the screen's refresh rate instead of only on frame boundaries
84 };
85 const frames = () => Math.max(1, Math.round(S.duration * S.fps));
86 const frameAt = t => Math.min(frames() - 1, Math.floor(t * S.fps + 1e-6));
87
88 const view = document.getElementById("view");
89 const dpr = Math.min(2, window.devicePixelRatio || 1);
90 const vscale = Math.min(dpr, 1280 / Math.max(S.W, S.H));
91 view.width = Math.round(S.W * vscale);
92 view.height = Math.round(S.H * vscale);
93 const vctx = view.getContext("2d");
94 let low = null;
95
96 function fail(e) {
97   if (S.broken) return;
98   S.broken = true; S.playing = false;
99   post("error", { message: String((e && e.message) || e || "Unknown error") });
100 }
101 window.addEventListener("error", e => fail(e.error || e.message));
102
103 function resetCtx(c) {
104   c.setTransform(1, 0, 0, 1, 0, 0);
105   c.globalAlpha = 1; c.globalCompositeOperation = "source-over";
106   c.shadowBlur = 0; c.shadowColor = "transparent"; c.filter = "none";
107 }
108
109 // ----- camera -----
110 // Returns { x, y, zoom, rot }: x and y are fractions of the frame. Every mode over-scans so edges never show.
111 function baseCamera(p) {
112   const k = S.camK, e = ease.inOut(p), a = p * TAU;
113   switch (S.camMode) {
114     case "push": return { x: 0, y: 0, zoom: 1 + 0.32 * k * e, rot: 0 };
115     case "pull": return { x: 0, y: 0, zoom: 1 + 0.32 * k * (1 - e), rot: 0 };
116     case "panl": return { x: (0.5 - p) * 0.14 * k, y: 0, zoom: 1 + 0.16 * k, rot: 0 };
117     case "panr": return { x: (p - 0.5) * 0.14 * k, y: 0, zoom: 1 + 0.16 * k, rot: 0 };
118     case "orbit": return { x: Math.sin(a) * 0.05 * k, y: -Math.cos(a) * 0.02 * k, zoom: 1.1 + 0.06 * k, rot: Math.sin(a) * 0.045 * k };
119     case "crane": return { x: 0, y: (0.5 - e) * 0.13 * k, zoom: 1 + 0.15 * k, rot: (e - 0.5) * 0.03 * k };
120     case "handheld": return { x: 0.007 * k * (Math.sin(a * 7 + 1) + 0.6 * Math.sin(a * 13 + 2)), y: 0.006 * k * (Math.sin(a * 9 + 3) + 0.5 * Math.sin(a * 17)), zoom: 1.05 + 0.02 * k
> , rot: 0.006 * k * Math.sin(a * 5 + 4) };
121     default: return { x: 0, y: 0, zoom: 1, rot: 0 };
122   }
123 }
124 // The viewer can also drag the picture to steer. The zoom grows with the drag so edges never show. Never part of exports.
125 function cameraAt(p) {
126   const c = baseCamera(p), L = S.look;
127   if (!exporting && (L.x || L.y)) { c.x += L.x; c.y += L.y; c.zoom *= 1 + (Math.abs(L.x) + Math.abs(L.y)) * 2.6; }
128   return c;
129 }
130 function applyCamera(c, cam) {
131   if (cam.zoom === 1 && !cam.x && !cam.y && !cam.rot) return;
132   c.translate(S.W / 2, S.H / 2); c.rotate(cam.rot); c.scale(cam.zoom, cam.zoom); c.translate(-S.W / 2 + cam.x * S.W, -S.H / 2 + cam.y * S.H);
133 }
134
135 // ----- lighting -----
136 // What scenes are told (info.light): dx, dy point TOWARD the light (screen coordinates, y down). Shadows fall the other way.
137 const LIGHTS = {
138   none: { style: "none", dx: -0.5, dy: -0.85, color: "#ffffff", color2: "#ffffff", ambient: 0.5, shadow: "rgba(0,0,0,0.25)" },
139   natural: { style: "natural", dx: -0.55, dy: -0.83, color: "#fffd6", color2: "#cfe3ff", ambient: 0.45, shadow: "rgba(20,30,60,0.28)" },
140   cinematic: { style: "cinematic", dx: 0.65, dy: -0.5, color: "#ffd2a0", color2: "#3c8fb0", ambient: 0.28, shadow: "rgba(6,24,40,0.45)" },

```

```

141   neon: { style: "neon", dx: 0, dy: -0.3, color: "#ff3df2", color2: "#22d3ee", ambient: 0.18, shadow: "rgba(0,0,0,0.55)" },
142   golden: { style: "golden", dx: 0.85, dy: -0.35, color: "#ffb45a", color2: "#ffd9a0", ambient: 0.36, shadow: "rgba(60,20,10,0.38)" },
143   moonlit: { style: "moonlit", dx: -0.4, dy: -0.9, color: "#a9c1ff", color2: "#6d86d6", ambient: 0.2, shadow: "rgba(0,0,20,0.5)" },
144   studio: { style: "studio", dx: -0.3, dy: -0.95, color: "#ffffff", color2: "#e8eefc", ambient: 0.6, shadow: "rgba(0,0,0,0.2)" }
145 };
146 // How the picture is graded after the scene is drawn. Blend modes: multiply darkens, screen and soft-light lift.
147 const LOOKS = {
148   none: null,
149   natural: { vig: 0.28, bloom: 0.16, tints: [ ["soft-light", "rgba(255,214,150,0.30)", "radial"] ], rays: 0.09 },
150   cinematic: { vig: 0.62, bloom: 0.26, tints: [ ["multiply", "rgba(24,70,96,0.24)", "flat"], ["soft-light", "rgba(255,150,70,0.34)", "radial"] ], bars: true, rays: 0.06 },
151   neon: { vig: 0.55, bloom: 0.8, tints: [ ["screen", "rgba(255,0,190,0.11)", "edgeL"], ["screen", "rgba(0,210,255,0.09)", "edgeR"] ], scan: 0.07 },
152   golden: { vig: 0.36, bloom: 0.34, tints: [ ["soft-light", "rgba(255,160,40,0.46)", "radial"] ], rays: 0.17 },
153   moonlit: { vig: 0.5, bloom: 0.22, tints: [ ["soft-light", "rgba(80,110,255,0.42)", "flat"], ["multiply", "rgba(130,150,225,0.16)", "flat"] ], rays: 0.05 },
154   studio: { vig: 0.14, bloom: 0.08, tints: [ ] }
155 };
156 let bl = null, bl2 = null, rays = null, scanPat = null;
157 function grade(c, cw, ch, t) {
158   const look = LOOKS[S.light]; if (!look) return;
159   const L = LIGHTS[S.light], lx = cw * (0.5 + L.dx * 0.42), ly = ch * (0.5 + L.dy * 0.42), diag = Math.hypot(cw, ch);
160   resetCtx(c);
161   if (look.bloom) { // bright-biased blur: shrink, square the colours (keeps highlights), stretch back and add
162     const w1 = Math.max(8, Math.round(cw / 8)), h1 = Math.max(8, Math.round(ch / 8));
163     if (!bl || bl.width !== w1 || bl.height !== h1) { bl = document.createElement("canvas"); bl.width = w1; bl.height = h1; bl2 = document.createElement("canvas"); bl2.width = Math.
> max(4, w1 >> 1); bl2.height = Math.max(4, h1 >> 1); }
164     const b = bl.getContext("2d"); resetCtx(b); b.imageSmoothingEnabled = true; b.drawImage(c.canvas, 0, 0, w1, h1);
165     b.globalCompositeOperation = "multiply"; b.drawImage(bl, 0, 0);
166     const b2 = bl2.getContext("2d"); resetCtx(b2); b2.drawImage(bl, 0, 0, bl2.width, bl2.height);
167     c.globalCompositeOperation = "screen"; c.imageSmoothingEnabled = true;
168     c.globalAlpha = look.bloom; c.drawImage(bl, 0, 0, cw, ch);
169     c.globalAlpha = look.bloom * 0.9; c.drawImage(bl2, 0, 0, cw, ch);
170     c.globalAlpha = 1;
171   }
172   for (const [mode, col, shape] of look.tints) {
173     c.globalCompositeOperation = mode;
174     let g;
175     if (shape === "radial") { g = c.createRadialGradient(lx, ly, 0, lx, ly, diag * 0.8); g.addColorStop(0, col); g.addColorStop(1, "rgba(0,0,0,0)"); }
176     else if (shape === "edgeL" || shape === "edgeR") { g = c.createLinearGradient(shape === "edgeL" ? 0 : cw, 0, cw / 2, 0); g.addColorStop(0, col); g.addColorStop(1, "rgba(0,0,0,0)"); }
>   }
177   else g = col;
178   c.fillStyle = g; c.fillRect(0, 0, cw, ch);
179 }
180 if (look.rays) { // soft light shafts from the light's side, drifting slowly (drawn small and layered, so the edges feather)
181   const rw = Math.max(8, Math.round(cw / 10)), rh = Math.max(8, Math.round(ch / 10)), k = rw / cw;
182   if (!rays || rays.width !== rw || rays.height !== rh) { rays = document.createElement("canvas"); rays.width = rw; rays.height = rh; }
183   const r = rays.getContext("2d"); resetCtx(r); r.clearRect(0, 0, rw, rh); r.globalCompositeOperation = "lighter";
184   const ox = lx * k, oy = ly * k, dg = diag * k;
185   for (let i = 0; i < 8; i++) {
186     const a = Math.atan2(-L.dy, -L.dx) + (i - 3.5) * 0.15 + Math.sin(t * 0.6 + i * 1.7) * 0.03, w = 0.03 + 0.012 * Math.sin(t * 0.9 + i), al = look.rays * (0.7 + 0.3 * Math.sin(t
> * 0.8 + i * 2)) / 3;
187     const g = r.createLinearGradient(ox, oy, ox + Math.cos(a) * dg, oy + Math.sin(a) * dg);
188     g.addColorStop(0, "rgba(255,240,210," + al + ")"); g.addColorStop(1, "rgba(255,240,210,0)");
189     r.fillStyle = g;
190     for (const f of [1, 0.62, 0.3]) { r.beginPath(); r.moveTo(ox, oy); r.lineTo(ox + Math.cos(a - w * f) * dg, oy + Math.sin(a - w * f) * dg); r.lineTo(ox + Math.cos(a + w * f) *
> dg, oy + Math.sin(a + w * f) * dg); r.closePath(); r.fill(); }
191   }
192   c.globalCompositeOperation = "screen"; c.imageSmoothingEnabled = true; c.drawImage(rays, 0, 0, cw, ch);
193 }
194 c.globalCompositeOperation = "source-over";
195 if (look.scan) {
196   if (!scanPat) { const s = document.createElement("canvas"); s.width = 4; s.height = 3; const sc = s.getContext("2d"); sc.fillStyle = "rgba(0,0,0,1)"; sc.fillRect(0, 0, 4, 1); sc
> anPat = c.createPattern(s, "repeat"); }
197   c.globalAlpha = look.scan; c.fillRect(0, 0, cw, ch); c.globalAlpha = 1;
198 }
199 if (look.vig) {
200   const g = c.createRadialGradient(cw / 2, ch / 2, Math.min(cw, ch) * 0.32, cw / 2, ch / 2, diag * 0.6);
201   g.addColorStop(0, "rgba(0,0,0,0)"); g.addColorStop(1, "rgba(0,0,0," + look.vig + ")");
202   c.fillStyle = g; c.fillRect(0, 0, cw, ch);
203 }
204 if (look.bars && cw / ch > 1.5) { const bar = Math.max(0, (ch - cw / 2.2) / 2); c.fillStyle = "#000"; c.fillRect(0, 0, cw, bar); c.fillRect(0, ch - bar, cw, bar); }
205 }
206

```

```

207 // ----- physics (baked once, so scrubbing and export are exact) -----
208 const cloneState = st => (typeof structuredClone === "function" ? structuredClone(st) : JSON.parse(JSON.stringify(st)));
209 // One whole loop is simulated first and thrown away (pre-roll), then the loop that is kept starts from that settled state.
210 // Cloth, hair and tails therefore arrive at frame 0 already moving, and the end of the loop flows into the start without a pop.
211 function bake() {
212   S.snaps = null;
213   if (typeof S.simFn !== "function") return;
214   seed(0);
215   const st = typeof S.setupFn === "function" ? S.setupFn(S.W, S.H) : {}, N = frames(), SUB = 4, dt = 1 / (S.fps * SUB), snaps = [];
216   S.state = st;
217   for (let pass = 0; pass < 2; pass++) {
218     for (let f = 0; f < N; f++) {
219       for (let s = 0; s < SUB; s++) { seed(f * SUB + s + 7); S.simFn(st, dt, f / S.fps + s * dt, { W: S.W, H: S.H, fps: S.fps, duration: S.duration, frame: f }); }
220       if (pass === 1) snaps.push(cloneState(st));
221     }
222   }
223   S.snaps = snaps;
224 }
225 // Numbers blend, arrays and objects blend member by member. A jump bigger than 30% of the frame is a respawn, not motion: no blending.
226 function lerpState(a, b, k, jump) {
227   if (typeof a === "number" && typeof b === "number") return Math.abs(b - a) > jump ? a : a + (b - a) * k;
228   if (Array.isArray(a) && Array.isArray(b) && a.length === b.length) return a.map((v, i) => lerpState(v, b[i], k, jump));
229   if (a && b && typeof a === "object" && typeof b === "object") { const o = {}; for (const key in a) o[key] = key in b ? lerpState(a[key], b[key], k, jump) : a[key]; return o; }
230   return a;
231 }
232 // State is a continuous function of time: exact on frame boundaries (exports, stepping), blended in between (smooth preview, motion blur).
233 function stateAtT(t) {
234   if (!S.snaps) return S.state;
235   const x = clamp(t * S.fps, 0, S.snaps.length - 1), f0 = Math.floor(x), k = x - f0;
236   if (k < 1e-6 || f0 >= S.snaps.length - 1) return S.snaps[f0];
237   return lerpState(S.snaps[f0], S.snaps[f0 + 1], k, Math.max(S.W, S.H) * 0.3);
238 }
239
240 // ----- drawing one picture -----
241 // paintScene: camera + scene into a context (used for the picture itself and for every motion-blur sub-frame).
242 function paintScene(c, cw, ch, t, frame) {
243   resetCtx(c);
244   c.fillStyle = "#000"; c.fillRect(0, 0, cw, ch);
245   let target = c, scale = cw / S.W;
246   if (S.pixel > 1) {
247     const lw = Math.max(1, Math.round(S.W / S.pixel)), lh = Math.max(1, Math.round(S.H / S.pixel));
248     if (!low || low.width !== lw || low.height !== lh) { low = document.createElement("canvas"); low.width = lw; low.height = lh; }
249     target = low.getContext("2d");
250     resetCtx(target); target.fillStyle = "#000"; target.fillRect(0, 0, lw, lh);
251     scale = lw / S.W;
252   }
253   target.save();
254   target.setTransform(scale, 0, 0, scale, 0, 0);
255   const p = t / S.duration, cam = cameraAt(p);
256   applyCamera(target, cam);
257   seed(frame + 1);
258   try {
259     S.drawFn(target, t, { W: S.W, H: S.H, duration: S.duration, progress: p, frame, frames: frames(), fps: S.fps, state: stateAtT(t), camera: cam, light: LIGHTS[S.light] || LIGHTS.n
260     one });
261   } catch (e) { fail(e); }
262   target.restore();
263   if (S.pixel > 1) { resetCtx(c); c.imageSmoothingEnabled = false; c.drawImage(low, 0, 0, cw, ch); c.imageSmoothingEnabled = true; }
264 }
265
266 // renderAt: the finished picture at time t. subs > 1 averages that many sub-frames across a 180-degree shutter.
267 let bufs = null;
268 function renderAt(c, cw, ch, t, subs) {
269   const n = Math.max(1, subs | 0), frame = frameAt(t);
270   if (n === 1) paintScene(c, cw, ch, t, frame);
271   else {
272     if (!bufs || bufs.w !== cw || bufs.h !== ch) { const mk = () => { const e = document.createElement("canvas"); e.width = cw; e.height = ch; return e; }; bufs = { w: cw, h: ch, su
273     b: mk(), acc: mk() }; }
274     const sc = bufs.sub.getContext("2d"), ac = bufs.acc.getContext("2d"), shutter = 0.5 / S.fps;
275     resetCtx(ac);
276     for (let k = 0; k < n; k++) {
277       const tk = clamp(t + (k / (n - 1) - 0.5) * shutter, 0, S.duration - 1e-6);

```

```

276     paintScene(sc, cw, ch, tk, frameAt(tk));
277     ac.globalAlpha = 1 / (k + 1); ac.drawImage(bufs.sub, 0, 0);
278   }
279   resetCtx(c); c.drawImage(bufs.acc, 0, 0);
280 }
281 grade(c, cw, ch, t);
282 }
283 const BLUR_PREVIEW = [1, 2, 3, 4], BLUR_EXPORT = [1, 4, 8, 12];
284 const previewSubs = () => Math.max(1, Math.min(BLUR_PREVIEW[S.blur] || 1, S.subCap));
285 const exportSubs = () => BLUR_EXPORT[S.blur] || 1;
286
287 // ----- playback clock -----
288 let lastTs = null, exporting = false, cancelExport = false;
289 function tick(ts) {
290   requestAnimationFrame(tick);
291   if (!S.ready || S.broken || exporting) { lastTs = ts; return; }
292   if (S.playing && lastTs !== null) {
293     S.t += Math.min(0.1, (ts - lastTs) / 1000);
294     if (S.t >= S.duration) {
295       if (S.loop) S.t %= S.duration;
296       else { S.t = S.duration; S.playing = false; S.dirty = true; post("ended"); }
297     }
298   }
299   const L = S.look, dtk = lastTs === null ? 0.016 : Math.min(0.05, (ts - lastTs) / 1000), kk = 1 - Math.exp(-dtk * (drag ? 14 : 5));
300   if (Math.abs(L.x - L.tx) > 1e-4 || Math.abs(L.y - L.ty) > 1e-4) { L.x += (L.tx - L.x) * kk; L.y += (L.ty - L.y) * kk; S.dirty = true; }
301   else if (L.x !== L.tx || L.y !== L.ty) { L.x = L.tx; L.y = L.ty; S.dirty = true; }
302   lastTs = ts;
303   const f = frameAt(S.t);
304   if (S.dirty || (S.smooth ? S.playing : f !== S.lastFrame)) {
305     S.lastFrame = f; S.dirty = false;
306     const t0 = performance.now(), subs = previewSubs();
307     renderAt(vctx, view.width, view.height, S.smooth ? S.t : f / S.fps, subs);
308     // If the preview can't keep up, lower the blur quality of the preview only (exports stay full quality). If even
309     // that is too slow for the screen's refresh rate, fall back to drawing on frame boundaries only.
310     const ms = performance.now() - t0; S.ema = S.ema ? S.ema * 0.9 + ms * 0.1 : ms;
311     const budget = S.smooth ? 1000 / 60 : 1000 / S.fps;
312     if (S.playing && subs > 1 && S.ema > budget * 1.15) { S.subCap = subs - 1; S.ema = 0; post("perf", { subs: S.subCap }); }
313     else if (S.playing && S.smooth && subs === 1 && S.ema > (1000 / S.fps) * 1.3 && S.t > 0.6) { S.smooth = false; S.ema = 0; post("perf", { smooth: false }); }
314     post("time", { t: S.t, frame: f, frames: frames(), playing: S.playing, duration: S.duration, fps: S.fps });
315   }
316 }
317
318 // Drag the picture to steer the camera (touch keeps vertical page scrolling). It springs back when released.
319 let drag = null;
320 view.style.cursor = "grab"; view.style.touchAction = "pan-y";
321 view.addEventListener("pointerdown", e => { drag = { x: e.clientX, y: e.clientY }; try { view.setPointerCapture(e.pointerId); } catch (_) {} view.style.cursor = "grabbing"; post("st
> eer", { on: true }); });
322 view.addEventListener("pointermove", e => {
323   if (!drag) return; const r = view.getBoundingClientRect();
324   S.look.tx = clamp(S.look.tx + ((e.clientX - drag.x) / r.width) * 0.16, -0.06, 0.06); S.look.ty = clamp(S.look.ty + ((e.clientY - drag.y) / r.height) * 0.16, -0.05, 0.05);
325   drag.x = e.clientX; drag.y = e.clientY;
326 });
327 const endDrag = () => { if (!drag) return; drag = null; S.look.tx = 0; S.look.ty = 0; view.style.cursor = "grab"; };
328 view.addEventListener("pointerup", endDrag); view.addEventListener("pointercancel", endDrag);
329
330 function snapshot(t, w) {
331   const c = document.createElement("canvas");
332   c.width = w; c.height = Math.round((w * S.H) / S.W);
333   renderAt(c.getContext("2d"), c.width, c.height, t, 1);
334   return c.toDataURL("image/jpeg", 0.78);
335 }
336
337 window.addEventListener("message", e => {
338   const m = e.data;
339   if (!m || !m.ptmCmd) return;
340   switch (m.ptmCmd) {
341     case "play": if (S.t >= S.duration) S.t = 0; S.playing = true; S.dirty = true; break;
342     case "pause": S.playing = false; S.dirty = true; break;
343     case "seek": S.t = clamp(+m.t || 0, 0, S.duration); S.dirty = true; break;
344     case "step": S.playing = false; S.t = clamp(S.lastFrame + (m.n | 0), 0, frames() - 1) / S.fps; S.dirty = true; break;
345     case "loop": S.loop = !!m.on; break;

```

```

346     case "config": {
347         let rebake = false;
348         if (m.fps && m.fps !== S.fps) { S.fps = m.fps; rebake = true; }
349         if (m.duration && m.duration !== S.duration) { S.duration = m.duration; rebake = true; }
350         if (m.pixel !== null) S.pixel = m.pixel;
351         if (m.camera !== null) S.camMode = m.camera;
352         if (m.camK !== null) S.camK = m.camK;
353         if (m.light !== null) S.light = m.light;
354         if (m.smooth !== null) { S.smooth = !!m.smooth; S.ema = 0; }
355         if (m.blur !== null) { S.blur = m.blur; S.subCap = 99; S.ema = 0; }
356         if (rebake && S.ready) { try { bake(); } catch (err) { fail(err); } }
357         S.t = Math.min(S.t, S.duration); S.dirty = true;
358         break;
359     }
360     case "thumb": if (S.ready && !S.broken) post("thumb", { url: snapshot(m.t !== null ? m.t : S.duration * 0.4, m.w || 320) }); break;
361     case "export": runExport(m.format); break;
362     case "cancel": cancelExport = true; break;
363 }
364 });
365
366 // ----- exporters -----
367 const progress = (p, label) => post("export-progress", { p, label });
368 const breathe = () => new Promise(r => setTimeout(r, 0));
369 const even = n => Math.max(2, Math.round(n / 2) * 2);
370 function checkCancel() { if (cancelExport) throw new Error("cancelled"); }
371
372 async function runExport(format) {
373     if (exporting || !S.ready || S.broken) return;
374     exporting = true; cancelExport = false;
375     try {
376         const out = format === "video" ? await exportVideo() : format === "gif" ? await exportGif() : await exportLottie();
377         post("export-done", out);
378     } catch (e) {
379         post("export-error", { message: String((e && e.message) || e), cancelled: cancelExport });
380     } finally {
381         exporting = false; S.dirty = true;
382     }
383 }
384
385 // Video is recorded in real time, so first measure how long one frame takes and pick a size and blur that keeps pace.
386 function pickVideoQuality() {
387     const budget = 1000 / S.fps, subs0 = exportSubs(), long0 = Math.max(S.W, S.H);
388     const options = [[1280, subs0], [1280, Math.ceil(subs0 / 2)], [960, Math.ceil(subs0 / 2)], [960, 1], [720, 1], [540, 1]];
389     let last = options[options.length - 1];
390     for (const [size, subs] of options) {
391         const w = even(S.W * (size / long0)), h = even(S.H * (size / long0));
392         const c = document.createElement("canvas"); c.width = w; c.height = h; const g = c.getContext("2d");
393         const t0 = performance.now(); for (let i = 0; i < 4; i++) renderAt(g, w, h, (i / 4) * S.duration, subs);
394         if ((performance.now() - t0) / 4 < budget * 0.7) return { size, subs, w, h };
395         last = [size, subs, w, h];
396     }
397     const size = last[0]; return { size, subs: last[1], w: even(S.W * (size / long0)), h: even(S.H * (size / long0)) };
398 }
399
400 // MP4 where the browser can record it (recent Chrome, Edge, Safari), otherwise WebM.
401 async function exportVideo() {
402     if (typeof MediaRecorder === "undefined") throw new Error("This browser can't record video.");
403     const types = ["video/mp4;codecs=avc1.42E01E", "video/mp4;codecs=avc1", "video/mp4", "video/webm;codecs=vp9", "video/webm;codecs=vp8", "video/webm"];
404     const mime = types.find(t => MediaRecorder.isTypeSupported(t));
405     if (!mime) throw new Error("This browser can't record video.");
406     progress(0, "Choosing a size that keeps your frame rate");
407     await breathe();
408     const q = pickVideoQuality();
409     const c = document.createElement("canvas"); c.width = q.w; c.height = q.h;
410     const g = c.getContext("2d");
411     const stream = c.captureStream(0), track = stream.getVideoTracks()[0];
412     const rec = new MediaRecorder(stream, { mimeType: mime, videoBitsPerSecond: q.size >= 1280 ? 10e6 : 6e6 });
413     const chunks = [];
414     rec.ondataavailable = e => { if (e.data && e.data.size) chunks.push(e.data); };
415     const stopped = new Promise(r => { rec.onstop = r; });
416     const N = frames();

```

```

417   renderAt(g, c.width, c.height, 0, q.subs);
418   rec.start(250);
419   const start = performance.now();
420   try {
421     for (let f = 0; f < N; f++) {
422       checkCancel();
423       renderAt(g, c.width, c.height, f / S.fps, q.subs);
424       if (track.requestFrame()) track.requestFrame();
425       progress((f + 1) / N, "Recording at " + q.w + " px" + (q.subs > 1 ? " with motion blur" : ""));
426       const wait = start + ((f + 1) * 1000) / S.fps - performance.now();
427       await new Promise(r => setTimeout(r, Math.max(0, wait)));
428     }
429   } finally {
430     rec.stop(); await stopped; track.stop();
431   }
432   const type = mime.split(";")[0];
433   return { blob: new Blob(chunks, { type }), ext: type === "video/mp4" ? "mp4" : "webm", mime: type, width: q.w, blurred: q.subs > 1 };
434 }
435
436 // GIF: one shared 256-colour palette (median cut over sampled frames), ordered dithering, LZW.
437 async function exportGif() {
438   const gfps = Math.min(S.fps, 24), N = Math.max(1, Math.round(S.duration * gfps)), subs = Math.min(exportSubs(), 4);
439   const sc = Math.min(1, 480 / Math.max(S.W, S.H)), w = Math.round(S.W * sc), h = Math.round(S.H * sc);
440   const c = document.createElement("canvas"); c.width = w; c.height = h;
441   const g = c.getContext("2d", { willReadFrequently: true });
442   const render = i => { renderAt(g, w, h, Math.min(S.duration - 1e-6, i / gfps), subs); return g.getImageData(0, 0, w, h).data; };
443
444   const hist = new Uint32Array(32768), every = Math.max(1, Math.floor(N / 24));
445   for (let i = 0; i < N; i += every) {
446     checkCancel();
447     const d = render(i);
448     for (let p = 0; p < d.length; p += 12) hist[((d[p] >> 3) << 10) | ((d[p + 1] >> 3) << 5) | (d[p + 2] >> 3)]++;
449     progress((0.25 * (i + 1)) / N, "Choosing colours"); await breathe();
450   }
451   const pal = medianCut(hist, 256);
452   const lut = new Int16Array(32768).fill(-1);
453   const nearest = k => {
454     let v = lut[k]; if (v >= 0) return v;
455     const r = ((k >> 10) & 31) * 8 + 4, gg = ((k >> 5) & 31) * 8 + 4, b = (k & 31) * 8 + 4;
456     let best = 1e9;
457     for (let i = 0; i < pal.length; i++) { const dr = pal[i][0] - r, dg = pal[i][1] - gg, db = pal[i][2] - b, d = dr * dr * 3 + dg * dg * 4 + db * db * 2; if (d < best) { best = d;
458     v = i; } }
459     lut[k] = v; return v;
460   };
461   const bayer = [0, 8, 2, 10, 12, 4, 14, 6, 3, 11, 1, 9, 15, 7, 13, 5];
462   const gif = gifWriter(w, h, pal), ix = new Uint8Array(w * h);
463   for (let i = 0; i < N; i++) {
464     checkCancel();
465     const d = render(i);
466     for (let y = 0, p = 0, q = 0; y < h; y++) for (let x = 0; x < w; x++, p += 4, q++) {
467       const o = (bayer[((y & 3) << 2) | (x & 3)] - 7.5) * 0.6;
468       const r = clamp(d[p] + o, 0, 255), gg = clamp(d[p + 1] + o, 0, 255), b = clamp(d[p + 2] + o, 0, 255);
469       ix[q] = nearest(((r >> 3) << 10) | ((gg >> 3) << 5) | (b >> 3));
470     }
471     const delay = Math.round(((i + 1) * 100) / gfps) - Math.round((i * 100) / gfps);
472     gif.frame(ix, delay);
473     progress(0.25 + (0.75 * (i + 1)) / N, "Encoding GIF"); await breathe();
474   }
475   return { blob: gif.finish(), ext: "gif", mime: "image/gif" };
476 }
477
478 function medianCut(hist, max) {
479   const ch = (k, a) => (a === 0 ? (k >> 10) & 31 : a === 1 ? (k >> 5) & 31 : k & 31);
480   const cols = [];
481   for (let k = 0; k < 32768; k++) if (hist[k]) cols.push(k);
482   const box = cs => {
483     const mn = [31, 31, 31], mx = [0, 0, 0]; let count = 0;
484     for (const k of cs) { count += hist[k]; for (let a = 0; a < 3; a++) { const v = ch(k, a); if (v < mn[a]) mn[a] = v; if (v > mx[a]) mx[a] = v; } }
485     const rg = [mx[0] - mn[0], mx[1] - mn[1], mx[2] - mn[2]], axis = rg.indexOf(Math.max(...rg));
486     return { cs, count, axis, range: rg[axis] };
487   };

```

```

487 let boxes = cols.length ? [box(cols)] : [];
488 while (boxes.length < max) {
489   let bi = -1, best = 0;
490   boxes.forEach((b, i) => { if (b.cs.length < 2) return; const s = b.count * (b.range + 1); if (s > best) { best = s; bi = i; } });
491   if (bi < 0) break;
492   const b = boxes[bi];
493   b.cs.sort((x, y) => ch(x, b.axis) - ch(y, b.axis));
494   let acc = 0, cut = 0;
495   for (; cut < b.cs.length - 2; cut++) { acc += hist[b.cs[cut]]; if (acc >= b.count / 2) break; }
496   boxes.splice(bi, 1, box(b.cs.slice(0, cut + 1)), box(b.cs.slice(cut + 1)));
497 }
498 const pal = boxes.map(b => {
499   let r = 0, g = 0, bl = 0;
500   for (const k of b.cs) { const n = hist[k]; r += (ch(k, 0) * 8 + 4) * n; g += (ch(k, 1) * 8 + 4) * n; bl += (ch(k, 2) * 8 + 4) * n; }
501   return [Math.round(r / b.count), Math.round(g / b.count), Math.round(bl / b.count)];
502 });
503 while (pal.length < 2) pal.push([0, 0, 0]);
504 return pal;
505 }
506
507 function gifWriter(w, h, pal) {
508   const parts = []; let buf = new Uint8Array(1 << 16), pos = 0;
509   const byte = b => { if (pos === buf.length) { parts.push(buf); buf = new Uint8Array(1 << 16); pos = 0; } buf[pos++] = b; };
510   const word = v => { byte(v & 255); byte(v >> 8 & 255); };
511   const str = s => { for (let i = 0; i < s.length; i++) byte(s.charCodeAt(i)); };
512   str("GIF89a"); word(w); word(h); byte(0xf7); byte(0); byte(0);
513   for (let i = 0; i < 256; i++) { const c = pal[i] || [0, 0, 0]; byte(c[0]); byte(c[1]); byte(c[2]); }
514   byte(0x21); byte(0xff); byte(11); str("NETSCAPE2.0"); byte(3); byte(1); word(0); byte(0);
515   function lzw(ix) {
516     const MIN = 8, CLEAR = 256, EOI = 257;
517     let size = MIN + 1, next = EOI + 1, cur = 0, bits = 0;
518     const dict = new Map(), block = [];
519     const flush = () => { byte(block.length); for (const b of block) byte(b); block.length = 0; };
520     const out = code => { cur |= code << bits; bits += size; while (bits >= 8) { block.push(cur & 255); cur >>= 8; bits -= 8; if (block.length === 255) flush(); } };
521     byte(MIN); out(CLEAR);
522     let prefix = ix[0];
523     for (let i = 1; i < ix.length; i++) {
524       const k = ix[i], key = prefix * 256 + k, code = dict.get(key);
525       if (code !== undefined) { prefix = code; continue; }
526       out(prefix);
527       if (next === 4096) { out(CLEAR); dict.clear(); next = EOI + 1; size = MIN + 1; }
528       else if (next >= 1 << size) size++; dict.set(key, next++); }
529     prefix = k;
530   }
531   out(prefix); out(EOI);
532   if (bits > 0) block.push(cur & 255);
533   if (block.length) flush();
534   byte(0);
535 }
536 return {
537   frame(ix, delay) {
538     byte(0x21); byte(0xf9); byte(4); byte(0); word(Math.max(2, delay)); byte(0); byte(0);
539     byte(0x2c); word(0); word(0); word(w); word(h); byte(0);
540     lzw(ix);
541   },
542   finish() { byte(0x3b); parts.push(buf.slice(0, pos)); return new Blob(parts, { type: "image/gif" }); }
543 };
544 }
545
546 // Lottie: a valid Lottie JSON with one embedded image per frame. It plays in any Lottie player,
547 // but it is raster, not vector (see docs/ARCHITECTURE.md for the vector path).
548 async function exportLottie() {
549   const lfps = Math.min(S.fps, 24), N = Math.max(1, Math.round(S.duration * lfps)), subs = Math.min(exportSubs(), 4);
550   const sc = Math.min(1, 480 / Math.max(S.W, S.H)), w = Math.round(S.W * sc), h = Math.round(S.H * sc);
551   const c = document.createElement("canvas"); c.width = w; c.height = h;
552   const g = c.getContext("2d");
553   const assets = [], layers = [];
554   const still = v => ({ a: 0, k: v });
555   for (let i = 0; i < N; i++) {
556     checkCancel();
557     renderAt(g, w, h, Math.min(S.duration - 1e-6, i / lfps), subs);

```

```
558     assets.push({ id: "frame_" + i, w, h, u: "", p: c.toDataURL("image/jpeg", 0.82), e: 1 });
559     layers.push({ ddd: 0, ind: i + 1, ty: 2, nm: "Frame " + (i + 1), refId: "frame_" + i, sr: 1,
560       ks: { o: still(100), r: still(0), p: still([0, 0, 0]), a: still([0, 0, 0]), s: still([100, 100, 100]) },
561       ao: 0, ip: i, op: i + 1, st: 0, bm: 0 });
562     progress((i + 1) / N, "Building Lottie frames");
563     if (i % 3 === 0) await breathe();
564   }
565   const json = { v: "5.7.4", fr: lfps, ip: 0, op: N, w, h, nm: CONFIG.title || "PromptToMotion", ddd: 0, assets, layers, markers: [] };
566   return { blob: new Blob([JSON.stringify(json)], { type: "application/json" }), ext: "json", mime: "application/json" };
567 }
568
569 // ----- start (called by the last script tag, after the scene code has run) -----
570 window.PTM_start = (drawFn, setupFn, simFn) => {
571   try {
572     if (typeof drawFn !== "function") throw new Error("The scene has no draw(ctx, t, info) function.");
573     S.drawFn = drawFn; S.setupFn = setupFn; S.simFn = simFn;
574     seed(0);
575     S.state = typeof setupFn === "function" ? setupFn(S.W, S.H) : null;
576     bake();
577     // Self-test: draw a few frames off screen so a scene that crashes mid-timeline is caught before it is shown.
578     const tc = document.createElement("canvas");
579     tc.width = Math.max(16, Math.round(S.W / 8)); tc.height = Math.max(16, Math.round(S.H / 8));
580     for (const k of [0, 0.3, 0.6, 0.97]) {
581       renderAt(tc.getContext("2d"), tc.width, tc.height, k * S.duration, 1);
582       if (S.broken) return;
583     }
584     S.ready = true; S.dirty = true;
585     post("ready", { frames: frames() });
586   } catch (e) { fail(e); }
587   requestAnimationFrame(tick);
588 };
589 }
```

```

1  /*
2  * PromptToMotion scenes.
3  *
4  * 1. EXAMPLES: hand-written scenes that follow the scene contract (see runtime.js).
5  * 2. PTM_mock(prompt, style): the offline "mock generator". It reads keywords from the prompt
6  *    and builds a real, playable scene from a library of layers. Use it to test the whole
7  *    app (player, export, history) without calling any AI.
8  */
9
10 // A scene is stored as source code. Writing the examples as real functions lets the
11 // browser syntax-check them; sceneSource() turns a function body into scene code.
12 function sceneSource(fn, prefix) {
13   const src = fn.toString();
14   return (prefix || "") + src.slice(src.indexOf("(") + 1, src.lastIndexOf(")")).replace(/\n/, "").replace(/\s+$/, "") + "\n";
15 }
16
17 // ----- examples
18 function exampleFireworks() {
19   // Fireworks over a city skyline. Every rocket's path is computed from time, so any frame can be drawn directly.
20   function setup(W, H) {
21     const blocks = [];
22     let x = 0, i = 0;
23     while (x < W + 60) {
24       const w = 34 + hash(i * 3.1) * 44, h = H * (0.12 + hash(i * 7.7) * 0.28), win = [];
25       for (let wy = 12; wy < h - 10; wy += 15) for (let wx = 7; wx < w - 9; wx += 11) if (hash(i * 13 + wx * 1.7 + wy * 0.31) < 0.35) win.push([wx, wy]);
26       blocks.push({ x, w, h, win });
27       x += w + 3; i++;
28     }
29     return { blocks };
30   }
31   function draw(ctx, t, { W, H, duration, state }) {
32     const sky = ctx.createLinearGradient(0, 0, 0, H);
33     sky.addColorStop(0, "#050817"); sky.addColorStop(1, "#231a4d");
34     ctx.fillStyle = sky; ctx.fillRect(0, 0, W, H);
35     for (let i = 0; i < 140; i++) {
36       ctx.fillStyle = "rgba(255,255,255," + (0.2 + 0.5 * Math.abs(Math.sin(t * 1.3 + i))) + ")";
37       ctx.fillRect(hash(i) * W, hash(i + 50) * H * 0.7, 1.6, 1.6);
38     }
39     const GAP = 0.45, LIFE = 2.9, count = Math.floor((duration - 1e-6) / GAP) + 1;
40     ctx.globalCompositeOperation = "lighter";
41     for (let k = 0; k < count; k++) {
42       for (const age of [t - k * GAP, t + duration - k * GAP]) {
43         if (age < 0 || age > LIFE) continue;
44         const x = W * (0.12 + hash(k * 1.9) * 0.76), top = H * (0.12 + hash(k * 4.3) * 0.28), hue = Math.floor(hash(k * 8.1) * 360);
45         const rise = 0.85;
46         if (age < rise) {
47           const y = H - (H - top) * ease.out(age / rise);
48           ctx.fillStyle = "hsl(" + hue + ",100%,85%)";
49           ctx.beginPath(); ctx.arc(x, y, 2.4, 0, TAU); ctx.fill();
50           for (let s = 1; s < 8; s++) { ctx.fillStyle = "hsla(" + hue + ",100%,70%," + (0.5 - s * 0.06) + ")"; ctx.fillRect(x - 1, y + s * 6, 2, 4); }
51           continue;
52         }
53         const a = age - rise, fade = Math.max(0, 1 - a / (LIFE - rise)), n = 70;
54         for (let i = 0; i < n; i++) {
55           const ang = (i / n) * TAU + hash(k + i) * 0.2, sp = (0.55 + hash(k * 3 + i) * 0.45) * Math.min(W, H) * 0.24;
56           const d = sp * (1 - Math.exp(-a * 2.6)), px = x + Math.cos(ang) * d, py = top + Math.sin(ang) * d + 34 * a * a;
57           ctx.fillStyle = "hsla(" + (hue + hash(i) * 40) + ",100%,62%," + fade + ")";
58           ctx.beginPath(); ctx.arc(px, py, 1 + 2.2 * fade, 0, TAU); ctx.fill();
59         }
60         ctx.fillStyle = "hsla(" + hue + ",100%,70%," + 0.12 * fade + ")";
61         ctx.fillRect(0, 0, W, H);
62       }
63     }
64     ctx.globalCompositeOperation = "source-over";
65     for (const b of state.blocks) {
66       ctx.fillStyle = "#04050d"; ctx.fillRect(b.x, H - b.h, b.w, b.h);
67       for (const [wx, wy] of b.win) {
68         ctx.fillStyle = "rgba(255,206,120," + (0.55 + 0.35 * Math.sin(t * 0.8 + wx * wy)) + ")";
69         ctx.fillRect(b.x + wx, H - b.h + wy, 4, 6);
70       }
71     }

```

```

72 }
73 }
74
75 function exampleSolar() {
76 // The solar system in a fake-3D perspective. Planets behind the sun are drawn first.
77 const PLANETS = [
78   { r: 3, d: 62, turns: 6, c: "#b8ada0" }, { r: 6, d: 88, turns: 4, c: "#e9c47c" },
79   { r: 6.5, d: 118, turns: 3, c: "#4c8fe3", moon: true }, { r: 5, d: 148, turns: 2, c: "#d9643c" },
80   { r: 16, d: 205, turns: 1, c: "#d9b58b" }, { r: 13, d: 262, turns: 1, c: "#eadaa8", ring: true }
81 ];
82 function draw(ctx, t, { W, H, progress }) {
83   ctx.fillStyle = "#03040b"; ctx.fillRect(0, 0, W, H);
84   for (let i = 0; i < 260; i++) {
85     ctx.fillStyle = "rgba(255,255,255," + (0.25 + 0.6 * Math.abs(Math.sin(progress * TAU * 2 + i))) + ")";
86     ctx.fillRect(hash(i) * W, hash(i + 99) * H, 1.4, 1.4);
87   }
88   const cx = W / 2, cy = H / 2, k = Math.min(W / 640, H / 380), tilt = 0.38 + 0.08 * Math.sin(progress * TAU);
89   const pos = PLANETS.map((p, i) => {
90     const a = progress * TAU * p.turns + i * 1.7;
91     return { p, x: cx + Math.cos(a) * p.d * k, y: cy + Math.sin(a) * p.d * k * tilt };
92   });
93   ctx.strokeStyle = "rgba(255,255,255,0.12)"; ctx.lineWidth = 1;
94   for (const p of PLANETS) { ctx.beginPath(); ctx.ellipse(cx, cy, p.d * k, p.d * k * tilt, 0, 0, TAU); ctx.stroke(); }
95   const planet = o => {
96     const r = o.p.r * k;
97     if (o.p.ring) { ctx.strokeStyle = "rgba(234,218,168,0.75)"; ctx.lineWidth = 2 * k; ctx.beginPath(); ctx.ellipse(o.x, o.y, r * 2.1, r * 0.7, -0.3, 0, TAU); ctx.stroke(); }
98     const g = ctx.createRadialGradient(o.x - r * 0.4, o.y - r * 0.4, r * 0.1, o.x, o.y, r);
99     g.addColorStop(0, "ffffff"); g.addColorStop(0.25, o.p.c); g.addColorStop(1, "#0b0b0b");
100    ctx.fillStyle = g; ctx.beginPath(); ctx.arc(o.x, o.y, r, 0, TAU); ctx.fill();
101    if (o.p.moon) { const ma = progress * TAU * 8; ctx.fillStyle = "#cfcfcf"; ctx.beginPath(); ctx.arc(o.x + Math.cos(ma) * r * 2.3, o.y + Math.sin(ma) * r * 2.3 * tilt, 1.8 * k, 0, TAU); ctx.fill(); }
102  };
103  pos.filter(o => o.y < cy).forEach(planet);
104  const sr = 30 * k, sun = ctx.createRadialGradient(cx, cy, 0, cx, cy, sr * 3);
105  sun.addColorStop(0, "ffff7d6"); sun.addColorStop(0.3, "ffbf3c"); sun.addColorStop(0.34, "rgba(255,140,40,0.5)"); sun.addColorStop(1, "rgba(255,120,20,0)");
106  ctx.fillStyle = sun; ctx.beginPath(); ctx.arc(cx, cy, sr * 3, 0, TAU); ctx.fill();
107  pos.filter(o => o.y >= cy).forEach(planet);
108 }
109 }
110
111 function exampleWarp() {
112 // Warp speed. Each star moves a whole number of times per loop, so the loop is seamless.
113 function draw(ctx, t, { W, H, progress }) {
114   ctx.fillStyle = "#01010a"; ctx.fillRect(0, 0, W, H);
115   const cx = W / 2 + Math.sin(progress * TAU) * W * 0.04, cy = H / 2 + Math.cos(progress * TAU) * H * 0.04, f = Math.max(W, H) * 0.5;
116   const glow = ctx.createRadialGradient(cx, cy, 0, cx, cy, f * 0.6);
117   glow.addColorStop(0, "rgba(90,120,255,0.25)"); glow.addColorStop(1, "rgba(0,0,0,0)");
118   ctx.fillStyle = glow; ctx.fillRect(0, 0, W, H);
119   for (let i = 0; i < 650; i++) {
120     const sx = (hash(i) - 0.5) * 2, sy = (hash(i + 1000) - 0.5) * 2, laps = 1 + Math.floor(hash(i + 2000) * 3);
121     let z = hash(i + 3000) - progress * laps; z -= Math.floor(z); z = 0.02 + z * 0.98;
122     const z2 = Math.min(1, z + 0.035);
123     const x1 = cx + (sx / z) * f, y1 = cy + (sy / z) * f, x2 = cx + (sx / z2) * f, y2 = cy + (sy / z2) * f, b = 1 - z;
124     ctx.strokeStyle = "hsla(" + (205 + b * 70) + ",90%," + (60 + b * 35) + "%," + b + ")";
125     ctx.lineWidth = 0.5 + b * 2.6;
126     ctx.beginPath(); ctx.moveTo(x2, y2); ctx.lineTo(x1, y1); ctx.stroke();
127   }
128 }
129 }
130
131 const PTM_EXAMPLES = [
132   { id: "ex-fireworks", title: "Fireworks over the city", note: "Rockets rise and burst over a skyline at night.", prompt: "Fireworks bursting over a city skyline at night",
133     params: { style: "vector", duration: 6, aspect: "16:9", fps: 30 }, code: sceneSource(exampleFireworks), engine: "example" },
134   { id: "ex-solar", title: "The solar system", note: "Planets orbit the sun in perspective; the loop is seamless.", prompt: "The planets orbiting the sun, seen at an angle",
135     params: { style: "3d", duration: 8, aspect: "16:9", fps: 30 }, code: sceneSource(exampleSolar), engine: "example" },
136   { id: "ex-warp", title: "Warp speed", note: "Flying through a star field at light speed.", prompt: "Flying through space at warp speed",
137     params: { style: "3d", duration: 5, aspect: "1:1", fps: 60 }, code: sceneSource(exampleWarp), engine: "example" }
138 ];
139
140 // ----- mock generator
141 // Keyword rules: layer name and the words that switch it on.

```

```

142 const PTM_KEYWORDS = [
143   ["stars", /\b(stars?|starry|night|space|galaxy|cosmos|universe|midnight|firework)/],
144   ["moon", /\b(moon|night|midnight|werewolf)/],
145   ["sun", /\b(sun|sunny|sunset|sunrise|dawn|dusk|summer|desert|beach)/],
146   ["clouds", /\b(clouds?|sky|rain|storm|windy|day)/],
147   ["mountains", /\b(mountains?|hills?|landscape|valley|dragon|volcano)/],
148   ["city", /\b(city|cities|buildings?|skyline|street|town|cyber|urban|skyscraper|rooftops?)/],
149   ["sea", /\b(sea|ocean|beach|waves?|boat|ship|sail|island)/],
150   ["boat", /\b(boat|ship|sail|pirate)/],
151   ["rain", /\b(rain|rainy|storm|stormy|drizzle|thunder)/],
152   ["lightning", /\b(storm|stormy|thunder|lightning)/],
153   ["snow", /\b(snow|snowy|winter|christmas|blizzard|frost)/],
154   ["fire", /\b(fire|flames?|campfire|burning|volcano|lava)/],
155   ["fish", /\b(fish|aquarium|underwater|coral|reef|shark)/],
156   ["bubbles", /\b(bubbles?|underwater|aquarium|fish|soda)/],
157   ["birds", /\b(birds?|flock|seagulls?)/],
158   ["balls", /\b(balls?|bounce|bouncing|bouncy|football|soccer|basketball)/],
159   ["planets", /\b(planets?|orbit|solar|saturn|jupiter)/],
160   ["rocket", /\b(rockets?|launch|spaceship|astronaut|lift ?off)/],
161   ["confetti", /\b(party|confetti|celebrat|birthday|congrat|win|winner)/],
162   ["hearts", /\b(love|hearts?|valentine|romantic|crush)/],
163   ["neongrid", /\b(neon|synthwave|retro|80s|cyber|vaporwave|arcade)/],
164   ["leaves", /\b(autumn|fall(ing)? leaves|leaf|leaves)/],
165   ["car", /\b(cars?|drive|driving|road|race|racing|truck)/],
166   ["dragon", /\b(dragons?|wyvern)\b/],
167   ["castle", /\b(castle|fortress|kingdom|knight|medieval|palace)\b/],
168   ["pines", /\b(forest|woods?|pines?|trees?|jungle)\b/],
169   ["ufo", /\b(ufo|ufos|saucer|aliens?|abductw*)\b/],
170   ["character", /\b(person|people|man|woman|girl|boy|kid|child|hero|character|runner|running|run|sprint|sprinting|jog|jogging|ninja|athlete|explorer|traveller|traveler|parkour)\b/],
171 ];
172
173 function PTM_analyze(prompt) {
174   const clean = String(prompt || "").replace(/\s+/g, " ").trim().slice(0, 600);
175   const low = clean.toLowerCase();
176   const layers = PTM_KEYWORDS.filter(([, re]) => re.test(low)).map(([name]) => name);
177   let sky = "day";
178   if (/\\b(space|galaxy|cosmos|universe|planet|orbit|rocket|astronaut|spaceship)/.test(low)) sky = "space";
179   else if (/\\b(underwater|aquarium|coral|reef|fish|deep sea)/.test(low)) sky = "underwater";
180   else if (/\\b(sunset|sunrise|dawn|dusk|evening|golden hour)/.test(low)) sky = "sunset";
181   else if (/\\b(night|midnight|dark|neon|cyber|moon|stars|firework|synthwave)/.test(low)) sky = "night";
182   if (sky === "night" && !layers.includes("stars")) layers.push("stars");
183   if (layers.includes("dragon")) layers.splice(layers.indexOf("fire"), layers.includes("fire") ? 1 : 0); // dragons breathe their own fire
184   const quoted = clean.match(/[""](?:\\.|[^\"]){1,40})[""]/);
185   const said = clean.match(/\\b(?:says?|saying|the words?|text|title|named|name is|spells?)\\s*[:\\-]?\\s*([A-Za-z0-9!?' ]{2,32})/i);
186   const text = quoted ? quoted[1].trim() : said ? said[1].trim() : "";
187   if (text) layers.push("text");
188   if (!layers.filter(l => !["stars", "clouds", "sun", "moon"].includes(l)).length) layers.push("abstract");
189   return { prompt: clean, sky, layers: [...new Set(layers)], text };
190 }
191
192 const PTM_SKIES = {
193   day: { sky: ["#5aa9e6", "#cfeafd"], ground: "#6fae5b", accent: "#ffcc4d", accent2: "#ff6b6b", ink: "#1b2230" },
194   sunset: { sky: ["#2b1b4d", "#ff8a5b"], ground: "#3b2a3f", accent: "#ffd166", accent2: "#ef476f", ink: "#1a1024" },
195   night: { sky: ["#070b1f", "#1f2a5c"], ground: "#141a2e", accent: "#8ecbff", accent2: "#ff5fa2", ink: "#05070f" },
196   space: { sky: ["#02030a", "#0d1033"], ground: "#1a1a26", accent: "#7cf0ff", accent2: "#ff7ad9", ink: "#02030a" },
197   underwater: { sky: ["#1b6fa8", "#052440"], ground: "#c9b27c", accent: "#ffd166", accent2: "#ff7b54", ink: "#031423" }
198 };
199
200 // Gait geometry for the running character. It lives out here so the scene (inside the preview) and the sound
201 // engine (in the page) agree on cadence. Everything follows from three rules:
202 // 1. the sole of a planted foot rests exactly on the ground line,
203 // 2. the legs are long enough to reach every planted foot, so the knee never locks out,
204 // 3. a planted foot moves back at exactly the speed the road scrolls, so it never skates.
205 // D is how far the road scrolls in one loop. With the foot speed = D / duration, one stride cycle covers 4 * stride.
206 function PTM_gait(W, H, dur) {
207   const U = Math.min(H * 0.36, W * 0.5), P = W * 1.6, L = U * 0.49;
208   const c0 = Math.max(2, Math.round(dur * 1.4)); // steps per second the eye finds natural
209   const g = Math.max(1, Math.round((U * 0.23 * 4 * c0) / P)); // laps of the near buildings the road covers per loop
210   const cycles = Math.max(c0, Math.ceil((g * P) / (4 * U * 0.28))); // never ask for a stride longer than 0.28 U
211   const D = g * P, stride = D / (4 * cycles), bob = U * 0.018, sole = U * 0.05;
212   const hv = Math.sqrt(Math.max(0, Math.pow(L * 0.985, 2) - stride * stride)) - bob; // hip height above the ankle line

```

```

213   return { U, P, L, cycles, D, stride, bob, sole, hv, roadSpeed: D / dur };
214 }
215
216 const PTM MOCK_VERSION = 3; // bump when the mock scenes change, so saved mock animations are rebuilt with the fix
217 function PTM_mock(prompt, style) {
218   const a = PTM_analyze(prompt);
219   const cfg = { style, sky: a.sky, layers: a.layers, text: a.text || (a.layers.includes("abstract") ? titleCase(a.prompt.split(" ").slice(0, 4).join(" ")) : ""), palette: PTM_SKIES[a.
> sky] };
220   const shown = a.layers.filter(l => l !== "abstract" && l !== "text");
221   return {
222     title: makeTitle(a.prompt),
223     note: shown.length ? "Mock preview built from: " + shown.join(", ") + (a.text ? ', plus the text "' + a.text + '"' : "") + ". : "Mock preview: no scene keywords found, so this is
> an abstract motion loop.",
224     code: sceneSource(mockSceneTemplate, "// Mock scene from PromptToMotion's offline generator (no AI).\nconst CFG = " + JSON.stringify(cfg) + ";\n" + PTM_gait.toString() + "\n"),
225     analysis: a, version: PTM MOCK_VERSION
226   };
227 }
228
229 // A short title from the prompt: drop the leading article and stop at a natural break.
230 function makeTitle(prompt) {
231   let words = String(prompt).replace(/["'"]/g, "").split(/[.,:!?]| while | with | and | as | at | in | on | from | through | under | over | when /i)[0].trim().split(/\s+/);
232   words = words.filter((w, i) => !(i === 0 && /^(a|an|the|some|my)$/i.test(w))).slice(0, 5);
233   return titleCase(words.join(" ")) || "Untitled";
234 }
235
236 function titleCase(s) { return String(s).replace(/[^w\s]!?-]/g, "").replace(/b\w/g, c => c.toUpperCase()).trim(); }
237
238 // The mock scene. CFG is prepended by PTM_mock(). Every layer is a pure function of progress p (0..1),
239 // so the loop is seamless and scrubbing works. The character's scarf and hair use simulate() (baked physics).
240 function mockSceneTemplate() {
241   const PAL = CFG.palette, STYLE = CFG.style, has = n => CFG.layers.includes(n);
242   const INKED = STYLE === "anime" || STYLE === "frames";
243   const RUN = has("character");
244   const wrap = v => v - Math.floor(v);
245   let DATA = null;
246   function ink(ctx, w) { if (INKED) { ctx.lineWidth = w || 2.5; ctx.strokeStyle = PAL.ink; ctx.stroke(); } }
247   function ball(ctx, x, y, r, col) {
248     if (STYLE === "3d") {
249       const g = ctx.createRadialGradient(x - r * 0.35, y - r * 0.35, r * 0.1, x, y, r);
250       g.addColorStop(0, "ffffff"); g.addColorStop(0.3, col); g.addColorStop(1, "rgba(0,0,0,0.9)");
251       ctx.fillStyle = g;
252     } else ctx.fillStyle = col;
253     ctx.beginPath(); ctx.arc(x, y, r, 0, TAU); ctx.fill(); ink(ctx, Math.max(1.5, r * 0.08));
254     if (STYLE === "anime") { ctx.fillStyle = "rgba(255,255,255,0.85)"; ctx.beginPath(); ctx.ellipse(x - r * 0.35, y - r * 0.4, r * 0.22, r * 0.12, -0.6, 0, TAU); ctx.fill(); }
255   }
256   function heart(ctx, x, y, s) {
257     ctx.beginPath(); ctx.moveTo(x, y + s * 0.3);
258     ctx.bezierCurveTo(x - s, y - s * 0.4, x - s * 0.5, y - s * 1.1, x, y - s * 0.5);
259     ctx.bezierCurveTo(x + s * 0.5, y - s * 1.1, x + s, y - s * 0.4, x, y + s * 0.3);
260   }
261   // Draw fn(offsetX) twice so a pattern of width P repeats without a seam.
262   // How a background layer scrolls. Layer speed is a fraction f of the road's speed. Every layer has to move a whole number of
263   // its own pattern periods per loop to repeat seamlessly, so the period is chosen as f * D / n and the pattern is stretched to fit.
264   function scroll(f, base, W, H, dur, p) {
265     if (!RUN) return { P: base, off: 0, s: 1 };
266     const D = PTM_gait(W, H, dur).D, ff = Math.max(f, (0.8 * W) / D); // a layer must move at least 0.8 of the frame per loop, or its pattern would have to be squashed flat
267     const n = Math.max(1, Math.round((ff * D) / base)), P = (ff * D) / n;
268     return { P, off: p * ff * D, s: P / base };
269   }
270   let VW = 960; // frame width, set at the start of draw()
271   // Draw a repeating pattern of period P so that it covers the whole frame, however short the period is.
272   function tile(P, off, fn) { const o = -((off % P) + P) % P, n = Math.ceil(VW / P) + 1; for (let i = 0; i <= n; i++) fn(o + i * P); }
273   function mix(a, b, k) { const pa = parseInt(a.slice(1), 16), pb = parseInt(b.slice(1), 16); const c = (s) => Math.round(((pa >> s) & 255) * (1 - k) + ((pb >> s) & 255) * k); return "r
> gb(" + c(16) + "," + c(8) + "," + c(0) + ")); }
274   function limb(ctx, pts, w, col, hi, L) {
275     ctx.lineCap = "round"; ctx.lineJoin = "round";
276     if (INKED) { ctx.strokeStyle = PAL.ink; ctx.lineWidth = w + 5; ctx.beginPath(); pts.forEach((q, i) => (i ? ctx.lineTo(q[0], q[1]) : ctx.moveTo(q[0], q[1]))); ctx.stroke(); }
277     ctx.strokeStyle = col; ctx.lineWidth = w; ctx.beginPath(); pts.forEach((q, i) => (i ? ctx.lineTo(q[0], q[1]) : ctx.moveTo(q[0], q[1]))); ctx.stroke();
278     if (hi && STYLE !== "vector" && STYLE !== "pixel") { // rim light on the side facing the light
279       ctx.strokeStyle = hi; ctx.globalAlpha = 0.55; ctx.lineWidth = Math.max(1.5, w * 0.28); ctx.beginPath();
280       pts.forEach((q, i) => (i ? ctx.lineTo(q[0] + L.dx * w * 0.28, q[1] + L.dy * w * 0.28) : ctx.moveTo(q[0] + L.dx * w * 0.28, q[1] + L.dy * w * 0.28))); ctx.stroke(); ctx.globalAlpha

```

```

> = 1;
281 }
282 }
283
284 // ---- runner: pose is a pure function of time, so physics (simulate) and drawing agree ----
285 const GAIT_CYCLES = dur => 0; // (kept for old saved scenes; the cadence now comes from PTM_gait)
286 function pose(t, W, H, dur) {
287   const G = PTM_gait(W, H, dur), U = G.U, p = t / dur, ground = H * 0.84, cx = W * (W > H ? 0.36 : 0.5);
288   const l1 = U * 0.245, l2 = U * 0.245, gait = p * TAU * G.cycles, stride = G.stride;
289   const groundAnkle = ground - G.sole - 1.5; // ankle height when the foot is planted (1.5 px so an outline never dips below)
290   const hipY = groundAnkle - G.hv - Math.cos(gait * 2) * G.bob, lean = 0.24 + Math.sin(gait * 2) * 0.02;
291   const foot = leg => {
292     const m = (((gait + leg * Math.PI) % TAU) + TAU) % TAU;
293     if (m < Math.PI) return { x: cx + stride * (1 - (2 * m) / Math.PI), y: groundAnkle, ang: 0, planted: true, lift: 0 }; // stance: foot slides back with the road
294     // swing: a Hermite curve from the back of the stride to the front. Its slope at both ends is the slope of the stance
295     // (the foot is carried back by the road at 2 * stride per half cycle), so the foot never stops or snaps at lift-off or landing.
296     const q = (m - Math.PI) / Math.PI, q2 = q * q, q3 = q2 * q, v = -2 * stride;
297     const hx = (2 * q3 - 3 * q2 + 1) * -stride + (q3 - 2 * q2 + q) * v + (-2 * q3 + 3 * q2) * stride + (q3 - q2) * v;
298     const lift = Math.pow(Math.sin(q * Math.PI), 0.75); // quick rise, soft landing
299     return { x: cx + hx, y: groundAnkle - lift * U * 0.2, ang: -0.35 * Math.sin(q * Math.PI), planted: false, lift };
300   };
301   const f0 = foot(0), f1 = foot(1), torso = U * 0.3;
302   const sx = cx + Math.sin(lean) * torso, sy = hipY - Math.cos(lean) * torso;
303   const hx = sx + Math.sin(lean * 0.6) * U * 0.14, hy = sy - Math.cos(lean * 0.6) * U * 0.14, r = U * 0.078;
304   const hand = leg => { const th = Math.sin(gait + leg * Math.PI + Math.PI) * 0.95; return { x: sx + Math.sin(th) * U * 0.3 + U * 0.07, y: sy + U * 0.3 - Math.abs(Math.sin(th)) * U *
> 0.09 }; };
305   return { U, ground, groundAnkle, sole: G.sole, stride, roadSpeed: G.roadSpeed, D: G.D, cx, l1, l2, hipY, lean, f: [f0, f1], sx, sy, hx, hy, r, hand: [hand(0), hand(1)], nx: sx - U *
> 0.01, ny: sy - U * 0.02, bx: hx - r * 0.75, by: hy - r * 0.35 };
306 }
307 // ---- dragon: flight path, wing flap and fire windows are closed-form; the tail is simulated ----
308 const FLAPS = dur => Math.max(3, Math.round(dur * 1.6));
309 function dragonPose(t, W, H, dur) {
310   const p = t / dur, U = Math.min(W, H) * 0.3, flap = Math.sin(p * TAU * FLAPS(dur));
311   return { p, U, flap, x: -W * 0.22 + W * 1.44 * p, y: H * 0.4 + Math.sin(p * TAU * 2) * H * 0.045 - flap * U * 0.05, bank: Math.cos(p * TAU * 2) * 0.07 };
312 }
313 const breathing = p => Math.sin(p * TAU * 2 + 0.9) > -0.25;
314 function dragonAnchors(q) { return { rx: q.x - q.U * 0.74, ry: q.y + q.U * 0.05, hx: q.x + q.U * 0.88, hy: q.y - q.U * 0.2 }; }
315 // ---- castle geometry (flag poles are where the cloth physics is pinned) ----
316 function castleGeom(W, H) {
317   const U = Math.min(W * 0.5, H * 0.55), cx = W * (W > H ? 0.68 : 0.5), base = H * 0.78;
318   const towers = [{ cx: cx - U * 0.3, w: U * 0.12, h: U * 0.5 }, { cx: cx + U * 0.3, w: U * 0.12, h: U * 0.56 }, { cx: cx, w: U * 0.15, h: U * 0.72 }];
319   return { U, cx, base, towers, poles: towers.map(t => ({ x: t.cx, y: base - t.h - t.w * 1.4 - U * 0.15 })); };
320 }
321 var simulate = (has("character") || has("dragon") || has("castle")) ? function (st, dt, t, info) {
322   if (st.scarf) {
323     const q = pose(t, info.W, info.H, info.duration);
324     const w = n => (TAU * Math.max(1, Math.round((n * info.duration) / TAU))) / info.duration; // gust speeds that fit a whole number of times into the loop
325     PHYS.chain(st.scarf, q.nx, q.ny, dt, { gravity: 520, wind: -2100 + Math.sin(t * w(7)) * 380, damping: 0.986, iters: 6 });
326     PHYS.chain(st.hair, q.bx, q.by, dt, { gravity: 260, wind: -1100 + Math.sin(t * w(9) + 1) * 250, damping: 0.98, iters: 4 });
327   }
328   if (st.tail) {
329     const q = dragonPose(t, info.W, info.H, info.duration), a = dragonAnchors(q), c = st.tail;
330     if (Math.hypot(c[0].x - a.rx, c[0].y - a.ry) > q.U * 2) for (let i = 0; i < c.length; i++) { c[i].x = c[i].px = a.rx - i * c[i].len; c[i].y = c[i].py = a.ry; } // the loop resta
> rts off screen
331     PHYS.chain(c, a.rx, a.ry, dt, { gravity: 260, wind: -1500 + Math.sin(t * (TAU * Math.max(1, Math.round((6 * info.duration) / TAU))) / info.duration) * 300, damping: 0.982, iters:
> 6 });
332   }
333   if (st.flags) { const g = castleGeom(info.W, info.H); st.flags.forEach((f, i) => PHYS.chain(f, g.poles[i].x, g.poles[i].y, dt, { gravity: 150, wind: 1400 + Math.sin(t * (TAU * Math.
> max(1, Math.round((5 * info.duration) / TAU))) / info.duration + i * 2) * 450, damping: 0.972, iters: 5 })); }
334 } : null;
335
336 const L = {
337   sky(ctx, p, W, H) {
338     const g = ctx.createLinearGradient(0, 0, 0, H);
339     g.addColorStop(0, PAL.sky[0]); g.addColorStop(1, PAL.sky[1]);
340     ctx.fillStyle = g; ctx.fillRect(0, 0, W, H);
341     if (CFG.sky === "underwater") {
342       ctx.save(); ctx.globalAlpha = 0.08; ctx.fillStyle = "#fff";
343       for (let i = 0; i < 5; i++) { const x = (i / 5) * W + Math.sin(p * TAU + i) * 40; ctx.beginPath(); ctx.moveTo(x, 0); ctx.lineTo(x + 60, 0); ctx.lineTo(x + 220, H); ctx.lineTo(x
> + 100, H); ctx.fill(); }
344       ctx.restore();

```

```

345 }
346 },
347 stars(ctx, p, W, H, S, cam) {
348   for (let i = 0; i < 190; i++) { const b = hash(i + 9); ctx.fillStyle = "rgba(255,255,255," + (0.15 + 0.75 * Math.abs(Math.sin(p * TAU * (1 + (i % 3)) + i))) + ")"; ctx.fillRect(((
> hash(i) * W + cam.x * W * 0.05) % W + W) % W, hash(i + 40) * H * 0.72, 1 + b * 1.4, 1 + b * 1.4); }
349 },
350 sun(ctx, p, W, H) {
351   const low = CFG.sky === "sunset", x = W * (low ? 0.5 : 0.78), y = H * (low ? 0.58 : 0.22), r = Math.min(W, H) * (low ? 0.16 : 0.09);
352   const g = ctx.createRadialGradient(x, y, r * 0.5, x, y, r * 3.4);
353   g.addColorStop(0, "rgba(255,220,120,0.6)"); g.addColorStop(1, "rgba(255,200,100,0)");
354   ctx.fillStyle = g; ctx.fillRect(0, 0, W, H);
355   const rr = r * (1 + 0.03 * Math.sin(p * TAU * 2)), sg = ctx.createRadialGradient(x - rr * 0.2, y - rr * 0.2, rr * 0.05, x, y, rr);
356   sg.addColorStop(0, "#ffffbe8"); sg.addColorStop(0.55, low ? "#ffb347" : PAL.accent); sg.addColorStop(1, low ? "#ff7a3a" : mix(PAL.accent, "#ff9a3a", 0.5));
357   ctx.fillStyle = sg; ctx.beginPath(); ctx.arc(x, y, rr, 0, TAU); ctx.fill(); ink(ctx, Math.max(1.5, rr * 0.06));
358 },
359 moon(ctx, p, W, H) {
360   const x = W * 0.8, y = H * 0.2, r = Math.min(W, H) * 0.07;
361   ctx.fillStyle = "rgba(220,230,255,0.15)"; ctx.beginPath(); ctx.arc(x, y, r * 2.2, 0, TAU); ctx.fill();
362   ball(ctx, x, y, r, "#f4f1e1");
363   ctx.fillStyle = PAL.sky[0]; ctx.beginPath(); ctx.arc(x + r * 0.45, y - r * 0.2, r * 0.9, 0, TAU); ctx.fill();
364 },
365 clouds(ctx, p, W, H, S, cam) {
366   for (let i = 0; i < 7; i++) {
367     const depth = 0.3 + hash(i + 2) * 0.7, x = wrap(hash(i) + p * (i % 2 ? 1 : 2)) * (W + 380) - 190 + cam.x * W * depth * 0.2, y = H * (0.08 + hash(i + 7) * 0.3), s = (26 + hash(i
> + 3) * 34) * (0.7 + depth * 0.5);
368     ctx.fillStyle = INKED ? (CFG.sky === "day" ? "#ffffff" : mix(PAL.sky[0], "#9aa6d6", 0.4)) : CFG.sky === "night" ? "rgba(160,170,210,0.22)" : "rgba(255,255,255," + (0.6 + depth *
> 0.3) + ")";
369     ctx.beginPath(); for (const [dx, dy, r] of [[0, 0, 1], [0.9, 0.2, 0.8], [-0.9, 0.25, 0.75], [0.3, -0.4, 0.8], [1.6, 0.3, 0.55]]) { ctx.moveTo(x + dx * s + r * s, y + dy * s); ct
> x.arc(x + dx * s, y + dy * s, r * s, 0, TAU); }
370     if (INKED) { ctx.lineWidth = 5; ctx.strokeStyle = PAL.ink; ctx.stroke(); }
371     ctx.fill();
372   }
373 },
374 mountains(ctx, p, W, H, S, cam, Lt, t, dur) {
375   [[0.6, "#8fb0c8", 0.5, 3], [0.7, "#5c7f9a", 0.58, 5], [0.8, "#3a5670", 0.66, 7]].forEach(([base, col, f, freq0], li) => {
376     const sc = RUN ? scroll(f, W * 1.5, W, H, dur, p) : { P: W * 1.5, off: 0, s: 1 };
377     const P = sc.P, off = sc.off, freq = RUN ? Math.max(1, Math.round(P / (W * 0.5))) : freq0, far = CFG.sky === "day" ? col : mix("#1c2a44", "#3e5a7a", li / 2);
378     const g = ctx.createLinearGradient(0, H * (base - 0.2), 0, H);
379     g.addColorStop(0, far); g.addColorStop(1, mix("#0b1424", "#2b4058", 0.4));
380     const ridge = x => H * base - (Math.sin((x / P) * TAU * freq) * 0.5 + Math.sin((x / P) * TAU * (freq * 2 + 1) + li) * 0.22 + 0.9) * H * 0.13;
381     ctx.fillStyle = g;
382     tile(P, off + cam.x * W * (0.2 + li * 0.25), o => {
383       ctx.beginPath(); ctx.moveTo(o, H);
384       for (let x = 0; x <= P + 8; x += 8) ctx.lineTo(o + x, ridge(x));
385       ctx.lineTo(o + P + 8, H); ctx.closePath(); ctx.fill();
386       ctx.strokeStyle = "rgba(255,255,255," + (0.16 + li * 0.05) + ")"; ctx.lineWidth = 2; ctx.beginPath();
387       for (let x = 0; x <= P + 8; x += 8) (x ? ctx.lineTo(o + x, ridge(x)) : ctx.moveTo(o + x, ridge(x))); ctx.stroke();
388     });
389     if (INKED) { ctx.strokeStyle = PAL.ink; ctx.lineWidth = 2; ctx.beginPath(); const sh = off + cam.x * W * (0.2 + li * 0.25); for (let x = 0; x <= W; x += 8) { const xx = ((x + s
> h) % P) + P) % P; (x ? ctx.lineTo(x, ridge(xx)) : ctx.moveTo(x, ridge(xx))); } ctx.stroke(); }
390     const haze = ctx.createLinearGradient(0, H * (base - 0.16), 0, H * (base + 0.06));
391     haze.addColorStop(0, "rgba(255,255,255,0)"); haze.addColorStop(1, "rgba(255,255,255," + (0.16 - li * 0.04) + ")");
392     ctx.fillStyle = haze; ctx.fillRect(0, H * (base - 0.16), W, H * 0.22);
393   });
394 },
395 city(ctx, p, W, H, S, cam, Lt, t, dur) {
396   const night = CFG.sky !== "day", P = DATA.P;
397   DATA.city.forEach((layer, li) => {
398     const depth = [0.3, 0.6, 1][li], sc = scroll([0.64, 0.78, 0.92][li], P, W, H, dur, p), base = H * (RUN ? 0.84 : 0.86) - H * 0.0;
399     const body = night ? mix("#0a0d1c", "#26305a", 0.55 - li * 0.25) : mix("#5b6a80", "#a7b4c8", 0.7 - li * 0.3);
400     tile(sc.P, sc.off - cam.x * W * depth * 0.5, oo => {
401       ctx.save(); ctx.translate(oo, 0); ctx.scale(sc.s, 1); const o = 0;
402       for (const b of layer) {
403         const x = o + b.x, y = base - b.h;
404         ctx.fillStyle = body; ctx.fillRect(x, y, b.w, b.h + 2);
405         if (li === 2) { ctx.fillStyle = "rgba(0,0,0,0.25)"; ctx.fillRect(x + b.w * (Lt.dx > 0 ? 0.72 : 0), y, b.w * 0.28, b.h); }
406         if (INKED) { ctx.lineWidth = 2; ctx.strokeStyle = PAL.ink; ctx.strokeRect(x, y, b.w, b.h + 2); }
407         if (b.tank) { ctx.fillStyle = mix("#3b4258", "#000000", 0.2); ctx.fillRect(x + b.w * 0.2, y - b.tank, b.w * 0.34, b.tank); ctx.fillRect(x + b.w * 0.26, y - b.tank - 6, b.w *
> 0.22, 6); }
408         if (b.ant) { ctx.fillStyle = "#333b52"; ctx.fillRect(x + b.w * 0.5 - 1, y - b.ant, 2, b.ant); if (night) { ctx.fillStyle = "rgba(255,70,70," + (0.4 + 0.6 * Math.abs(Math.sin
> (p * TAU * 8 + b.x))) + ")"; ctx.beginPath(); ctx.arc(x + b.w * 0.5, y - b.ant, 2.2, 0, TAU); ctx.fill(); } }

```

```

409     for (const [wx, wy, ph, on] of b.win) {
410         if (night) { if (on) { ctx.fillStyle = "rgba(255," + (190 + ph * 30) + ",110," + (0.5 + 0.4 * Math.sin(p * TAU * 2 + ph * 6)) + ")"; ctx.fillRect(x + wx, y + wy, 4, 6); }
411     }
412     else { ctx.fillStyle = "rgba(255,255,255," + (0.12 + 0.18 * Math.abs(Math.sin(p * TAU + ph * 6))) + ")"; ctx.fillRect(x + wx, y + wy, 4, 6); }
413     }
414     ctx.restore();
415     });
416     if (li < 2) { const g = ctx.createLinearGradient(0, base - H * 0.34, 0, base); g.addColorStop(0, "rgba(255,255,255,0)"); g.addColorStop(1, "rgba(" + (night ? "120,130,190" : "25
417     5,255,255") + "," + (0.16 - li * 0.06) + ")"); ctx.fillStyle = g; ctx.fillRect(0, base - H * 0.34, W, H * 0.34); }
418     });
419     },
420     ground(ctx, p, W, H, S, cam, Lt, t, dur) {
421         const y0 = H * 0.84, night = CFG.sky !== "day";
422         const g = ctx.createLinearGradient(0, y0, 0, H); g.addColorStop(0, night ? "#1a2038" : "#6f7686"); g.addColorStop(1, night ? "#0a0d1c" : "#3c4252");
423         ctx.fillStyle = g; ctx.fillRect(0, y0, W, H - y0);
424         ctx.fillStyle = "rgba(255,255,255," + (night ? 0.14 : 0.3) + ")"; ctx.fillRect(0, y0, W, 2);
425         // lane marks move at exactly the speed the planted foot moves, so the feet grip the road
426         const D = RUN ? PTM_gait(W, H, dur).D : W * 7.2, K = Math.max(2, Math.round(D / (W * 0.24))), sp = D / K;
427         tile(D, p * D, o => { for (let i = 0; i < K; i++) ctx.fillRect(o + i * sp, y0 + (H - y0) * 0.45, sp * 0.42, 4); });
428     },
429     character(ctx, p, W, H, S, cam, Lt, t, dur) {
430         const q = pose(t, W, H, dur), U = q.U, st = S || DATA.init;
431         const hi = Lt.color, body = STYLE === "pixel" ? PAL.accent2 : PAL.accent2, pants = mix("#22305a", "#0a1024", 0.2), skin = "#f0c39b", scarf = PAL.accent;
432         // long soft shadow, cast away from the light
433         const sh = ctx.createRadialGradient(q.cx - Lt.dx * U * 0.3, q.ground + U * 0.02, 1, q.cx - Lt.dx * U * 0.3, q.ground + U * 0.02, U * 0.55);
434         sh.addColorStop(0, Lt.shadow); sh.addColorStop(1, "rgba(0,0,0,0)");
435         ctx.save(); ctx.translate(0, q.ground); ctx.scale(1, 0.14); ctx.translate(0, -q.ground); ctx.fillStyle = sh; ctx.fillRect(q.cx - U, q.ground - U, U * 2, U * 2); ctx.restore();
436         // Contact shadows: a small dark oval under each shoe that fades as the foot lifts. They are what tells the eye the foot is on the floor.
437         const legsPre = [0, 1].map(i => PHYS.ik2(q.cx, q.hipY, q.f[i].x, q.f[i].y, q.l1, q.l2, -1));
438         for (let i = 0; i < 2; i++) {
439             const lift = clamp((q.groundAnkle - q.f[i].y) / (U * 0.2), 0, 1);
440             ctx.globalAlpha = (1 - lift) * 0.9; ctx.fillStyle = Lt.shadow; ctx.beginPath(); ctx.ellipse(legsPre[i].ex + U * 0.035, q.ground + U * 0.008, U * (0.1 - 0.04 * lift), U * 0.02, 0
441             , 0, TAU); ctx.fill();
442         }
443         ctx.globalAlpha = 1;
444         // Dust kicked up at every footfall. Each puff is born where the foot landed and is carried back by the road, so it is a pure function of time.
445         const G = PTM_gait(W, H, dur), omega = (TAU * G.cycles) / dur, dustCol = CFG.sky === "day" ? "214,200,170" : "150,165,200";
446         for (let i = 0; i < 2; i++) {
447             const m = (((t * omega) + i * Math.PI) % TAU) + TAU, age = m / omega, k = age / 0.55; // m = 0 is the instant the foot lands
448             if (k >= 1) continue;
449             for (let j = 0; j < 5; j++) {
450                 const r0 = hash(j * 3.7 + i * 11.3), px = q.cx + q.stride - G.roadSpeed * age - (r0 * 0.5 + j * 0.12) * U * 0.1, rise = U * (0.01 + 0.07 * k) * (0.5 + hash(j + 4.1)), rad = U
451                 * (0.014 + 0.04 * k) * (0.7 + r0 * 0.6);
452                 ctx.fillStyle = "rgba(" + dustCol + "," + (0.42 * (1 - k) * (1 - k)) + ")"; ctx.beginPath(); ctx.arc(px, q.ground - rise, rad, 0, TAU); ctx.fill();
453             }
454         }
455         // back arm and back leg first
456         const legs = legsPre, arms = [0, 1].map(i => PHYS.ik2(q.sx, q.sy, q.hand[i].x, q.hand[i].y, U * 0.19, U * 0.19, -1));
457         const leg = i => {
458             limb(ctx, [[q.cx, q.hipY], [legs[i].x, legs[i].y], [legs[i].ex, legs[i].ey]], U * 0.085, pants, hi, Lt);
459             // The shoe is drawn in a frame whose origin is the ankle. Its flat sole is at y = +sole, so with the ankle at
460             // groundAnkle the sole rests on the ground line. While the foot swings, the toe tips up a little.
461             const f = q.f[i], a = q.sole;
462             ctx.save(); ctx.translate(legs[i].ex, legs[i].ey); ctx.rotate(f.ang);
463             ctx.fillStyle = "#1b1d24"; ctx.beginPath();
464             ctx.moveTo(-U * 0.065, -U * 0.03); ctx.lineTo(U * 0.035, -U * 0.034); ctx.quadraticCurveTo(U * 0.115, -U * 0.02, U * 0.135, a - U * 0.014);
465             ctx.lineTo(U * 0.135, a); ctx.lineTo(-U * 0.07, a); ctx.lineTo(-U * 0.07, -U * 0.005); ctx.closePath(); ctx.fill(); ink(ctx, 2);
466             ctx.fillStyle = "rgba(255,255,255,0.22)"; ctx.fillRect(-U * 0.07, a - U * 0.012, U * 0.205, U * 0.012); // pale sole edge, so the contact line reads
467             ctx.restore();
468         };
469         const arm = i => limb(ctx, [[q.sx, q.sy], [arms[i].x, arms[i].y], [arms[i].ex, arms[i].ey]], U * 0.06, i ? skin : mix("#000000", body, 0.75), hi, Lt);
470         leg(1); arm(1);
471         // scarf tail streams from the physics chain
472         const chain = st.scarf;
473         ctx.lineCap = "round"; ctx.lineJoin = "round";
474         for (let i = chain.length - 1; i >= 1; i--) { const a = chain[i - 1], b = chain[i], w = U * (0.05 - 0.036 * (i / chain.length)); ctx.strokeStyle = i % 4 < 2 ? scarf : mix(scarf, "
475         #000000", 0.18); ctx.lineWidth = w * 1.5; ctx.beginPath(); ctx.moveTo(a.x, a.y); ctx.lineTo(b.x, b.y); ctx.stroke(); }
476         // torso
477         limb(ctx, [[q.cx, q.hipY], [q.sx, q.sy]], U * 0.15, body, hi, Lt);
478         ctx.strokeStyle = scarf; ctx.lineWidth = U * 0.07; ctx.lineCap = "round"; ctx.beginPath(); ctx.moveTo(q.sx - U * 0.03, q.sy - U * 0.005); ctx.lineTo(q.sx + U * 0.06, q.sy - U * 0.

```

```

> 02); ctx.stroke();
475 // head and hair
476 const hc = st.hair; ctx.strokeStyle = "#2a1a12"; ctx.lineCap = "round";
477 for (let i = hc.length - 1; i >= 1; i--) { ctx.lineWidth = q.r * (0.9 - 0.14 * i); ctx.beginPath(); ctx.moveTo(hc[i - 1].x, hc[i - 1].y); ctx.lineTo(hc[i].x, hc[i].y); ctx.stroke(
> ); }
478 ctx.fillStyle = skin; ctx.beginPath(); ctx.arc(q.hx, q.hy, q.r, 0, TAU); ctx.fill(); ink(ctx, 2.5);
479 ctx.fillStyle = "#2a1a12"; ctx.beginPath(); ctx.arc(q.hx - q.r * 0.1, q.hy - q.r * 0.15, q.r * 1.02, Math.PI * 0.95, Math.PI * 1.85); ctx.fill();
480 ctx.fillStyle = "#111"; ctx.beginPath(); ctx.arc(q.hx + q.r * 0.42, q.hy - q.r * 0.05, q.r * 0.11, 0, TAU); ctx.fill();
481 ctx.fillStyle = "rgba(255,255,255,0.8)"; ctx.beginPath(); ctx.arc(q.hx + q.r * 0.45, q.hy - q.r * 0.09, q.r * 0.04, 0, TAU); ctx.fill();
482 leg(0); arm(0);
483 },
484 pines(ctx, p, W, H, S, cam, Lt, t, dur) {
485   const night = CFG.sky !== "day", snowy = has("snow"), P = DATA.P;
486   const g = ctx.createLinearGradient(0, H * 0.78, 0, H); g.addColorStop(0, snowy ? (night ? "#8da0b8" : "#eef4fb") : (night ? "#0c1d1c" : "#3c6d44")); g.addColorStop(1, snowy ? (nig
> ht ? "#3a4860" : "#b9c8da") : (night ? "#050d12" : "#1e4029"));
487   DATA.pines.forEach((layer, li) => {
488     const depth = [0.35, 0.65, 1][li], sc = scroll([0.66, 0.8, 0.95][li], P, W, H, dur, p), base = H * (0.8 + li * 0.045), col = mix(night ? "#10302f" : "#4a8a5c", night ? "#04121a"
> : "#1d4a33", li / 2);
489     tile(sc.P, sc.off - cam.x * W * depth * 0.5, oo => {
490       ctx.save(); ctx.translate(oo, 0); ctx.scale(sc.s, 1); const o = 0;
491       for (const t of layer) {
492         const h = t.h, w = h * 0.4, sway = Math.sin(p * TAU * 2 + t.ph) * 0.028 * (1 + li * 0.5);
493         ctx.save(); ctx.translate(o + t.x, base); ctx.rotate(sway);
494         ctx.fillStyle = "#3a2a1c"; ctx.fillRect(-w * 0.05, -h * 0.16, w * 0.1, h * 0.16);
495         for (let k = 0; k < 4; k++) {
496           const y0 = -h * (0.12 + k * 0.24), tw = w * (1 - k * 0.2);
497           ctx.fillStyle = col; ctx.beginPath(); ctx.moveTo(-tw / 2, y0); ctx.lineTo(0, y0 - h * 0.4); ctx.lineTo(tw / 2, y0); ctx.closePath(); ctx.fill();
498           if (INKED) { ctx.lineWidth = 2; ctx.strokeStyle = PAL.ink; ctx.stroke(); }
499           if (snowy) { ctx.fillStyle = "rgba(240,247,255,0.92)"; ctx.beginPath(); ctx.moveTo(-tw * 0.22, y0 - h * 0.26); ctx.lineTo(0, y0 - h * 0.4); ctx.lineTo(tw * 0.22, y0 - h *
> 0.26); ctx.quadraticCurveTo(0, y0 - h * 0.2, -tw * 0.22, y0 - h * 0.26); ctx.fill(); }
500           else if (Lt.dx !== 0) { ctx.fillStyle = "rgba(255,240,200,0.1)"; ctx.beginPath(); ctx.moveTo(0, y0 - h * 0.4); ctx.lineTo(Lt.dx > 0 ? tw / 2 : -tw / 2, y0); ctx.lineTo(0,
> y0); ctx.closePath(); ctx.fill(); }
501         }
502         ctx.restore();
503       }
504       ctx.restore();
505     });
506     const mist = ctx.createLinearGradient(0, base - H * 0.18, 0, base); mist.addColorStop(0, "rgba(200,215,235,0)"); mist.addColorStop(1, "rgba(" + (night ? "110,140,180" : "235,245
> ,255") + "," + (0.2 - li * 0.05) + ")");
507     ctx.fillStyle = mist; ctx.fillRect(0, base - H * 0.18, W, H * 0.18);
508   });
509   ctx.fillStyle = g; ctx.fillRect(0, H * 0.86, W, H * 0.14);
510 },
511 castle(ctx, p, W, H, S, cam, Lt) {
512   const G = castleGeom(W, H), U = G.U, night = CFG.sky !== "day", stone = night ? "#4a5068" : "#9aa0b4", dark = night ? "#2a2f44" : "#6f768c", roof = PAL.accent2;
513   const hill = ctx.createLinearGradient(0, G.base - U * 0.1, 0, H); hill.addColorStop(0, night ? "#1c2b2a" : "#5f9c5a"); hill.addColorStop(1, night ? "#0a1414" : "#2d5a33");
514   ctx.fillStyle = hill; ctx.beginPath(); ctx.moveTo(0, H); ctx.lineTo(0, G.base + U * 0.22); ctx.quadraticCurveTo(G.cx, G.base - U * 0.16, W, G.base + U * 0.22); ctx.lineTo(W, H); c
> tx.closePath(); ctx.fill();
515   const wallTop = G.base - U * 0.34, wallL = G.cx - U * 0.3, wallW = U * 0.6;
516   const rect = (x, y, w, h, c) => { ctx.fillStyle = c; ctx.fillRect(x, y, w, h); if (INKED) { ctx.lineWidth = 2; ctx.strokeStyle = PAL.ink; ctx.strokeRect(x, y, w, h); } };
517   rect(wallL, wallTop, wallW, U * 0.36, stone);
518   for (let i = 0; i < 9; i++) rect(wallL + i * wallW / 9 + wallW / 60, wallTop - U * 0.035, wallW / 13, U * 0.04, stone);
519   ctx.fillStyle = "rgba(0,0,0,0.18)"; for (let r = 0; r < 4; r++) for (let c = 0; c < 10; c++) ctx.fillRect(wallL + c * wallW / 10 + (r % 2) * wallW / 20, wallTop + U * 0.04 + r * U
> * 0.08, wallW / 12, 1.5);
520   G.towers.forEach((t, i) => {
521     const x = t.cx - t.w / 2, top = G.base - t.h;
522     rect(x, top, t.w, t.h + U * 0.05, i === 2 ? dark : stone);
523     ctx.fillStyle = "rgba(0,0,0,0.22)"; ctx.fillRect(x + (Lt.dx > 0 ? t.w * 0.65 : 0), top, t.w * 0.35, t.h + U * 0.05);
524     ctx.fillStyle = roof; ctx.beginPath(); ctx.moveTo(x - t.w * 0.14, top); ctx.lineTo(t.cx, top - t.w * 1.4); ctx.lineTo(x + t.w * 1.14, top); ctx.closePath(); ctx.fill(); if (INKE
> D) { ctx.lineWidth = 2; ctx.strokeStyle = PAL.ink; ctx.stroke(); }
525     ctx.fillStyle = "#3a2e22"; ctx.fillRect(t.cx - 1.2, top - t.w * 1.4 - U * 0.15, 2.4, U * 0.15);
526     for (let k = 0; k < (i === 2 ? 3 : 2); k++) { const lit = night || CFG.sky === "sunset"; ctx.fillStyle = lit ? "rgba(255," + (190 + k * 20) + ",100," + (0.6 + 0.3 * Math.sin(p *
> TAU * 3 + i + k * 2)) + ")": "#2b3246"; ctx.beginPath(); ctx.ellipse(t.cx, top + U * 0.1 + k * U * 0.16, t.w * 0.14, t.w * 0.26, 0, 0, TAU); ctx.fill(); }
527   });
528   ctx.fillStyle = "#1a1410"; ctx.beginPath(); ctx.moveTo(G.cx - U * 0.06, G.base + U * 0.02); ctx.lineTo(G.cx - U * 0.06, G.base - U * 0.1); ctx.arc(G.cx, G.base - U * 0.1, U * 0.06
> , Math.PI, 0); ctx.lineTo(G.cx + U * 0.06, G.base + U * 0.02); ctx.fill();
529   ctx.fillStyle = "rgba(255,170,70," + (0.35 + 0.15 * Math.sin(p * TAU * 6)) + ")"; ctx.fillRect(G.cx - U * 0.04, G.base - U * 0.1, U * 0.08, U * 0.11);
530   // flags: cloth physics
531   if (S && S.flags) S.flags.forEach((f, i) => {
532     const fh = U * 0.075, cols = [PAL.accent, PAL.accent2, PAL.accent];
533     ctx.fillStyle = cols[i]; ctx.beginPath();

```

```

534     f.forEach((q, j) => { const w = fh * (1 - (j / f.length) * 0.55) / 2; j ? ctx.lineTo(q.x, q.y - w) : ctx.moveTo(q.x, q.y - w); });
535     for (let j = f.length - 1; j >= 0; j--) { const q = f[j], w = fh * (1 - (j / f.length) * 0.55) / 2; ctx.lineTo(q.x, q.y + w); }
536     ctx.closePath(); ctx.fill(); if (INKED) { ctx.lineWidth = 2; ctx.strokeStyle = PAL.ink; ctx.stroke(); }
537     ctx.strokeStyle = "rgba(255,255,255,0.45)"; ctx.lineWidth = 1.5; ctx.beginPath(); f.forEach((q, j) => (j ? ctx.lineTo(q.x, q.y) : ctx.moveTo(q.x, q.y))); ctx.stroke();
538   });
539 },
540 dragon(ctx, p, W, H, S, cam, Lt, t, dur) {
541   const q = dragonPose(t, W, H, dur), U = q.U, A = dragonAnchors(q), scale = "#2f7a4f", belly = "#d8c27a", dark = "#1a4a33", wingC = mix(PAL.accent2, "#000000", 0.25);
542   // fire breath: particles are born at the mouth, so each one's path is a pure function of time
543   ctx.globalCompositeOperation = "lighter";
544   for (let i = 0; i < 90; i++) {
545     const age = ((t * 2.4 + hash(i) * 1.0) % 1) * 0.95, tb = t - age, pb = tb / dur;
546     if (!breathing(pb)) continue;
547     const qb = dragonPose(tb, W, H, dur), ab = dragonAnchors(qb), sp = U * (1.8 + hash(i + 7) * 1.2), k = age / 0.95;
548     const fx = ab.hx + U * 0.12 + sp * age * 0.75, fy = ab.hy + U * 0.1 + (hash(i + 3) - 0.5) * U * 0.35 * k + k * k * U * 0.35, r = U * (0.07 + k * 0.26);
549     const g = ctx.createRadialGradient(fx, fy, 0, fx, fy, r); g.addColorStop(0, "rgba(255, " + Math.round(245 - k * 110) + ", " + Math.round(170 - k * 140) + ", " + (0.85 * (1 - k * 0.
550     9)) + ")"); g.addColorStop(1, "rgba(255,80,20,0)");
551     ctx.fillStyle = g; ctx.fillRect(fx - r, fy - r, r * 2, r * 2);
552   }
553   ctx.globalCompositeOperation = "source-over";
554   ctx.save(); ctx.translate(q.x, q.y); ctx.rotate(q.bank);
555   // tail from the physics chain (drawn in world space)
556   ctx.restore();
557   const tail = S.tail; ctx.lineCap = "round"; ctx.lineJoin = "round";
558   for (let i = tail.length - 1; i >= 1; i--) { const a = tail[i - 1], b = tail[i], w = U * (0.13 - 0.1 * (i / tail.length)); ctx.strokeStyle = i % 2 ? scale : dark; ctx.lineWidth =
559   w; ctx.beginPath(); ctx.moveTo(a.x, a.y); ctx.lineTo(b.x, b.y); ctx.stroke();
560   if (i % 2 === 0) { ctx.fillStyle = PAL.accent2; ctx.beginPath(); ctx.moveTo(b.x - w * 0.3, b.y - w * 0.4); ctx.lineTo(b.x + w * 0.1, b.y - w * 1.3); ctx.lineTo(b.x + w * 0.5, b.
561   y - w * 0.4); ctx.closePath(); ctx.fill(); } }
562   ctx.save(); ctx.translate(q.x, q.y); ctx.rotate(q.bank);
563   const wing = (dx, dy, bright) => { // a wing is a shoulder, a tip that flaps, finger bones and a membrane
564     const ang = -1.2 - q.flap * 0.9, sx = dx, sy = dy, tx = sx + Math.cos(ang) * U * 0.88, ty = sy + Math.sin(ang) * U * 0.88;
565     const mid = [0.35, 0.7].map(f => ({ x: sx + (tx - sx) * f - U * 0.18 * (1 - f) + Math.cos(ang + 1.5) * U * 0.18 * f, y: sy + (ty - sy) * f + Math.sin(ang + 1.5) * U * 0.22 * f }
566     ));
567     ctx.fillStyle = bright ? wingC : mix(PAL.accent2, "#000000", 0.5); ctx.beginPath(); ctx.moveTo(sx, sy); ctx.lineTo(tx, ty); ctx.quadraticCurveTo(mid[1].x, mid[1].y + U * 0.12, m
568     id[1].x - U * 0.05, mid[1].y + U * 0.3); ctx.quadraticCurveTo(mid[0].x, mid[0].y + U * 0.3, sx - U * 0.25, sy + U * 0.22); ctx.closePath(); ctx.fill(); if (INKED) { ctx.lineWidth
569     = 2.5; ctx.strokeStyle = PAL.ink; ctx.stroke(); }
570     ctx.strokeStyle = dark; ctx.lineWidth = U * 0.03; ctx.beginPath(); ctx.moveTo(sx, sy); ctx.lineTo(tx, ty); ctx.moveTo(sx, sy); ctx.lineTo(mid[0].x, mid[0].y + U * 0.28); ctx.mov
571     eTo(sx, sy); ctx.lineTo(mid[1].x, mid[1].y + U * 0.26); ctx.stroke();
572   };
573   wing(-U * 0.12, -U * 0.16, false);
574   const bg = ctx.createLinearGradient(0, -U * 0.3, 0, U * 0.3); bg.addColorStop(0, scale); bg.addColorStop(0.65, dark); bg.addColorStop(1, belly);
575   ctx.fillStyle = bg; ctx.beginPath(); ctx.ellipse(0, 0, U * 0.6, U * 0.24, 0, 0, TAU); ctx.fill(); ink(ctx, 3);
576   ctx.strokeStyle = "rgba(255,255,255,0.12)"; ctx.lineWidth = 1.5; for (let i = -4; i <= 4; i++) { ctx.beginPath(); ctx.arc(i * U * 0.11, -U * 0.04, U * 0.07, Math.PI, 0); ctx.strok
577   e(); }
578   ctx.strokeStyle = scale; ctx.lineWidth = U * 0.15; ctx.lineCap = "round"; ctx.beginPath(); ctx.moveTo(U * 0.42, -U * 0.04); ctx.quadraticCurveTo(U * 0.74, -U * 0.04, U * 0.82, -U
579   * 0.22 - q.flap * U * 0.02); ctx.stroke();
580   const hx = U * 0.88, hy = -U * 0.26 - q.flap * U * 0.02, open = breathing(q.p) ? 0.3 : 0.06;
581   ctx.fillStyle = scale; ctx.beginPath(); ctx.moveTo(hx - U * 0.1, hy - U * 0.08); ctx.lineTo(hx + U * 0.26, hy - U * 0.03 - open * U * 0.1); ctx.lineTo(hx + U * 0.27, hy + U * 0.0)
582   ; ctx.lineTo(hx - U * 0.06, hy + U * 0.1); ctx.closePath(); ctx.fill(); ink(ctx, 2.5);
583   ctx.fillStyle = "#7ald12"; ctx.beginPath(); ctx.moveTo(hx + U * 0.02, hy + U * 0.03); ctx.lineTo(hx + U * 0.25, hy + U * 0.02); ctx.lineTo(hx + U * 0.22, hy + U * 0.03 + open * U
584   * 0.3); ctx.lineTo(hx - U * 0.02, hy + U * 0.1); ctx.closePath(); ctx.fill();
585   ctx.fillStyle = PAL.accent2; ctx.beginPath(); ctx.moveTo(hx - U * 0.06, hy - U * 0.08); ctx.lineTo(hx - U * 0.22, hy - U * 0.2); ctx.lineTo(hx - U * 0.02, hy - U * 0.05); ctx.fill
586   ();
587   ctx.fillStyle = "#ffe066"; ctx.beginPath(); ctx.arc(hx + U * 0.05, hy - U * 0.03, U * 0.03, 0, TAU); ctx.fill(); ctx.fillStyle = "#111"; ctx.fillRect(hx + U * 0.05, hy - U * 0.055
588   , U * 0.012, U * 0.05);
589   wing(-U * 0.02, -U * 0.2, true);
590   ctx.restore();
591 },
592 ufo(ctx, p, W, H, S, cam, Lt) {
593   const a = p * TAU, U = Math.min(W, H) * 0.2, x = W * (0.5 + 0.14 * Math.sin(a)) + cam.x * W * 0.3, y = H * (0.3 + 0.03 * Math.sin(a * 2)), tilt = Math.cos(a) * 0.09, gy = H * 0.86
594   , top = y + U * 0.12;
595   ctx.globalCompositeOperation = "lighter";
596   const bw = h => U * (0.22 + 0.5 * ((h - top) / (gy - top))), beam = ctx.createLinearGradient(0, top, 0, gy);
597   beam.addColorStop(0, "rgba(170,255,210," + (0.4 + 0.1 * Math.sin(a * 8)) + ")"); beam.addColorStop(1, "rgba(120,220,255,0.1)");
598   ctx.fillStyle = beam; ctx.beginPath(); ctx.moveTo(x - bw(top), top); ctx.lineTo(x + bw(top), top); ctx.lineTo(x + bw(gy), gy); ctx.lineTo(x - bw(gy), gy); ctx.closePath(); ctx.fil
599   l();
600   for (let i = 0; i < 44; i++) { const k = wrap(hash(i) + p * 3), h = gy - k * (gy - top), w = bw(h) * 0.85; ctx.fillStyle = "rgba(220,255,240," + Math.sin(k * Math.PI) * 0.8 + ")";
601   ctx.beginPath(); ctx.arc(x + (hash(i + 5) - 0.5) * 2 * w + Math.sin(k * 14 + i) * 6, h, 1.4 + hash(i + 9) * 2, 0, TAU); ctx.fill(); }
602   const gg = ctx.createRadialGradient(x, gy, 2, x, gy, U * 0.9); gg.addColorStop(0, "rgba(160,255,210,0.45)"); gg.addColorStop(1, "rgba(160,255,210,0)"); ctx.save(); ctx.translate(0
603   , gy); ctx.scale(1, 0.22); ctx.translate(0, -gy); ctx.fillStyle = gg; ctx.fillRect(x - U, gy - U, U * 2, U * 2); ctx.restore();
604   ctx.globalCompositeOperation = "source-over";

```

```

588   ctx.save(); ctx.translate(x, y); ctx.rotate(tilt);
589   const hull = ctx.createLinearGradient(0, -U * 0.14, 0, U * 0.16); hull.addColorStop(0, "#e5ebf5"); hull.addColorStop(0.5, "#8b97ad"); hull.addColorStop(1, "#3a4358");
590   ctx.fillStyle = hull; ctx.beginPath(); ctx.ellipse(0, 0, U * 0.52, U * 0.14, 0, 0, TAU); ctx.fill(); ink(ctx, 2.5);
591   const dome = ctx.createRadialGradient(-U * 0.06, -U * 0.22, 2, 0, -U * 0.14, U * 0.22); dome.addColorStop(0, "rgba(220,255,255,0.95)"); dome.addColorStop(1, "rgba(90,200,220,0.55)");
592   ctx.fillStyle = dome; ctx.beginPath(); ctx.ellipse(0, -U * 0.08, U * 0.22, U * 0.17, 0, Math.PI, 0); ctx.fill(); ink(ctx, 2);
593   for (let i = 0; i < 9; i++) { const th = (i / 8 - 0.5) * 2.4, lx = Math.sin(th) * U * 0.42, ly = Math.cos(th) * U * 0.05 + U * 0.02, on = Math.max(0, Math.sin(a * 6 - i * 0.8)); c
>   tx.fillStyle = "rgba(255, " + Math.round(200 + 55 * on) + ", " + Math.round(80 + 120 * on) + ", " + (0.5 + 0.5 * on) + ")"; ctx.beginPath(); ctx.arc(lx, ly, U * 0.022, 0, TAU); ctx.f
>   ill(); }
594   ctx.restore();
595 }
596 sea(ctx, p, W, H, S, cam, Lt) {
597   const lightX = has("moon") || CFG.sky === "night" ? W * 0.8 : CFG.sky === "sunset" ? W * 0.5 : W * 0.78;
598   ["#1f6fa3", "#185b87", "#12486c"].forEach((col, i) => {
599     const base = H * (0.68 + i * 0.08);
600     ctx.fillStyle = col; ctx.beginPath(); ctx.moveTo(0, H);
601     for (let x = 0; x <= W; x += 6) ctx.lineTo(x, base + Math.sin((x / W) * TAU * (2 + i) + p * TAU * (i % 2 ? -1 : 1)) * H * 0.018);
602     ctx.lineTo(W, H); ctx.closePath(); ctx.fill(); ink(ctx, 2);
603     ctx.fillStyle = "rgba(255,255,255,0.5)";
604     for (let k = 0; k < 14; k++) { const x = wrap(hash(k + i * 20) + p * (i % 2 ? -1 : 1)) * W, y = base + Math.sin((x / W) * TAU * (2 + i) + p * TAU * (i % 2 ? -1 : 1)) * H * 0.018
>     ; ctx.fillRect(x, y - 1, 8 + hash(k) * 10, 2); }
605   });
606   if (has("sun") || has("moon") || CFG.sky !== "day") { // glitter path under the light source
607     ctx.globalCompositeOperation = "lighter";
608     for (let i = 0; i < 70; i++) { const y = H * (0.68 + hash(i) * 0.3), spread = (y - H * 0.66) * 0.5, x = lightX + (hash(i + 30) - 0.5) * spread * 2, a = Math.max(0, Math.sin(p *
>   TAU * (2 + (i % 4)) + i * 3)); ctx.fillStyle = "rgba(255,240,200," + a * 0.5 + ")"; ctx.fillRect(x, y, 6 + hash(i + 3) * 10, 1.6); }
609     ctx.globalCompositeOperation = "source-over";
610   }
611 }
612 boat(ctx, p, W, H) {
613   const x = wrap(p) * (W + 240) - 120, y = H * 0.69 + Math.sin(p * TAU * 3) * 5, tilt = Math.sin(p * TAU * 3 + 1) * 0.06;
614   ctx.save(); ctx.translate(x, y); ctx.rotate(tilt);
615   ctx.fillStyle = "#7a4a2a"; ctx.beginPath(); ctx.moveTo(-60, 0); ctx.lineTo(60, 0); ctx.lineTo(42, 22); ctx.lineTo(-42, 22); ctx.closePath(); ctx.fill(); ink(ctx);
616   ctx.fillStyle = "#6b4a33"; ctx.fillRect(-3, -90, 6, 90);
617   const flap = Math.sin(p * TAU * 6) * 4;
618   ctx.fillStyle = "#f7f3e8"; ctx.beginPath(); ctx.moveTo(4, -86); ctx.quadraticCurveTo(34 + flap, -50, 52, -12); ctx.lineTo(4, -12); ctx.closePath(); ctx.fill(); ink(ctx);
619   ctx.restore();
620 }
621 planets(ctx, p, W, H) {
622   const cx = W / 2, cy = H / 2, k = Math.min(W, H) / 400;
623   ball(ctx, cx, cy, 34 * k, "#ffb43b");
624   [[70, 3, 6, "#6fb3ff"], [110, 2, 9, "#ff7a59"], [155, 1, 14, "#d7c08f"]].forEach(([d, turns, r, c], i) => {
625     ctx.strokeStyle = "rgba(255,255,255,0.15)"; ctx.lineWidth = 1; ctx.beginPath(); ctx.ellipse(cx, cy, d * k, d * k * 0.4, 0, 0, TAU); ctx.stroke();
626     const a = p * TAU * turns + i * 2; ball(ctx, cx + Math.cos(a) * d * k, cy + Math.sin(a) * d * k * 0.4, r * k, c);
627   });
628 }
629 rocket(ctx, p, W, H) {
630   const u = ease.inOut(wrap(p)), x = W * 0.5 + Math.sin(p * TAU * 2) * 12, y = H * 1.15 - u * H * 1.5;
631   for (let i = 0; i < 34; i++) {
632     const q = wrap(hash(i) + p * 6), sx = x + (hash(i + 5) - 0.5) * 34 * q, sy = y + 60 + q * 150;
633     ctx.fillStyle = "rgba(200,200,210," + (0.4 * (1 - q)) + ")"; ctx.beginPath(); ctx.arc(sx, sy, 8 + q * 28, 0, TAU); ctx.fill();
634   }
635   const fl = 26 + Math.sin(p * TAU * 20) * 8;
636   ctx.globalCompositeOperation = "lighter";
637   const fg = ctx.createRadialGradient(x, y + 50, 2, x, y + 50, 70 + fl); fg.addColorStop(0, "rgba(255,200,90,0.7)"); fg.addColorStop(1, "rgba(255,120,30,0)"); ctx.fillStyle = fg; ct
>   x.fillRect(x - 120, y - 40, 240, 240);
638   ctx.globalCompositeOperation = "source-over";
639   ctx.fillStyle = "#ffb703"; ctx.beginPath(); ctx.moveTo(x - 11, y + 36); ctx.quadraticCurveTo(x, y + 36 + fl * 2, x + 11, y + 36); ctx.fill();
640   ctx.fillStyle = STYLE === "3d" ? "#dfe6ee" : "#f1f4f8"; ctx.beginPath(); ctx.moveTo(x, y - 60); ctx.quadraticCurveTo(x + 24, y - 30, x + 20, y + 36); ctx.lineTo(x - 20, y + 36); c
>   tx.quadraticCurveTo(x - 24, y - 30, x, y - 60); ctx.fill(); ink(ctx);
641   ctx.fillStyle = PAL.accent2; ctx.beginPath(); ctx.moveTo(x - 20, y + 10); ctx.lineTo(x - 36, y + 40); ctx.lineTo(x - 20, y + 36); ctx.fill(); ctx.beginPath(); ctx.moveTo(x + 20, y
>   + 10); ctx.lineTo(x + 36, y + 40); ctx.lineTo(x + 20, y + 36); ctx.fill();
642   ball(ctx, x, y - 14, 8, "#6fd3ff");
643 }
644 fish(ctx, p, W, H) {
645   for (let i = 0; i < 11; i++) {
646     const dir = i % 2 ? 1 : -1, s = 12 + hash(i) * 16, x = wrap(hash(i + 2) + p * dir * (1 + (i % 3))) * (W + 120) - 60, y = H * (0.15 + hash(i + 9) * 0.6) + Math.sin(p * TAU * 2 +
>   i) * 10;
647     const wig = Math.sin(p * TAU * 12 + i) * 0.35;
648     ctx.save(); ctx.translate(x, y); ctx.scale(dir, 1);
649     ctx.fillStyle = "hsl(" + [20, 45, 190, 330, 280][i % 5] + ",85%,58%)";

```

```

650     ctx.beginPath(); ctx.moveTo(-s * 0.8, 0); ctx.lineTo(-s * 1.6, -s * (0.55 + wig)); ctx.lineTo(-s * 1.6, s * (0.55 - wig)); ctx.closePath(); ctx.fill(); ink(ctx, 1.5);
651     ctx.beginPath(); ctx.ellipse(0, 0, s, s * 0.55, 0, 0, TAU); ctx.fill(); ink(ctx, 1.5);
652     ctx.fillStyle = "rgba(255,255,255,0.3)"; ctx.beginPath(); ctx.ellipse(-s * 0.1, -s * 0.22, s * 0.55, s * 0.14, 0, 0, TAU); ctx.fill();
653     ctx.fillStyle = "#fff"; ctx.beginPath(); ctx.arc(s * 0.5, -s * 0.12, s * 0.16, 0, TAU); ctx.fill();
654     ctx.fillStyle = "#111"; ctx.beginPath(); ctx.arc(s * 0.55, -s * 0.12, s * 0.08, 0, TAU); ctx.fill();
655     ctx.restore();
656   }
657 },
658 bubbles(ctx, p, W, H) {
659   ctx.strokeStyle = "rgba(255,255,255,0.6)"; ctx.lineWidth = 1.2;
660   for (let i = 0; i < 55; i++) { const y = H - wrap(hash(i) + p * (1 + (i % 3))) * (H + 20), x = hash(i + 3) * W + Math.sin(p * TAU * 4 + i) * 6; ctx.beginPath(); ctx.arc(x, y, 2 +
> hash(i + 8) * 5, 0, TAU); ctx.stroke(); }
661 },
662 birds(ctx, p, W, H) {
663   ctx.strokeStyle = CFG.sky === "day" ? "#243042" : "#dfe6ff"; ctx.lineWidth = 2.2; ctx.lineCap = "round";
664   for (let i = 0; i < 7; i++) {
665     const x = wrap(p + (i % 4) * 0.03) * (W + 200) - 100 - Math.abs(i - 3) * 26, y = H * 0.28 + Math.abs(i - 3) * 16 + Math.sin(p * TAU * 2 + i) * 6, f = Math.sin(p * TAU * 14 + i)
> * 8;
666     ctx.beginPath(); ctx.moveTo(x - 12, y - f); ctx.quadraticCurveTo(x - 5, y - 4, x, y); ctx.quadraticCurveTo(x + 5, y - 4, x + 12, y - f); ctx.stroke();
667   }
668 },
669 balls(ctx, p, W, H) {
670   const floor = H * 0.84;
671   for (let i = 0; i < 5; i++) {
672     const x = W * (0.16 + i * 0.17), r = Math.min(W, H) * (0.04 + hash(i) * 0.03), n = 2 + (i % 3), ph = wrap(p * n + hash(i + 4));
673     const hgt = 4 * ph * (1 - ph), y = floor - r - hgt * H * 0.5, squash = ph < 0.06 || ph > 0.94 ? 0.75 : 1;
674     ctx.fillStyle = "rgba(0,0,0,0.22)"; ctx.beginPath(); ctx.ellipse(x, floor, r * (1.2 - hgt * 0.6), r * 0.25, 0, 0, TAU); ctx.fill();
675     ctx.save(); ctx.translate(x, y + r * (1 - squash)); ctx.scale(1 / squash, squash); ball(ctx, 0, 0, r, "hsl(" + i * 67 + ",80%,58%)"); ctx.restore();
676   }
677 },
678 car(ctx, p, W, H) {
679   const road = H * 0.82;
680   ctx.fillStyle = "#2b2f38"; ctx.fillRect(0, road, W, H - road);
681   ctx.fillStyle = "#f5d547"; for (let i = -1; i < 12; i++) ctx.fillRect(wrap(-p * 4 + i / 10) * (W + 80) - 40, road + (H - road) * 0.45, 40, 5);
682   const x = W * 0.45, y = road - 8 + Math.sin(p * TAU * 16) * 1.5;
683   ctx.fillStyle = PAL.accent2; ctx.beginPath(); ctx.moveTo(x - 90, y - 10); ctx.lineTo(x - 80, y - 38); ctx.lineTo(x - 40, y - 40); ctx.lineTo(x - 18, y - 66); ctx.lineTo(x + 40, y
> - 66); ctx.lineTo(x + 62, y - 40); ctx.lineTo(x + 92, y - 34); ctx.lineTo(x + 96, y - 10); ctx.closePath(); ctx.fill(); ink(ctx);
684   ctx.fillStyle = "#bfe3ff"; ctx.beginPath(); ctx.moveTo(x - 10, y - 60); ctx.lineTo(x + 36, y - 60); ctx.lineTo(x + 52, y - 42); ctx.lineTo(x - 26, y - 42); ctx.closePath(); ctx.fi
> ll();
685   ctx.globalCompositeOperation = "lighter"; const hl = ctx.createLinearGradient(x + 92, y - 22, x + 300, y - 22); hl.addColorStop(0, "rgba(255,240,180,0.55)"); hl.addColorStop(1, "r
> gba(255,240,180,0)"); ctx.fillStyle = hl; ctx.beginPath(); ctx.moveTo(x + 92, y - 26); ctx.lineTo(x + 300, y - 60); ctx.lineTo(x + 300, y + 10); ctx.lineTo(x + 92, y - 16); ctx.fi
> ll(); ctx.globalCompositeOperation = "source-over";
686   for (const wx of [x - 55, x + 60]) {
687     ball(ctx, wx, y - 6, 18, "#1c1c1c");
688     ctx.strokeStyle = "bbb"; ctx.lineWidth = 3; const a = -p * TAU * 12;
689     ctx.beginPath(); ctx.moveTo(wx + Math.cos(a) * 12, y - 6 + Math.sin(a) * 12); ctx.lineTo(wx - Math.cos(a) * 12, y - 6 - Math.sin(a) * 12); ctx.stroke();
690   }
691 },
692 fire(ctx, p, W, H) {
693   const bx = W / 2, by = H * 0.82;
694   ctx.fillStyle = "#5a3a22"; ctx.save(); ctx.translate(bx, by + 8); ctx.rotate(0.25); ctx.fillRect(-70, -8, 140, 16); ctx.rotate(-0.5); ctx.fillRect(-70, -8, 140, 16); ctx.restore()
> ;
695   ctx.globalCompositeOperation = "lighter";
696   const glow = ctx.createRadialGradient(bx, by - 30, 4, bx, by - 30, H * 0.5); glow.addColorStop(0, "rgba(255,150,50," + (0.32 + 0.06 * Math.sin(p * TAU * 12)) + ")"); glow.addColor
> Stop(1, "rgba(255,90,20,0)"); ctx.fillStyle = glow; ctx.fillRect(0, 0, W, H);
697   for (let i = 0; i < 130; i++) {
698     const q = wrap(hash(i) + p * (2 + (i % 3))), x = bx + (hash(i + 9) - 0.5) * 70 * (1 - q) + Math.sin(q * 8 + i) * 12, y = by - q * H * 0.42, r = (1 - q) * 22 + 3;
699     ctx.fillStyle = "hsla(" + (50 - q * 45) + ",100%," + (60 - q * 20) + "%," + (0.5 * (1 - q)) + ")";
700     ctx.beginPath(); ctx.arc(x, y, r, 0, TAU); ctx.fill();
701   }
702   for (let i = 0; i < 26; i++) { const q = wrap(hash(i + 300) + p * (1 + (i % 2))), x = bx + (hash(i + 310) - 0.5) * 120 + Math.sin(q * 9 + i) * 26, y = by - q * H * 0.7; ctx.fillSt
> yle = "rgba(255,190,90," + (1 - q) + ")"; ctx.fillRect(x, y, 2, 2); }
703   ctx.globalCompositeOperation = "source-over";
704 },
705 leaves(ctx, p, W, H) {
706   for (let i = 0; i < 40; i++) {
707     const y = wrap(hash(i) + p * (1 + (i % 2))) * (H + 40) - 20, x = hash(i + 6) * W + Math.sin(p * TAU * 2 + i) * 30, rot = p * TAU * (i % 2 ? 2 : -2) + i;
708     ctx.save(); ctx.translate(x, y); ctx.rotate(rot); ctx.fillStyle = ["#e76f51", "#f4a261", "#e9c46a", "#c1440e"][i % 4];
709     ctx.beginPath(); ctx.ellipse(0, 0, 11, 5, 0, 0, TAU); ctx.fill(); ink(ctx, 1.2); ctx.restore();
710   }
711 },

```

```

712 snow(ctx, p, W, H) {
713   ctx.fillStyle = "rgba(255,255,255,0.9)";
714   for (let i = 0; i < 190; i++) { const y = wrap(hash(i) + p * (1 + (i % 2))) * (H + 10) - 5, x = hash(i + 4) * W + Math.sin(p * TAU * 2 + i) * 14; ctx.beginPath(); ctx.arc(x, y, 1
>   + hash(i + 2) * 2.6, 0, TAU); ctx.fill(); }
715 },
716 rain(ctx, p, W, H) {
717   ctx.strokeStyle = "rgba(190,210,255,0.55)"; ctx.lineWidth = 1.3;
718   for (let i = 0; i < 260; i++) { const y = wrap(hash(i) + p * (3 + (i % 3))) * (H + 40) - 20, x = hash(i + 3) * (W + 60); ctx.beginPath(); ctx.moveTo(x, y); ctx.lineTo(x - 6, y + 1
>   8); ctx.stroke(); }
719 },
720 lightning(ctx, p, W, H) {
721   for (const at of [0.28, 0.71]) {
722     const d = p - at; if (d < 0 || d > 0.05) continue;
723     const a = 1 - d / 0.05;
724     ctx.fillStyle = "rgba(230,235,255," + 0.45 * a + ")"; ctx.fillRect(0, 0, W, H);
725     ctx.strokeStyle = "rgba(255,255,255," + a + ")"; ctx.lineWidth = 3; ctx.beginPath();
726     let x = W * (at < 0.5 ? 0.3 : 0.65), y = 0; ctx.moveTo(x, y);
727     for (let s = 0; s < 9; s++) { x += (hash(s + at * 100) - 0.5) * 60; y += H * 0.07; ctx.lineTo(x, y); }
728     ctx.stroke();
729   }
730 },
731 confetti(ctx, p, W, H) {
732   for (let i = 0; i < 170; i++) {
733     const y = wrap(hash(i) + p * (1 + (i % 3))) * (H + 30) - 15, x = hash(i + 7) * W + Math.sin(p * TAU * 3 + i) * 20, rot = p * TAU * 4 + i;
734     ctx.save(); ctx.translate(x, y); ctx.rotate(rot); ctx.scale(1, Math.cos(rot * 1.3));
735     ctx.fillStyle = "hsl(" + Math.floor(hash(i + 1) * 360) + ",90%,60%)"; ctx.fillRect(-5, -3, 10, 6); ctx.restore();
736   }
737 },
738 hearts(ctx, p, W, H) {
739   for (let i = 0; i < 26; i++) {
740     const q = wrap(hash(i) + p * (1 + (i % 2))), y = H + 30 - q * (H + 60), x = hash(i + 5) * W + Math.sin(q * 6 + i) * 20, s = 8 + hash(i + 2) * 14;
741     ctx.fillStyle = "hsla(" + (340 + hash(i) * 30) + ",85%,62%," + (1 - q * 0.6) + ")"; heart(ctx, x, y, s); ctx.fill(); ink(ctx, 1.5);
742   }
743 },
744 neogrid(ctx, p, W, H) {
745   const hz = H * 0.58;
746   const sunG = ctx.createLinearGradient(0, hz - H * 0.3, 0, hz); sunG.addColorStop(0, "#ffd166"); sunG.addColorStop(1, "#ff3d8b");
747   ctx.fillStyle = sunG; ctx.beginPath(); ctx.arc(W / 2, hz, H * 0.22, Math.PI, 0); ctx.fill();
748   ctx.fillStyle = PAL.sky[1]; for (let i = 0; i < 5; i++) ctx.fillRect(0, hz - H * 0.02 - i * H * 0.035, W, 3 + i);
749   ctx.fillStyle = "#0b0520"; ctx.fillRect(0, hz, W, H - hz);
750   ctx.strokeStyle = "#ff3df2"; ctx.lineWidth = 1.5; ctx.shadowColor = "#ff3df2"; ctx.shadowBlur = 10;
751   for (let i = -12; i <= 12; i++) { ctx.beginPath(); ctx.moveTo(W / 2 + i * 12, hz); ctx.lineTo(W / 2 + i * W * 0.12, H); ctx.stroke(); }
752   for (let i = 0; i < 12; i++) { const z = wrap(i / 12 - p * 2), y = hz + (H - hz) * z * z; ctx.beginPath(); ctx.moveTo(0, y); ctx.lineTo(W, y); ctx.stroke(); }
753   ctx.shadowBlur = 0;
754 },
755 abstract(ctx, p, W, H) {
756   ctx.globalCompositeOperation = "lighter";
757   for (let i = 0; i < 8; i++) {
758     const a = p * TAU * (i % 2 ? 1 : -1) + i, x = W / 2 + Math.cos(a) * W * 0.22 * (0.5 + hash(i)), y = H / 2 + Math.sin(a * 2) * H * 0.2, r = Math.min(W, H) * (0.12 + hash(i + 3) *
>   0.12);
759     const g = ctx.createRadialGradient(x, y, 0, x, y, r); g.addColorStop(0, "hsla(" + (i * 50 + p * 360) + ",90%,60%,0.7)"); g.addColorStop(1, "rgba(0,0,0,0)");
760     ctx.fillStyle = g; ctx.beginPath(); ctx.arc(x, y, r, 0, TAU); ctx.fill();
761   }
762   ctx.globalCompositeOperation = "source-over";
763 },
764 motes(ctx, p, W, H, S, cam, Lt) { // floating dust lit by the light, drifting with the camera
765   ctx.globalCompositeOperation = "lighter";
766   for (let i = 0; i < 46; i++) {
767     const depth = 0.3 + hash(i + 70) * 0.9, x = ((hash(i) * W + Math.sin(p * TAU + i) * 14 * depth + cam.x * W * depth * 0.6) % W + W) % W, y = ((hash(i + 20) * H - p * H * 0.12 * d
>   epth * (1 + (i % 2))) % H + H) % H;
768     const r = 1 + depth * 2.2, tw = 0.35 + 0.65 * Math.abs(Math.sin(p * TAU * (1 + (i % 4)) + i));
769     const g = ctx.createRadialGradient(x, y, 0, x, y, r * 3); g.addColorStop(0, "rgba(255,240,210," + 0.35 * tw + ")"); g.addColorStop(1, "rgba(255,240,210,0)");
770     ctx.fillStyle = g; ctx.fillRect(x - r * 3, y - r * 3, r * 6, r * 6);
771   }
772   ctx.globalCompositeOperation = "source-over";
773 },
774 text(ctx, p, W, H) {
775   const s = CFG.text; if (!s) return;
776   const size = Math.min(W / (s.length * 0.62 + 1), H * 0.18);
777   ctx.font = "800 " + size + "px system-ui, sans-serif"; ctx.textBaseline = "middle"; ctx.textAlign = "center";
778   const total = ctx.measureText(s).width; let x = W / 2 - total / 2;

```

```

779     const out = p > 0.88 ? ease.in((p - 0.88) / 0.12) : 0;
780     for (let i = 0; i < s.length; i++) {
781         const cw = ctx.measureText(s[i]).width, k = ease.outBack(clamp(p * 5 - i * 0.12, 0, 1)) * (1 - out);
782         ctx.save(); ctx.translate(x + cw / 2, H * 0.45 + Math.sin(p * TAU * 2 + i * 0.5) * 6); ctx.scale(k, k);
783         ctx.fillStyle = "#fff"; ctx.shadowColor = PAL.accent; ctx.shadowBlur = 18; ctx.fillText(s[i], 0, 0);
784         if (INKED) { ctx.shadowBlur = 0; ctx.lineWidth = 3; ctx.strokeStyle = PAL.ink; ctx.strokeText(s[i], 0, 0); ctx.fillText(s[i], 0, 0); }
785         ctx.restore(); x += cw;
786     }
787 }
788 };
789 const ORDER = ["sky", "stars", "sun", "moon", "neongrid", "clouds", "mountains", "pines", "castle", "city", "sea", "boat", "planets", "abstract", "rocket", "fish", "bubbles", "ground"
> , "car", "fire", "balls", "birds", "dragon", "ufo", "character", "leaves", "snow", "rain", "lightning", "confetti", "hearts", "motes", "text"];
790 function setup(W, H) {
791     const P = W * 1.6, mkLayer = (n, minH, maxH, seedOff) => {
792         const out = []; let x = 0, i = 0, widths = [];
793         while (x < P) { const w = 30 + hash(i * 3.3 + seedOff) * 54; widths.push(w); x += w; i++; }
794         const k = P / x; x = 0;
795         widths.forEach((w0, j) => {
796             const w = w0 * k, h = H * (minH + hash(j * 5.1 + seedOff) * (maxH - minH)), win = [];
797             for (let wy = 12; wy < h - 10; wy += 15) for (let wx = 7; wx < w - 9; wx += 11) win.push([wx, wy, hash(j * 17 + wx * 1.3 + wy * 0.7 + seedOff), hash(j * 3 + wx * 7.7 + wy * 1.9
> + seedOff) < 0.42]);
798             out.push({ x, w, h, win, tank: hash(j + seedOff * 2) < 0.22 ? 14 + hash(j) * 14 : 0, ant: hash(j + seedOff * 3) < 0.18 ? 18 + hash(j + 4) * 26 : 0 });
799             x += w;
800         });
801         return out;
802     };
803     DATA = { P, city: [mkLayer(0, 0.16, 0.34, 1), mkLayer(1, 0.14, 0.3, 5), mkLayer(2, 0.09, 0.26, 9)] };
804     const mkTrees = (n, hmin, hmax, off) => Array.from({ length: n }, (_, i) => ({ x: ((i + hash(i * 2.3 + off) * 0.8) / n) * P, h: H * (hmin + hash(i * 4.1 + off) * (hmax - hmin)), ph:
> hash(i + off) * TAU }));
805     DATA.pines = [mkTrees(14, 0.16, 0.26, 2), mkTrees(11, 0.2, 0.32, 6), mkTrees(8, 0.26, 0.42, 11)];
806     const st = {};
807     if (has("character")) { const q = pose(0, W, H, 6); st.scarf = PHYS.makeChain(16, q.nx, q.ny, q.U * 0.085); st.hair = PHYS.makeChain(5, q.bx, q.by, q.U * 0.05); }
808     if (has("dragon")) { const q = dragonPose(0, W, H, 6), an = dragonAnchors(q); st.tail = PHYS.makeChain(15, an.rx, an.ry, q.U * 0.1); }
809     if (has("castle")) { const g = castleGeom(W, H); st.flags = g.poles.map(pl => PHYS.makeChain(7, pl.x, pl.y, g.U * 0.05)); }
810     DATA.init = st;
811     return st;
812 }
813 function draw(ctx, t, info) {
814     const { W, H, duration, state } = info, Lt = info.light, cam = info.camera; VW = W;
815     let p = info.progress, tt = t;
816     if (STYLE === "frames") {
817         const drawings = Math.max(1, Math.round(duration * 8)), d = Math.floor(p * drawings);
818         p = d / drawings; tt = p * duration;
819         ctx.translate((hash(d) - 0.5) * 3, (hash(d + 50) - 0.5) * 3);
820     }
821     for (const name of ORDER) {
822         if (name !== "sky" && !has(name) && !(name === "ground" && RUN) && !(name === "motes" && CFG.sky !== "underwater")) continue;
823         ctx.save(); L[name](ctx, p, W, H, state, cam, Lt, tt, duration); ctx.restore();
824     }
825     const needsGround = (has("fire") || has("balls")) && !has("sea") && !has("city") && CFG.sky !== "underwater" && CFG.sky !== "space";
826     if (needsGround) { ctx.fillStyle = PAL.ground; ctx.fillRect(0, H * 0.84, W, H * 0.16); }
827     if (CFG.sky === "underwater") { ctx.fillStyle = PAL.ground; ctx.beginPath(); ctx.moveTo(0, H); for (let x = 0; x <= W; x += 20) ctx.lineTo(x, H * 0.93 - Math.sin(x * 0.02) * 8); ctx
> .lineTo(W, H); ctx.fill(); }
828     if (STYLE === "frames") { ctx.fillStyle = "rgba(245,235,215,0.12)"; ctx.fillRect(-10, -10, W + 20, H + 20); }
829 }
830 }
831
832 // Built-in examples that use the mock generator's engine, so they show camera, light and physics together.
833 PTM_EXAMPLES.push({ id: "ex-runner", title: "Rooftop runner", note: "A runner sprints past a skyline at golden hour. The scarf and hair are simulated physics.",
834     prompt: "A runner sprinting across city rooftops at sunset with a flowing scarf", params: { style: "3d", duration: 6, aspect: "16:9", fps: 30, camera: "handheld", camK: 1, light: "g
> olden", blur: 2, detail: "high", physics: true },
835     code: PTM_mock("A runner sprinting across city rooftops at sunset with a flowing scarf", "3d").code, engine: "example" });
836 PTM_EXAMPLES.push({ id: "ex-neon-storm", title: "Neon storm over the city", note: "Rain, lightning and a synthwave skyline, with a slow orbit camera.",
837     prompt: "Neon rain and lightning over a cyberpunk city at night", params: { style: "anime", duration: 6, aspect: "16:9", fps: 30, camera: "orbit", camK: 1, light: "neon", blur: 1, d
> etail: "high", physics: false },
838     code: PTM_mock("Neon rain and lightning over a cyberpunk city at night with a storm", "anime").code, engine: "example" });

```

```

1  /*
2  * PromptToMotion sound design.
3  *
4  * A procedural soundscape built with the Web Audio API: no audio files, everything is synthesised.
5  * Each animation carries a set of tags (rain, sea, fire, city, dragon, ...), either from the mock generator's analysis
6  * of the prompt or from the "sound" list Claude returns. The tags become:
7  *   - continuous beds    (rain hiss, ocean swell, wind, engine drones, a musical pad chosen from the sky's mood)
8  *   - random events      (crickets, birds, bubbles, fire crackle, chimes, a synth arpeggio)
9  *   - synced events      (footsteps on every foot-plant, thunder after each lightning flash, wing whooshes,
10 *                          firework bursts), triggered from the player's clock so they line up with the picture
11 * Sound is for the preview only. Downloads are silent. It starts only from a click (browser rule), and is off by default.
12 */
13 (() => {
14   "use strict";
15   const AC = window.AudioContext || window.webkitAudioContext;
16   const S = { W: 960, H: 540, ctx: null, master: null, bus: null, analyser: null, white: null, brown: null, nodes: [], timers: [], enabled: false, playing: false, take: null, dur: 6,
17   >   tags: new Set(), sky: "day", prevT: 0, idx: {} };
18   const KNOWN = ["rain", "wind", "sea", "fire", "city", "lightning", "rocket", "car", "ufo", "dragon", "character", "pines", "birds", "bubbles", "confetti", "hearts", "neongrid", "pla
19   >   nets", "snow", "fireworks", "warp", "stars", "boat", "fish", "castle"];
20   const rnd = (a, b) => a + Math.random() * (b - a);
21
22   function ensure() {
23     if (S.ctx) return S.ctx;
24     S.ctx = new AC();
25     S.master = S.ctx.createGain(); S.master.gain.value = 0;
26     const comp = S.ctx.createDynamicsCompressor(); comp.threshold.value = -20; comp.ratio.value = 6; comp.attack.value = 0.01; comp.release.value = 0.25;
27     S.analyser = S.ctx.createAnalyser(); S.analyser.fftSize = 1024;
28     S.master.connect(comp); comp.connect(S.analyser); S.analyser.connect(S.ctx.destination);
29     const n = S.ctx.sampleRate * 3, w = S.ctx.createBuffer(1, n, S.ctx.sampleRate), b = S.ctx.createBuffer(1, n, S.ctx.sampleRate), wd = w.getChannelData(0), bd = b.getChannelData(0);
30     let last = 0; for (let i = 0; i < n; i++) { const r = Math.random() * 2 - 1; wd[i] = r; last = (last + 0.02 * r) / 1.02; bd[i] = last * 3.5; }
31     S.white = w; S.brown = b;
32     return S.ctx;
33   }
34
35   // ----- small building blocks -----
36   const keep = n => { S.nodes.push(n); return n; };
37   const gain = v => { const g = S.ctx.createGain(); g.gain.value = v; return g; };
38   const filt = (type, f, q) => { const b = S.ctx.createBiquadFilter(); b.type = type; b.frequency.value = f; if (q) b.Q.value = q; return b; };
39   const osc = (type, f) => { const o = S.ctx.createOscillator(); o.type = type; o.frequency.value = f; o.start(); return keep(o); };
40   const noise = buf => { const s = S.ctx.createBufferSource(); s.buffer = buf; s.loop = true; s.loopStart = Math.random(); s.start(); return keep(s); };
41   const lfo = (rate, depth, param) => { const o = osc("sine", rate), g = gain(depth); o.connect(g); g.connect(param); };
42   const chain = (...n) => { for (let i = 0; i < n.length - 1; i++) n[i].connect(n[i + 1]); return n[n.length - 1]; };
43   const later = (fn, ms) => { const id = setTimeout(fn, ms); S.timers.push(id); return id; };
44   function every(min, max, fn) { const go = () => { fn(); later(go, rnd(min, max)); }; later(go, rnd(min, max)); }
45   function envTone(type, f0, f1, dur, vol, dest, filterHz) {
46     const t = S.ctx.currentTime, o = S.ctx.createOscillator(), g = S.ctx.createGain(); o.type = type; o.frequency.setValueAtTime(f0, t); if (f1) o.frequency.exponentialRampToValueAtTime(
47     >   f1, t + dur);
48     g.gain.setValueAtTime(0.0001, t); g.gain.exponentialRampToValueAtTime(vol, t + Math.min(0.02, dur / 4)); g.gain.exponentialRampToValueAtTime(0.0001, t + dur);
49     let n = o; if (filterHz) { const f = filt("lowpass", filterHz); o.connect(f); n = f; } n.connect(g); g.connect(dest || S.bus); o.start(t); o.stop(t + dur + 0.05);
50   }
51   function burst(buf, dur, vol, filterType, hz, q, dest) {
52     const t = S.ctx.currentTime, s = S.ctx.createBufferSource(), g = S.ctx.createGain(), f = filt(filterType, hz, q); s.buffer = buf; s.loop = true; s.loopStart = Math.random();
53     g.gain.setValueAtTime(0.0001, t); g.gain.exponentialRampToValueAtTime(vol, t + 0.008); g.gain.exponentialRampToValueAtTime(0.0001, t + dur);
54     s.connect(f); f.connect(g); g.connect(dest || S.bus); s.start(t); s.stop(t + dur + 0.05);
55   }
56
57   // ----- one-shot sounds -----
58   const thud = () => { burst(S.brown, 0.13, 0.5, "lowpass", 220, 1); envTone("sine", 95, 48, 0.14, 0.45); };
59   const thunder = () => { burst(S.brown, 2.4, 0.9, "lowpass", 170, 1); later(() => burst(S.white, 0.5, 0.25, "lowpass", 900, 0.7), 40); };
60   const whoosh = () => { const t = S.ctx.currentTime, s = S.ctx.createBufferSource(), f = filt("bandpass", 300, 1.1), g = S.ctx.createGain(); s.buffer = S.white; s.loop = true; f.frequency.set
61   >   ValueAtTime(260, t); f.frequency.exponentialRampToValueAtTime(950, t + 0.2); f.frequency.exponentialRampToValueAtTime(260, t + 0.5);
62   >   g.gain.setValueAtTime(0.0001, t); g.gain.exponentialRampToValueAtTime(0.35, t + 0.18); g.gain.exponentialRampToValueAtTime(0.0001, t + 0.55); s.connect(f); f.connect(g); g.connect
63   >   (S.bus); s.start(t); s.stop(t + 0.6); };
64   const firework = () => { burst(S.brown, 0.5, 0.9, "lowpass", 260, 1); for (let i = 0; i < 6; i++) later(() => burst(S.white, 0.05, 0.22, "highpass", 3000, 1), 60 + i * rnd(30, 90));
65   >   };
66   const chirp = () => { const f = rnd(2200, 4200); envTone("sine", f, f * rnd(1.15, 1.5), rnd(0.07, 0.14), 0.05, S.bus); if (Math.random() < 0.6) later(() => envTone("sine", f * 1.2,
67   >   f * 1.6, 0.09, 0.04), 130); };
68   const blip = () => { const f = rnd(420, 900); envTone("sine", f, f * 1.8, 0.09, 0.07); };
69   const crackle = () => burst(S.white, rnd(0.012, 0.04), rnd(0.12, 0.4), "highpass", rnd(1500, 4500), 1);
70   const PENTA = [523.25, 587.33, 659.25, 783.99, 880, 1046.5];
71   const chime = () => { const f = PENTA[Math.random() * PENTA.length | 0]; envTone("sine", f, 0, 1.3, 0.06); envTone("sine", f * 2, 0, 0.8, 0.02); };

```

```

65 const ARP = [110, 130.81, 164.81, 220, 164.81, 130.81, 146.83, 174.61];
66 let arpI = 0;
67 const arp = () => { const f = ARP[arpI++ % ARP.length]; envTone("sawtooth", f * 2, 0, 0.22, 0.05, S.bus, 1500); if (arpI % 4 === 1) envTone("sawtooth", f, 0, 0.5, 0.06, S.bus, 400);
>   };
68
69 // ----- beds and music -----
70 const PADS = { day: [261.63, 329.63, 392, 493.88, 587.33], sunset: [174.61, 220, 261.63, 329.63, 392], night: [110, 130.81, 164.81, 196, 246.94], space: [55, 82.41, 110, 164.81, 246
>   .94], underwater: [98, 146.83, 196, 246.94, 293.66] };
71 function pad(sky, vol) {
72   const notes = PADS[sky] || PADS.day, lp = filt("lowpass", sky === "space" ? 700 : 1100, 0.4), out = gain(vol); lp.connect(out); out.connect(S.bus);
73   notes.forEach((f, i) => { for (const d of [-4, 5]) { const o = osc(i % 2 ? "triangle" : "sine", f); o.detune.value = d; const g = gain(0.0001); o.connect(g); g.connect(lp); lfo(rn
>   d(0.05, 0.12), 0.05, g.gain); g.gain.value = 0.06; } });
74   lfo(0.04, 300, lp.frequency);
75 }
76 const bed = {
77   rain() { chain(noise(S.white), filt("highpass", 900), filt("bandpass", 5200, 0.4), gain(0.2), S.bus); },
78   wind() { const lp = filt("lowpass", 520, 0.8); chain(noise(S.brown), lp, gain(0.3), S.bus); lfo(0.13, 260, lp.frequency); },
79   sea() { const g = gain(0.14); chain(noise(S.brown), filt("lowpass", 440), g, S.bus); lfo(0.11, 0.09, g.gain); },
80   fire() { chain(noise(S.brown), filt("lowpass", 700), gain(0.1), S.bus); every(0.05, 0.25, crackle); },
81   city() { const lp = filt("lowpass", 230); lp.connect(S.bus); const g = gain(0.06); g.connect(lp); for (const f of [55, 55.4, 82.5]) { const o = osc("sawtooth", f); o.connect(g); }
>   chain(noise(S.brown), filt("lowpass", 300), gain(0.05), S.bus); },
82   lightning() { chain(noise(S.brown), filt("lowpass", 120), gain(0.1), S.bus); },
83   rocket() { chain(noise(S.white), filt("lowpass", 420), gain(0.22), S.bus); chain(noise(S.brown), filt("lowpass", 160), gain(0.3), S.bus); },
84   car() { const lp = filt("lowpass", 260), g = gain(0.09); lp.connect(S.bus); g.connect(lp); const o = osc("sawtooth", 48); o.connect(g); lfo(3.5, 4, o.frequency); chain(noise(S.whi
>   te), filt("bandpass", 800, 0.6), gain(0.03), S.bus); },
85   ufo() { const g = gain(0.08), lp = filt("lowpass", 1400); g.connect(lp); lp.connect(S.bus); const a = osc("sine", 220), b = osc("sine", 330.6); a.connect(g); b.connect(g); lfo(6,
>   34, a.frequency); lfo(0.4, 60, b.frequency); },
86   dragon() { chain(noise(S.brown), filt("lowpass", 190), gain(0.22), S.bus); const lp = filt("lowpass", 420); chain(noise(S.brown), lp, gain(0.12), S.bus); lfo(0.09, 180, lp.frequen
>   cy); },
87   pines() { const lp = filt("lowpass", 700); chain(noise(S.brown), lp, gain(0.1), S.bus); lfo(0.17, 220, lp.frequency); },
88   snow() { const lp = filt("lowpass", 480); chain(noise(S.brown), lp, gain(0.1), S.bus); lfo(0.1, 200, lp.frequency); },
89   warp() { chain(noise(S.white), filt("lowpass", 300), gain(0.14), S.bus); const o = osc("sawtooth", 62), g = gain(0.06); o.connect(filt("lowpass", 240)).connect(g); g.connect(S.bus
>   ); },
90   birds() { every(0.6, 2.4, chirp); },
91   bubbles() { every(0.15, 0.7, blip); },
92   confetti() { every(0.1, 0.4, chime); },
93   hearts() { every(0.3, 0.9, chime); },
94   neongrid() { later(function loop() { arp(); later(loop, 240); }, 100); },
95   planets() { pad("space", 0.05); }
96 };
97
98 function tagsFor(take) {
99   let layers = [], sky = "day";
100   const prompt = String(take.prompt || "").replace(/ → /g, " ");
101   const a = window.PTM_analyze ? window.PTM_analyze(prompt) : { layers: [], sky: "day" };
102   if (take.sound && Array.isArray(take.sound.layers)) { layers = take.sound.layers.filter(t => KNOWN.includes(t)); sky = take.sound.sky || a.sky; }
103   else { layers = a.layers; sky = a.sky; }
104   const set = new Set(layers);
105   if (/fireworks/.test(take.id || "")) set.add("fireworks");
106   if (/solar/.test(take.id || "")) { set.add("planets"); sky = "space"; }
107   if (/warp/.test(take.id || "")) { set.add("warp"); sky = "space"; }
108   if (!set.has("rain") && !set.has("city") && (sky === "night" || set.has("stars"))) && !set.has("ufo") && !set.has("rocket") && sky !== "space" set.add("crickets");
109   return { set, sky: ["day", "sunset", "night", "space", "underwater"].includes(sky) ? sky : "day" };
110 }
111
112 function teardown() {
113   S.timers.forEach(clearTimeout); S.timers = [];
114   for (const n of S.nodes) { try { n.stop && n.stop(); } catch (_) {} try { n.disconnect(); } catch (_) {} }
115   S.nodes = [];
116   if (S.bus) { const old = S.bus, t = S.ctx.currentTime; old.gain.cancelScheduledValues(t); old.gain.setTargetAtTime(0, t, 0.05); setTimeout(() => { try { old.disconnect(); } catch
>   (_) {} }, 400); }
117   S.bus = null;
118 }
119 function build() {
120   if (!S.ctx || !S.take) return;
121   teardown();
122   S.bus = S.ctx.createGain(); S.bus.gain.value = 0; S.bus.connect(S.master); S.bus.gain.setTargetAtTime(1, S.ctx.currentTime, 0.25);
123   const { set, sky } = tagsFor(S.take); S.tags = set; S.sky = sky; arpI = 0;
124   pad(sky, set.has("neongrid") ? 0.06 : 0.12);
125   for (const t of set) if (bed[t]) bed[t]();
126   if (set.has("crickets")) { const g = gain(0.012), o = osc("sine", 4400); o.connect(g); g.connect(S.bus); lfo(24, 0.012, g.gain); lfo(0.3, 0.008, g.gain); }

```

```

127     if (set.has("sea") || set.has("boat")) if (!set.has("sea")) bed.sea();
128     S.idx = {}; S.prevT = 0;
129 }
130 function applyPlaying() { if (!S.ctx || !S.master) return; const t = S.ctx.currentTime; S.master.gain.cancelScheduledValues(t); S.master.gain.setTargetAtTime(S.enabled && S.playing
>     ? 0.85 : 0, t, S.playing ? 0.12 : 0.08); }
131
132 // synced events: fire when the clock crosses a moment, but not when the viewer scrubs across it
133 function crossed(key, prev, t, at) { return prev < at && t >= at && t - prev < 0.35; }
134 function onTime(m) {
135     const was = S.playing; S.playing = !!m.playing; S.dur = m.duration;
136     if (was !== S.playing) applyPlaying();
137     if (!S.enabled || !S.bus || !S.playing) { S.prevT = m.t; return; }
138     const t = m.t, prev = S.prevT, dur = m.duration; S.prevT = t;
139     if (t < prev - 0.35) return; // looped or scrubbed back
140     if (S.tags.has("character")) { const cycles = window.PTM_gait ? PTM_gait(S.W, S.H, dur).cycles : Math.max(2, Math.round(dur * 1.4)), interval = dur / (2 * cycles), i = Math.floor(
>     t / interval), p = Math.floor(prev / interval); if (i !== p && i - p < 3) thud(); } // one thud per foot landing, in step with the picture
141     if (S.tags.has("dragon")) { const interval = dur / Math.max(3, Math.round(dur * 1.6)), i = Math.floor(t / interval), p = Math.floor(prev / interval); if (i !== p && i - p < 3) who
>     osh(); }
142     if (S.tags.has("lightning")) for (const at of [0.28, 0.71]) if (crossed("l" + at, prev, t, at * dur + 0.2)) thunder();
143     if (S.tags.has("fireworks")) { const i = Math.floor((t - 0.85) / 0.45), p = Math.floor((prev - 0.85) / 0.45); if (t > 0.85 && i !== p && i - p < 3) firework(); }
144 }
145
146 window.PTM_SND = {
147     supported: !!AC,
148     get enabled() { return S.enabled; },
149     async toggle() {
150         if (!AC) return false;
151         S.enabled = !S.enabled;
152         if (S.enabled) { ensure(); try { await S.ctx.resume(); } catch (_) {} build(); S.playing = true; applyPlaying(); }
153         else { applyPlaying(); setTimeout(() => { if (!S.enabled) teardown(); }, 500); }
154         return S.enabled;
155     },
156     setTake(take, params) { S.take = take; S.dur = params && params.duration || 6; const wh = params && window.PTM_ASPECTS && window.PTM_ASPECTS[params.aspect]; if (wh) { S.W = wh[0];
>     S.H = wh[1]; } if (S.enabled) build(); },
157     onTime,
158     tags: () => [...S.tags],
159     level() { if (!S.analyser) return 0; const d = new Float32Array(S.analyser.fftSize); S.analyser.getFloatTimeDomainData(d); let e = 0; for (const v of d) e += v * v; return Math.sq
>     rt(e / d.length); },
160     state: () => (S.ctx ? S.ctx.state : "none"),
161     KNOWN
162 };
163 }());

```

```

1  /*
2  * PromptToMotion app.
3  *
4  * Flow: prompt + parameters -> generator (mock or Claude) -> scene code -> sandboxed iframe player -> export / history.
5  * Depends on runtime.js (PTM_runtime) and scenes.js (PTM_EXAMPLES, PTM_mock).
6  */
7  (() => {
8    "use strict";
9    const $ = (s, r = document) => r.querySelector(s);
10   const clamp = (v, a, b) => Math.min(b, Math.max(a, v));
11
12   // ----- options
13   const ASPECTS = { "16:9": [960, 540], "9:16": [540, 960], "1:1": [720, 720] };
14   window.PTM_ASPECTS = ASPECTS;
15   const FPS_OPTIONS = [12, 24, 30, 60];
16   const CAMERAS = [
17     ["static", "Static"], ["push", "Push in"], ["pull", "Pull out"], ["panl", "Pan left"], ["panr", "Pan right"], ["orbit", "Orbit"], ["crane", "Crane"], ["handheld", "Handheld"]];
18   const LIGHTS = [
19     ["natural", "Natural sunlight", "#fff1d6,#8fb6e8"], ["cinematic", "Cinematic", "#ffb066,#2b6f8f"], ["neon", "Neon glow", "#ff3df2,#22d3ee"], ["golden", "Golden hour", "#ffb45a,#ff6a3a"], ["moonlit", "Moonlit", "#a9c1ff,#3b4a8f"], ["studio", "Studio soft", "#ffffff,#c9d3ea"], ["none", "As drawn", "#7a7f90,#3a3f50"]];
20   const BLURS = ["Off", "Light", "Medium", "Heavy"];
21   const DETAILS = ["standard", "Standard"], ["high", "High"], ["ultra", "Ultra"];
22   const CAM_NAME = Object.fromEntries(CAMERAS), LIGHT_NAME = Object.fromEntries(LIGHTS.map(l => [l[0], l[1]]));
23   const DEFAULT_LOOK = { camera: "static", camK: 1, light: "natural", blur: 1, detail: "high", physics: true };
24   const normParams = p => Object.assign({}, DEFAULT_LOOK, p);
25   // Viewing preference (not part of an animation): draw the preview at the screen's refresh rate. Downloads always use the chosen frame rate.
26   const prefs = { smooth: true };
27   try { prefs.smooth = localStorage.getItem("ptm-smooth") !== "0"; } catch (e) { /* storage blocked: keep the default */ }
28   const STYLES = [
29     { id: "vector", name: "2D Vector", short: "2D", hint: "Flat shapes and bold colour" },
30     { id: "3d", name: "3D Render", short: "3D", hint: "Depth, light and shadow" },
31     { id: "anime", name: "Anime", short: "Anime", hint: "Ink outlines, cel shading" },
32     { id: "pixel", name: "Pixel Art", short: "Pixel", hint: "Chunky pixels, few colours" },
33     { id: "frames", name: "Frame-by-Frame", short: "Hand-drawn", hint: "Hand-drawn, 8 drawings a second" }
34   ];
35   const STYLE_NAME = Object.fromEntries(STYLES.map(s => [s.id, s.name]));
36   // The hero preview plays these in turn, generated live by the real engine as each prompt is typed.
37   const HERO_SHOW = [
38     { prompt: "A runner sprinting across city rooftops at sunset with a flowing scarf", style: "3d", light: "golden", camera: "handheld" },
39     { prompt: "A dragon flying over snowy mountains breathing fire at night", style: "anime", light: "cinematic", camera: "push" },
40     { prompt: "A knight castle on a hill at sunset with waving flags", style: "3d", light: "golden", camera: "orbit" },
41     { prompt: "A UFO hovering over a quiet pine forest at night", style: "3d", light: "moonlit", camera: "pull" },
42     { prompt: "Neon rain over a cyberpunk city", style: "anime", light: "neon", camera: "panr" },
43     { prompt: "A pirate ship sailing through a storm at night with lightning", style: "vector", light: "cinematic", camera: "crane" },
44     { prompt: "A rocket lifting off from a snowy launch pad while stars twinkle", style: "3d", light: "natural", camera: "push" },
45     { prompt: "Fish swimming in an aquarium with bubbles", style: "vector", light: "studio", camera: "panl" }
46   ];
47   const IDEAS = HERO_SHOW.map(h => h.prompt);
48   const HISTORY_KEY = "ptm-history-v1", HISTORY_MAX = 24;
49
50   // ----- capabilities (present inside claude.ai only)
51   const useCap = name => (window.claude && window.claude.use ? window.claude.use(name).catch(() => null) : Promise.resolve(null));
52   const samplePromise = useCap("sample");
53   const downloadsPromise = useCap("downloads");
54
55   // ----- player: one sandboxed iframe + messaging
56   const players = new Set();
57   let SID = 0;
58   const escScript = s => String(s).replace(/<\/script/gi, "<\\\/script");
59
60   function buildDoc(code, cfg) {
61     return `<!doctype html><html><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><style>html,body{margin:0;height:100%;overflow:hidden;background:#000}#view{display:block;width:100%;height:100%;object-fit:contain}</style></head><body><canvas id="view"></canvas><script>(${escScript(PTM_runtime.toString())})(${escScript(JSON.stringify(cfg))});</script><script>${escScript(code)}</script><script>PTM_start(typeof draw=="function"?draw:null,typeof setup=="function"?setup:null,typeof simulate=="function"?simulate:null);</script></body></html>`;
62   }
63
64   class Player {
65     constructor(host, opts = {}) {
66       this.host = host; this.frame = null; this.sid = 0; this.loop = opts.loop !== false;
67       this.onTime = opts.onTime || (() => {}); this.onError = opts.onError || (() => {}); this.onEnded = opts.onEnded || (() => {}); this.onPerf = opts.onPerf || (() => {});
68     }
69   }

```

```

69     this.pending = null; this.thumbWait = null; this.exportWait = null; this.last = null;
70     players.add(this);
71 }
72 load(take, { autoplay = true } = {}) {
73     const p = take.params, [W, H] = ASPECTS[p.aspect];
74     this.sid = ++SID; this.last = null;
75     const cfg = { sid: this.sid, W, H, fps: p.fps, duration: p.duration, pixel: p.style === "pixel" ? 4 : 1, smooth: prefs.smooth, camera: p.camera || "static", camK: p.camK == null
> ? 1 : p.camK, light: p.light || "none", blur: p.blur || 0, loop: this.loop, autoplay, title: take.title };
76     const f = document.createElement("iframe");
77     f.setAttribute("sandbox", "allow-scripts");
78     f.title = "Animation preview";
79     f.srcdoc = buildDoc(take.code, cfg);
80     if (this.frame) this.frame.remove();
81     this.host.style.aspectRatio = W + " / " + H; this.host.style.setProperty("--ar", W / H);
82     this.host.prepend(f);
83     this.frame = f;
84     if (this.pending) this.pending({ ok: false, message: "replaced" });
85     return new Promise(res => {
86         this.pending = res;
87         clearTimeout(this.readyTimer);
88         this.readyTimer = setTimeout(() => { if (this.pending === res) { this.pending = null; res({ ok: false, message: "The animation didn't start in time." }); } }, 8000);
89     });
90 }
91 cmd(name, extra) { if (this.frame && this.frame.contentWindow) this.frame.contentWindow.postMessage(Object.assign({ ptmCmd: name }, extra), "*"); }
92 thumb(t, w = 320) {
93     return new Promise(res => {
94         this.thumbWait = res;
95         this.cmd("thumb", { t, w });
96         setTimeout(() => { if (this.thumbWait === res) { this.thumbWait = null; res(null); } }, 3000);
97     });
98 }
99 export(format, onProgress) {
100     return new Promise((resolve, reject) => { this.exportWait = { resolve, reject, onProgress }; this.cmd("export", { format }); });
101 }
102 destroy() { players.delete(this); if (this.frame) this.frame.remove(); clearTimeout(this.readyTimer); }
103 handle(m) {
104     switch (m.ev) {
105         case "ready": clearTimeout(this.readyTimer); if (this.pending) { this.pending({ ok: true }); this.pending = null; } break;
106         case "error":
107             clearTimeout(this.readyTimer);
108             if (this.pending) { this.pending({ ok: false, message: m.message }); this.pending = null; }
109             else this.onError(m.message);
110             break;
111         case "time": this.last = m; this.onTime(m); break;
112         case "ended": this.onEnded(); break;
113         case "perf": this.onPerf(m.smooth === false ? false : m.subs); break;
114         case "thumb": if (this.thumbWait) { this.thumbWait(m.url); this.thumbWait = null; } break;
115         case "export-progress": if (this.exportWait && this.exportWait.onProgress) this.exportWait.onProgress(m.p, m.label); break;
116         case "export-done": if (this.exportWait) { this.exportWait.resolve(m); this.exportWait = null; } break;
117         case "export-error": if (this.exportWait) { this.exportWait.reject(Object.assign(new Error(m.message), { cancelled: m.cancelled })); this.exportWait = null; } break;
118     }
119 }
120 }
121 window.addEventListener("message", e => {
122     const m = e.data;
123     if (!m || !m.ptm) return;
124     for (const p of players) if (p.frame && e.source === p.frame.contentWindow && m.sid === p.sid) p.handle(m);
125 });
126
127 // ----- small helpers
128 const fmtTime = s => { s = Math.max(0, s); const m = Math.floor(s / 60), r = s - m * 60; return String(m).padStart(2, "0") + ":" + r.toFixed(2).padStart(5, "0"); };
129 const slug = s => String(s).toLowerCase().replace(/[a-z0-9]+/g, "-").replace(/^-|-$/g, "").slice(0, 40) || "animation";
130 const ago = ts => { const s = (Date.now() - ts) / 1000; return s < 60 ? "just now" : s < 3600 ? Math.floor(s / 60) + " min ago" : Math.
> floor(s / 86400) + " d ago"; };
131 const uid = () => "t" + Date.now().toString(36) + Math.random().toString(36).slice(2, 6);
132 const mk = (tag, cls, text) => { const e = document.createElement(tag); if (cls) e.className = cls; if (text != null) e.textContent = text; return e; };
133 const sleep = ms => new Promise(r => setTimeout(r, ms));
134 const reduceMotion = () => matchMedia("(prefers-reduced-motion: reduce)").matches;
135 const bringInView = (el, block = "center") => { const r = el.getBoundingClientRect(); if (r.top < 70 || r.bottom > innerHeight) el.scrollIntoView({ behavior: reduceMotion() ? "aut
> o" : "smooth", block }); };
136 const bytes = n => (n > 1048576 ? (n / 1048576).toFixed(1) + " MB" : Math.max(1, Math.round(n / 1024)) + " KB");

```

```

137 const metaLine = t => t.params.aspect + " · " + t.params.duration + " s · " + t.params.fps + " fps · " + STYLE_NAME[t.params.style];
138 const lookLine = t => { const p = normParams(t.params); return [LIGHT_NAME[p.light], CAM_NAME[p.camera], p.blur ? BLURS[p.blur] + " blur" : "No blur"].join(" · "); };
139
140 // ----- state
141 let cur = null; // the take shown in the viewer (its params are the live settings)
142 let engine = "mock", busy = null, repairs = 0, tab = "mine";
143 let history = [];
144 try { history = JSON.parse(localStorage.getItem(HISTORY_KEY) || []).filter(t => t && t.code && t.params).map(t => Object.assign(t, { params: normParams(t.params) })); } catch { hi
> story = []; }
145 // Animations made by the offline generator are rebuilt from their prompt when the generator improves, so old saves get fixes too.
146 const rebuilt = [];
147 for (const t of history) if (t.engine === "mock" && t.mv !== PTM MOCK_VERSION) { try { const m = PTM_mock(String(t.prompt).split(" → ")[0], t.params.style); t.code = m.code; t.sound
> = { layers: m.analysis.layers, sky: m.analysis.sky }; t.mv = PTM MOCK_VERSION; rebuilt.push(t); } catch (e) { /* keep the old code */ } }
148 const saveHistory = () => {
149   for (let n = history.length; n >= 0; n--) {
150     try { localStorage.setItem(HISTORY_KEY, JSON.stringify(history.slice(0, n))); return; } catch { /* quota: keep fewer */ }
151   }
152 };
153
154 // ----- UI: style, aspect, fps, ideas
155 function renderControls() {
156   const seg = (host, items, attr, onPick, label) => {
157     host.innerHTML = "";
158     for (const [id, text, tip] of items) { const b = mk("button", "", text); b.type = "button"; b.setAttribute("role", "radio"); b.dataset[attr] = id; if (tip) b.title = tip; b.oncl
> ick = () => onPick(id); host.append(b); }
159   };
160   seg$("#styleSeg"), STYLES.map(s => [s.id, s.short, s.hint]), "style", v => setParam("style", v));
161   seg$("#aspectSeg"), Object.keys(ASPECTS).map(a => [a, a]), "aspect", v => setParam("aspect", v));
162   seg$("#fpsSeg"), FPS_OPTIONS.map(f => [String(f), String(f)]), "fps", v => setParam("fps", +v));
163   seg$("#blurSeg"), BLURS.map((n, i) => [String(i), n]), "blur", v => setParam("blur", +v));
164   seg$("#detailSeg"), DETAILS.map(([id, n]) => [id, n]), "detail", v => setParam("detail", v));
165   const fill = (sel, items) => { sel.innerHTML = ""; for (const [id, name] of items) { const o = mk("option", "", name); o.value = id; sel.append(o); } };
166   fill($("#cameraSel"), CAMERAS); fill($("#lightSel"), LIGHTS.map(l => [l[0], l[1]]));
167   $("#cameraSel").onchange = e => setParam("camera", e.target.value);
168   $("#lightSel").onchange = e => setParam("light", e.target.value);
169   $("#surpriseBtn").onclick = () => { const pool = IDEAS.filter(t => t !== $("#prompt").value); setVal($("#prompt"), pool[Math.floor(Math.random() * pool.length)]); $("#prompt").foc
> us(); };
170 }
171
172 function syncControls() {
173   const p = cur.params, on = (sel, attr, val) => document.querySelectorAll(sel).forEach(b => b.setAttribute("aria-checked", String(b.dataset[attr]) === String(val)));
174   on($("#styleSeg button", "style", p.style); on($("#aspectSeg button", "aspect", p.aspect); on($("#fpsSeg button", "fps", p.fps);
175   on($("#blurSeg button", "blur", p.blur); on($("#detailSeg button", "detail", p.detail);
176   $("#duration").value = p.duration; $("#durationOut").textContent = p.duration + " s";
177   $("#cameraSel").value = p.camera; $("#lightSel").value = p.light;
178   $("#camK").value = p.camK; $("#camKOut").textContent = Number(p.camK).toFixed(1) + "x"; $("#camK").disabled = p.camera === "static";
179   $("#physics").checked = !!p.physics;
180   $("#lookSummary").textContent = lookLine(cur);
181   $("#takeTitle").textContent = cur.title;
182   $("#takeMeta").textContent = metaLine(cur);
183   $("#code").textContent = cur.code.trim();
184   $("#loopBtn").setAttribute("aria-pressed", player.loop);
185   updateTimecode(player.last);
186 }
187
188 // Changing a setting updates the take that is on screen straight away.
189 let reloadTimer = 0;
190 function setParam(key, value) {
191   if (busy || !cur) return;
192   if (cur.params[key] === value) return;
193   cur.params[key] = value;
194   if (key === "style") {
195     if (cur.engine === "mock") { const m = PTM_mock(cur.prompt.split(" → ")[0], value); cur.code = m.code; cur.note = m.note; cur.mv = PTM MOCK_VERSION; }
196     $("#styleNote").textContent = cur.engine === "mock" ? "" : "This animation keeps its look. Pick Generate to redraw it in the new style. Pixel Art applies right away.";
197   }
198   if ((key === "duration" || key === "fps") && player.last) { // show the new frame count right away, even if the preview is off screen
199     const L = player.last, d = cur.params.duration, f = cur.params.fps, t = Math.min(L.t, d);
200     player.last = Object.assign({}, L, { t, duration: d, fps: f, frames: Math.round(d * f), frame: Math.min(Math.round(d * f) - 1, Math.floor(t * f)) });
201   }
202   syncControls();
203   if (key === "duration" || key === "fps" || key === "camera" || key === "camK" || key === "light" || key === "blur") player.cmd("config", { [key]: value });

```

```

204     else if (key === "detail" || key === "physics") setStatus("Scene detail and physics shape the next animation you generate.");
205     else { clearTimeout(reloadTimer); reloadTimer = setTimeout(() => showTake(cur), 120); }
206     persistCurrent();
207 }
208
209 // ----- viewer
210 const monitor = $("#monitor");
211 let scrubbing = false;
212 const player = new Player(monitor, {
213   onTime: m => { updateTimecode(m); if (window.PTM_SND) PTM_SND.onTime(m); },
214   onError: msg => showError("This animation hit an error: " + msg),
215   onEnded: () => setPlayIcon(false),
216   onPerf: info => {
217     if (info === false) { prefs.smooth = false; $("#smooth").checked = false; setStatus("This computer couldn't keep up with smooth preview, so it switched to drawing on your chosen
>     frame rate. Downloads are not affected."); }
218     else setStatus("The preview lowered its motion blur to keep the frame rate. Downloads use full quality.");
219   }
220 });
221 function setPlayIcon(playing) {
222   $("#playIcon").innerHTML = playing ? '<path d="M3 2h4v12H3zM9 2h4v12H9z"/>' : '<path d="M4 2l10 6-10 6z"/>';
223   $("#playBtn").setAttribute("aria-label", playing ? "Pause" : "Play");
224 }
225 function updateTimecode(m) {
226   const d = cur ? cur.params.duration : 0;
227   if (!m) { $("#timecode").textContent = fmtTime(0) + " / " + fmtTime(d); $("#frameNo").textContent = "frame 1/" + Math.round(d * (cur ? cur.params.fps : 30)); return; }
228   $("#timecode").textContent = fmtTime(m.t) + " / " + fmtTime(m.duration);
229   $("#frameNo").textContent = "frame " + (m.frame + 1) + "/" + m.frames;
230   setPlayIcon(m.playing);
231   if (!scrubbing) $("#scrub").value = Math.round((m.t / m.duration) * 1000);
232 }
233 function showError(msg) {
234   $("#errText").textContent = msg;
235   $("#errBox").classList.remove("hidden"); $("#errBox").classList.add("flex");
236   $("#fixBtn").hidden = !(cur && cur.engine === "claude");
237 }
238 function clearError() { $("#errBox").classList.add("hidden"); $("#errBox").classList.remove("flex"); }
239
240 async function showTake(take, { autoplay = true } = {}) {
241   cur = take; clearError();
242   if (window.PTM_SND) PTM_SND.setTake(take, take.params);
243   syncControls();
244   const r = await player.load(take, { autoplay });
245   if (r.message === "replaced") return r;
246   if (!r.ok) showError("This animation couldn't start: " + r.message);
247   return r;
248 }
249
250 $("#playBtn").onclick = () => { const playing = player.last ? player.last.playing : true; player.cmd(playing ? "pause" : "play"); };
251 $("#prevBtn").onclick = () => player.cmd("step", { n: -1 });
252 $("#nextBtn").onclick = () => player.cmd("step", { n: 1 });
253 $("#loopBtn").onclick = () => { player.loop = !player.loop; player.cmd("loop", { on: player.loop }); syncControls(); };
254 const soundBtn = $("#soundBtn");
255 if (window.PTM_SND && PTM_SND.supported) soundBtn.onclick = async () => { const on = await PTM_SND.toggle(); soundBtn.setAttribute("aria-pressed", on); soundBtn.title = on ? "Sound
> on (downloads are silent)" : "Sound for the preview (downloads are silent)"; }; else soundBtn.hidden = true;
256 $("#fullBtn").onclick = () => { (monitor.requestFullscreen ? monitor.requestFullscreen() : Promise.reject()).catch(() => setStatus("Full screen isn't available here.", "err")); };
257 const scrub = $("#scrub");
258 scrub.addEventListener("pointerdown", () => { scrubbing = true; });
259 addEventListener("pointerup", () => { scrubbing = false; });
260 scrub.addEventListener("input", () => { scrubbing = true; if (cur) player.cmd("seek", { t: (scrub.value / 1000) * cur.params.duration }); });
261 scrub.addEventListener("change", () => { scrubbing = false; });
262 document.addEventListener("keydown", e => {
263   const tag = e.target.tagName;
264   if (tag === "INPUT" && e.target.type !== "range" || tag === "TEXTAREA" || tag === "SELECT" || e.metaKey || e.ctrlKey) return;
265   if (e.key === " " && tag !== "BUTTON") { e.preventDefault(); $("#playBtn").click(); }
266   else if (e.key === "ArrowLeft" && tag !== "INPUT") { e.preventDefault(); $("#prevBtn").click(); }
267   else if (e.key === "ArrowRight" && tag !== "INPUT") { e.preventDefault(); $("#nextBtn").click(); }
268   else if ((e.key === "l" || e.key === "L") && tag !== "BUTTON") $("#loopBtn").click();
269 });
270 $("#duration").addEventListener("input", e => setParam("duration", +e.target.value));
271 $("#camK").addEventListener("input", e => setParam("camK", +e.target.value));
272 $("#physics").addEventListener("change", e => setParam("physics", e.target.checked));

```

```

273 $("#smooth").checked = prefs.smooth;
274 $("#smooth").addEventListener("change", e => { prefs.smooth = e.target.checked; try { localStorage.setItem("ptm-smooth", prefs.smooth ? "1" : "0"); } catch (err) { /* ignore */ } pL
> ayer.cmd("config", { smooth: prefs.smooth }); heroPlayer.cmd("config", { smooth: prefs.smooth }); });
275
276 // ----- status + busy overlay (a tiny job runner)
277 function setStatus(text, kind) { const s = $("#status"); s.textContent = text; s.style.color = kind === "err" ? "var(--err)" : kind === "ok" ? "var(--ok)" : "var(--mute)"; }
278
279 const FX = window.PTM_FX || { play: () => Promise.resolve(), typeInto: (el, t) => { el.textContent = t; return () => {} }, orb: () => ({ start() {}, stop() {}, burst() {} }), reduc
> e: true };
280 const orb = FX.orb($("#orbCanvas"));
281 let stopTyping = () => {};
282 const setVal = (el, v) => { el.value = v; el.dispatchEvent(new Event("input")); };
283
284 // The finish of every transition: the orb bursts, the new preview pulls into focus and an iris closes over the overlay.
285 async function irisReveal(busyEl, frame, orbObj) {
286   orbObj.burst();
287   if (frame) FX.play(frame, { filter: ["blur(16px)", "blur(0px)"], scale: [1.09, 1], opacity: [0.15, 1] }, { duration: 1, ease: "out" });
288   await FX.play(busyEl, { clipPath: ["circle(150% at 50% 50%)", "circle(0% at 50% 50%)"] }, { duration: 0.9, ease: "inOut", keep: true });
289   busyEl.hidden = true; busyEl.style.cssText = ""; orbObj.stop();
290 }
291
292 function startJob(title, steps, promptText) {
293   const ul = $("#busySteps"); ul.innerHTML = "";
294   const items = steps.map(s => { const li = mk("li"); li.dataset.s = "todo"; li.append(mk("span", "dot"), mk("span", "", s)); ul.append(li); return li; });
295   const busyEl = $("#busy"), mon = $("#monitor");
296   $("#busyTitle").textContent = title; $("#busyDetail").textContent = ""; $("#busyBar").style.width = "0%";
297   busyEl.hidden = false; busyEl.style.cssText = "";
298   mon.classList.add("making");
299   $("#generateBtn").dataset.busy = "1"; $("#generateBtn .btn-label").textContent = "Making your animation...";
300   // The overlay fades in over the old preview, which blurs and dims behind it. The prompt is typed into the orb.
301   FX.play(busyEl, { opacity: [0, 1], scale: [1.04, 1] }, { duration: 0.4 });
302   if (player.frame) FX.play(player.frame, { filter: ["blur(0px)", "blur(10px)"], scale: [1, 1.05] }, { duration: 0.5, keep: true });
303   orb.start(); stopTyping(); stopTyping = FX.typeInto($("#promptEcho"), promptText ? "" + promptText + "" : "", 46);
304   const job = {
305     ctl: new AbortController(), step: -1,
306     next(detail) { if (this.step >= 0) items[this.step].dataset.s = "done"; this.step++; if (items[this.step]) items[this.step].dataset.s = "run"; $("#busyBar").style.width = Math.m
> in(100, ((this.step + 1) / items.length) * 100 - 6) + "%"; if (detail != null) this.detail(detail); },
307     detail(t) { $("#busyDetail").textContent = t; },
308     // On success the orb bursts, an iris closes over the overlay and the new preview pulls into focus.
309     async end(success) {
310       items.forEach(li => (li.dataset.s = "done")); $("#busyBar").style.width = "100%";
311       busy = null; $("#generateBtn").disabled = ($("#changeBtn").disabled = false;
312       $("#generateBtn").dataset.busy = "0"; $("#generateBtn .btn-label").textContent = "Generate animation";
313       stopTyping(); mon.classList.remove("making");
314       if (success && player.frame) { await irisReveal(busyEl, player.frame, orb); return; }
315       if (player.frame) FX.play(player.frame, { filter: ["blur(10px)", "blur(0px)"], scale: [1.05, 1] }, { duration: 0.4 });
316       await FX.play(busyEl, { opacity: [1, 0] }, { duration: 0.3, keep: true });
317       busyEl.hidden = true; busyEl.style.cssText = ""; orb.stop();
318     }
319   };
320   busy = job; $("#generateBtn").disabled = ($("#changeBtn").disabled = true;
321   job.next();
322   return job;
323 }
324 $("#stopBtn").onclick = () => { if (busy) busy.ctl.abort(); };
325
326 // ----- generators
327 const STYLE_GUIDE = {
328   vector: "2D vector look: flat colours, clean geometric shapes, a tight harmonious palette, minimal gradients, smooth easing, playful overlapping motion.",
329   "3d": "3D render look: convincing depth from perspective, radial and linear gradients for volume, soft drop shadows, glow, parallax layers and lighting that moves with the action.
> Fake 3D is fine.",
330   anime: "Anime look: cel shading with two or three tones per object, bold dark ink outlines, dramatic sky, speed lines and sparkles on fast moments, punchy saturated colour.",
331   pixel: "Pixel art look: the canvas is rendered at one quarter resolution and scaled up with hard edges, so use chunky bold shapes, avoid thin lines and smooth gradients, and limit
> yourself to about 12 colours.",
332   frames: "Frame-by-frame hand-drawn look: quantise time to about 8 drawings per second (const q = Math.floor(progress * drawings) / drawings, with drawings = Math.round(duration *
> 8)) and drive all motion from q so it moves in held steps. Add a small wobble to every line using hash(...) of the drawing number, and draw slightly imperfect outlines on a warm p
> aper-coloured background."
333 };
334
335 const DETAIL_GUIDE = {
336   standard: "Detail level Standard: about 6 to 8 layers and clear, tidy shapes. Keep it under about 300 lines.",

```

```

337   high: "Detail level High: a rich background with at least 3 parallax depth layers and atmospheric haze; secondary details (small props, birds, lights, reflections); particle effec
>   ts (dust, sparks, mist); soft shadows and gradient shading on every subject; secondary motion on everything that moves. Keep it under about 380 lines.",
338   ultra: "Detail level Ultra: maximum detail. 5 or more depth layers; textured surfaces (windows, bricks, leaves, waves) built from many small elements; light shafts and glow; parti
>   cles; reflections; contact shadows; rim lighting; overlapping secondary animation. It may reach about 450 lines but must still hold the frame rate, so batch similar fills and neve
>   r create gradients inside big loops."
339   };
340
341   function rulesFor(params) {
342     const [W, H] = ASPECTS[params.aspect];
343     const p = normParams(params);
344     return `You write high-detail animations for a web app. Your JavaScript runs as a classic <script> in a page that already has a full-window <canvas>, and you must follow this exac
>     t contract.
345
346   CONTRACT
347   - Define these top-level functions: setup(W, H) (optional; runs once; returns a small state object), simulate(state, dt, t, info) (optional; physics, see below) and draw(ctx, t, info)
>     (required; called once per frame, and several times per frame when motion blur is on).
348   - ctx is a Canvas 2D context. Draw in a ${W} by ${H} pixel coordinate space (info.W and info.H); it is scaled for you. The canvas is already cleared to black before each call, so pain
>     t your own background every frame.
349   - t is seconds from 0 to info.duration. info = { W, H, duration: ${p.duration}, progress (t / duration, 0 to 1), frame, frames, fps: ${p.fps}, state, camera, light }.
350   - draw() must be a PURE function of t and info.state: the same t always draws the same picture. Never accumulate state between draw calls, never use Date or performance.now. Compute p
>     ositions from t or progress. Build per-object data in setup() using hash(i), never Math.random for things that must stay put.
351   - Make it a seamless loop: every repeating motion should complete a whole number of cycles over progress 0 to 1, or fade or start off screen, so frame 1 follows the last frame smoothl
>     y.
352   - Helpers already defined, do not redeclare them: TAU, clamp(v,a,b), lerp(a,b,k), hash(n) (deterministic 0 to 1), noise(x) (smooth 1D noise), ease.in, ease.out, ease.inOut, ease.sine,
>     ease.outBack, ease.outElastic (each takes 0 to 1), and PHYS (see below).
353   - Use only Canvas 2D and plain JavaScript. No imports, libraries, network, images, or external fonts. Text may use system-ui with ctx.fillText. Do not touch the DOM.
354   - Fill the entire canvas and place everything relative to W and H, so it works at any aspect ratio.
355   - The picture is drawn up to ${p.fps} times a second, so keep draw() efficient.
356
357   CAMERA AND LIGHT (the app applies these after you draw)
358   - The app already applies the camera move (${CAM_NAME[p.camera]}) and the colour grade (${LIGHT_NAME[p.light]}: bloom, vignette, tint) to whatever you draw, and it adds motion blur ($
>     {BLURS[p.blur]}). Do NOT draw vignettes, bloom, film grain, letterbox bars, camera shake or motion blur yourself.
359   - info.camera = { x, y, zoom, rot } (x and y are fractions of the frame). Use it to add parallax: shift each layer by info.camera.x * W * depth, with depth from 0.1 (far) to 1 (near).
360   - info.light = { dx, dy, color, color2, ambient, shadow }. dx and dy are a unit vector pointing TOWARD the light in screen coordinates (y is down). Put highlights and rim light on the
>     side that faces the light, cast shadows the opposite way in the shadow colour, and tint highlights with info.light.color. Shade every subject with at least a light and a dark ton
>     e so it looks lit, not flat.
361
362   PHYSICS AND MOVEMENT
363   - ${p.physics ? "Use physics for anything soft or springy (hair, scarves, cloth, ropes, tails, flags, splashes, bouncing)." : "Physics is optional here."} Define simulate(state, dt, t
>     , info); it changes state one small fixed step at a time (info has W, H, fps, duration, frame). The app runs it once and stores every frame, so scrubbing, looping and export stay
>     exact. draw() reads the result from info.state. Keep state small and JSON-friendly (numbers, arrays, plain objects); keep static scene data (buildings, stars) in module-level vari
>     ables made in setup, not in state.
364   - PHYS.spring(s, target, k, d, dt) with s = {x, v}; PHYS.makeChain(n, x, y, len) and PHYS.chain(points, anchorX, anchorY, dt, {gravity, wind, damping, iters}) for verlet cloth and rop
>     e; PHYS.ik2(ax, ay, tx, ty, l1, l2, dir) for two-bone legs and arms (returns the knee or elbow {x, y} and the end point {ex, ey}; dir is 1 or -1 for which way the joint bends).
365   - Characters need a real walk or run cycle: feet planted on the ground during stance, opposite arm swing, body bob and lean, anticipation before a jump, squash and stretch on landings
>     , and follow-through on cloth and hair. Move joints with easing, never linearly.
366
367   GROUND CONTACT (no foot may ever sink into the floor)
368   - Choose one ground line, for example groundY = H * 0.84, and draw the floor from there down. Every foot is a shape with a flat sole, and the lowest point of the sole must never be be
>     low groundY, in any frame.
369   - A planted foot has its sole exactly on groundY: with the shoe drawn around the ankle, put the ankle at groundY minus the sole thickness minus 1 pixel. Never put the ankle itself on
>     groundY, that pushes the shoe into the floor. While a foot swings, lift it along an arc.
370   - Legs must be able to reach the planted foot. Use PHYS.hipHeight(l1, l2, stride, bob) for the hip height above the ankle line and keep the stride under 0.6 of the leg length, so PHYS
>     .ik2 never has to clamp.
371   - A planted foot is carried backward at exactly the speed the ground or background scrolls (distance scrolled per loop divided by duration), so it grips the floor instead of skating.
>     The swing foot must leave and land with that same velocity: use a smooth curve (Hermite or eased), never a straight jump.
372   - Add a soft contact shadow under every planted foot that fades as the foot lifts. On stairs, slopes or platforms, sample the surface height under each foot and plant on that surface.
373   - Apply the same rule to any body that touches a surface: hands on a wall, wheels on a road, paws on the ground.
374
375   SMOOTHNESS
376   - No value may jump from one frame to the next. Every motion segment starts and ends with continuous speed: blend between poses with smoothstep or ease.inOut, never with a straight cu
>     t. Add follow-through (cloth, hair, tail) and overlapping action so parts do not all start and stop together.
377   - The preview is drawn up to 60 times a second at fractional times, so draw() must accept any t and be continuous in t.
378
379   DETAIL
380   ${DETAIL_GUIDE[p.detail]}
381
382   STYLE
383   ${STYLE_GUIDE[p.style]}
384
385   OUTPUT

```

```

386 Reply with ONLY one JSON object and nothing else: {"title": "short title, at most 6 words", "note": "one sentence on what to watch for", "sky": "one of day, sunset, night, space, unde
> rwater", "sound": ["up to 4 tags that describe what the scene should sound like, chosen from: rain, wind, sea, fire, city, lightning, rocket, car, ufo, dragon, character, pines, b
> irds, bubbles, confetti, hearts, neongrid, planets, snow"], "code": "the complete JavaScript"};
387 }
388
389 async function claudeCall(prompt, params, job) {
390   const sample = await samplePromise;
391   if (!sample) throw { code: "no_sample" };
392   let chars = 0;
393   const out = await sample.json(prompt, {
394     signal: job.ctl.signal, cache: false, modelTier: $("#tier").value,
395     onText: ({ text }) => { chars = text.length; job.detail("Claude has written " + Math.round(chars / 40) + " lines so far"); }
396   });
397   if (!out || typeof out.code !== "string" || !out.code.trim()) throw { code: "invalid_json" };
398   return { title: String(out.title || "Untitled").slice(0, 60), note: String(out.note || ""), code: out.code, sound: Array.isArray(out.sound) ? { layers: out.sound.map(String).slice
> (0, 6), sky: String(out.sky || "") } : null };
399 }
400
401 function errorCopy(e) {
402   const c = (e && e.code) || "";
403   return ({
404     cancelled: "Stopped.", no_sample: "Claude isn't available here, so use the Mock generator or open this page on claude.ai.",
405     not_granted: "Claude wasn't allowed to run for this page. Switch to the Mock generator, or allow it and try again.",
406     sampling_disabled: "Claude isn't available on this account. Use the Mock generator.",
407     rate_limited: "That was a lot of requests in a short time. Wait a minute and try again.",
408     refused: "Claude won't make that one. Try describing something different.",
409     invalid_json: "Claude's answer came back in the wrong shape. Press the button again.",
410     session_expired: "Your claude.ai sign-in expired. Sign in again, then retry."
411   })[c] || "Something went wrong reaching Claude. Try again in a moment.";
412 }
413
414 // Runs the scene once in the player; if it crashes and Claude wrote it, asks Claude to repair it (once).
415 async function previewWithRepair(take, job, params) {
416   let r = await showTake(take);
417   while (!r.ok && r.message !== "replaced" && take.engine === "claude" && repairs < 1 && !job.ctl.signal.aborted) {
418     repairs++;
419     job.detail("It hit an error (" + r.message.slice(0, 80) + "). Asking Claude to fix it.");
420     const fixed = await claudeCall(`${rulesFor(params)}\n\nThis animation code crashes with the error: ${r.message}\nHere is the code:\n\\`\\`js\n${take.code}\n\\`\\`\\nReturn the co
> mplete fixed code. Keep the animation the same.`, params, job);
421     take.code = fixed.code; r = await showTake(take);
422   }
423   return r;
424 }
425
426 async function finishTake(take) {
427   // save a thumbnail, then add to history
428   const dur = take.params.duration;
429   take.thumb = await player.thumb(dur * 0.4) || "";
430   history = [take, ...history.filter(t => t.id !== take.id)].slice(0, HISTORY_MAX);
431   tab = "mine"; saveHistory(); renderGallery();
432 }
433 let persistTimer = 0;
434 function persistCurrent() {
435   if (!cur || !history.some(t => t.id === cur.id)) return;
436   clearTimeout(persistTimer); persistTimer = setTimeout(() => { history = history.map(t => (t.id === cur.id ? cur : t)); saveHistory(); renderGallery(); }, 400);
437 }
438
439 async function generate({ prompt, refineFrom, change }) {
440   if (busy) return;
441   const params = Object.assign({}, cur.params);
442   repairs = 0; clearError(); bringIntoView($("#monitor"));
443   const useClaude = engine === "claude";
444   let ok = false;
445   const job = startJob(refineFrom ? "Changing your animation" : "Making your animation",
446     useClaude ? ["Sending your brief", "Claude is writing the scene", "Checking that it runs", "Rendering the preview"]
447     : ["Reading your prompt", "Planning the scene", "Building layers", "Rendering the preview"], refineFrom ? change : prompt);
448   const aborted = () => job.ctl.signal.aborted;
449   try {
450     let out;
451     if (useClaude) {
452       job.detail("Style: " + STYLE_NAME[params.style] + ", " + params.aspect + ", " + params.duration + " s");

```

```

453     const brief = refineFrom
454     ? `${rulesFor(params)}\n\nHere is the current animation, made from the request "${refineFrom.prompt}":\n\n\`\`\`js\n${refineFrom.code}\n\n\`\`\`\nChange it like this:\n\n\`\`\`${chan
> ge}"""\nReturn the complete new code, not a diff. Keep everything the viewer didn't ask to change.`
455     : `${rulesFor(params)}\n\nThe viewer asked for this animation:\n\n\`\`\`${prompt}""`;
456     job.next("Waiting for Claude...");
457     out = await claudeCall(brief, params, job);
458     job.next();
459   } else {
460     // Mock: a real generator with fake queue timing, so the job UI can be tested without any AI.
461     await sleep(350); if (aborted()) throw { code: "cancelled" };
462     job.next(); const a = PTM_analyze(prompt);
463     job.detail("Found: " + (a.layers.filter(l => l !== "abstract").join(", ") || "nothing specific")); await sleep(450);
464     if (aborted()) throw { code: "cancelled" };
465     job.next(); out = PTM_mock(prompt, params.style); await sleep(300);
466     if (aborted()) throw { code: "cancelled" };
467   }
468   const take = { id: uid(), title: out.title, note: out.note, prompt: refineFrom ? refineFrom.prompt.split(" → ")[0] + " → " + change : prompt, params, code: out.code, engine, mv:
> engine === "mock" ? PTM MOCK_VERSION : undefined, created: Date.now(), sound: out.analysis ? { layers: out.analysis.layers, sky: out.analysis.sky } : out.sound || null };
469   const r = await previewWithRepair(take, job, params);
470   if (r.message === "replaced") return;
471   if (r.ok) { ok = true; job.next(); await finishTake(take); setStatus("Done. Your animation is playing.", "ok"); }
472   else setStatus("The animation was made but it doesn't run. Try again or change the prompt.", "err");
473 } catch (e) {
474   setStatus(errorCopy(e), e && e.code === "cancelled" ? "" : "err");
475 } finally { await job.end(ok); }
476 }
477
478 // ----- forms
479 $("#studioForm").addEventListener("submit", e => {
480   e.preventDefault();
481   const prompt = $("#prompt").value.trim();
482   if (!prompt) { setStatus("Describe your animation first. What should happen, and what should be in it?", "err"); $("#prompt").focus(); return; }
483   generate({ prompt });
484 });
485 $("#prompt").addEventListener("keydown", e => { if (e.key === "Enter" && (e.metaKey || e.ctrlKey)) $("#studioForm").requestSubmit(); });
486
487 $("#changeBtn").onclick = () => {
488   const change = $("#change").value.trim();
489   if (!change) { setStatus("Type what you want to change first.", "err"); $("#change").focus(); return; }
490   if (engine === "mock") {
491     // Mock: fold the change into the prompt and rebuild, so keywords in it take effect.
492     const prompt = (cur.prompt.split(" → ")[0] + ". " + change).slice(0, 600);
493     setVal($("#prompt"), prompt); $("#change").value = ""; generate({ prompt });
494   } else { generate({ refineFrom: cur, change }); $("#change").value = ""; }
495 };
496 $("#change").addEventListener("keydown", e => { if (e.key === "Enter") { e.preventDefault(); $("#changeBtn").click(); } });
497 $("#fixBtn").onclick = () => generate({ refineFrom: cur, change: "It crashes with this error: " + ($("#errText").textContent + ". Fix the bug so it runs, and keep the animation the s
> ame." });
498
499 $("#heroForm").addEventListener("submit", e => {
500   e.preventDefault();
501   const v = $("#heroPrompt").value.trim(); if (!v) { $("#heroPrompt").focus(); return; }
502   setVal($("#prompt"), v);
503   $("#studio").scrollIntoView({ behavior: reduceMotion() ? "auto" : "smooth" });
504   setTimeout(() => $("#studioForm").requestSubmit(), 350);
505 });
506
507 // engine switch
508 function setEngine(name, quiet) {
509   engine = name;
510   document.querySelectorAll("#engineSeg button").forEach(b => b.setAttribute("aria-checked", b.dataset.engine === name));
511   $("#tierRow").hidden = name !== "claude";
512   $("#engineNote").textContent = name === "claude" ? "Claude writes a new scene for every prompt." : "Offline builds a scene from keywords in your prompt. It needs no AI.";
513   if (!quiet) setStatus("");
514 }
515 document.querySelectorAll("#engineSeg button").forEach(b => (b.onclick = () => { if (!b.disabled) setEngine(b.dataset.engine); }));
516 samplePromise.then(s => {
517   const btn = document.querySelector("#engineSeg [data-engine="claude"]');
518   if (s) setEngine("claude", true);
519   else { btn.disabled = true; btn.title = "Claude is only available when this page is open on claude.ai."; setEngine("mock", true); }
520 });

```

```

521 // ----- export
522 let exporting = false;
523 async function saveBlob(blob, filename) {
524   const d = await downloadsPromise;
525   if (d) { await d.save({ filename, data: blob }); return; }
526   const a = mk("a"); a.href = URL.createObjectURL(blob); a.download = filename; document.body.append(a); a.click(); a.remove();
527   setTimeout(() => URL.revokeObjectURL(a.href), 5000);
528 }
529 document.querySelectorAll("#exportBtns button").forEach(b => (b.onclick = async () => {
530   if (exporting || busy || !cur) return;
531   exporting = true; clearError();
532   const buttons = document.querySelectorAll("#exportBtns button"); buttons.forEach(x => (x.disabled = true));
533   $("#exportProg").classList.remove("hidden"); $("#exportProg").classList.add("grid");
534   $("#exportBar").style.width = "0%"; $("#exportLabel").textContent = "Starting...";
535   player.cmd("pause");
536   try {
537     const out = await player.export(b.dataset.format, (p, label) => { $("#exportBar").style.width = Math.round(p * 100) + "%"; $("#exportLabel").textContent = label + " · " + Math.r
538   >   ound(p * 100) + "%"; });
539     const name = slug(cur.title) + "-" + cur.params.aspect.replace(":", "x") + "." + out.ext;
540     await saveBlob(out.blob, name);
541     const fallback = b.dataset.format === "video" ? (out.ext !== "mp4" ? " This browser can't record MP4, so it saved WebM." : "") + " Recorded at " + out.width + " px" + (out.blurr
542   >   ed ? " with motion blur" : "") + (out.width < 1280 ? " so it could keep " + cur.params.fps + " fps in real time." : ".") : "";
543     setStatus("Saved " + name + " (" + bytes(out.blob.size) + ")." + fallback, "ok");
544   } catch (e) {
545     if (e && e.cancelled) setStatus("Export cancelled.");
546     else if (e && e.code === "declined") setStatus("Save cancelled.");
547     else setStatus("Couldn't export: " + ((e && e.message) || "unknown error"), "err");
548   } finally {
549     exporting = false; buttons.forEach(x => (x.disabled = false));
550     $("#exportProg").classList.add("hidden"); $("#exportProg").classList.remove("grid");
551     player.cmd("play");
552   }
553 }));
554 $("#exportCancel").onclick = () => player.cmd("cancel");
555
556 $("#codeBtn").onclick = () => { const c = $("#code"); c.hidden = !c.hidden; $("#codeBtn").textContent = c.hidden ? "Show scene code" : "Hide scene code"; $("#codeBtn").setAttribute(
557   >   "aria-expanded", !c.hidden); };
558 $("#copyBtn").onclick = () => {
559   (navigator.clipboard ? navigator.clipboard.writeText(cur.code.trim()) : Promise.reject()).then(
560     () => setStatus("Code copied.", "ok"),
561     () => { const c = $("#code"); c.hidden = false; const r = document.createRange(); r.selectNodeContents(c); const s = getSelection(); s.removeAllRanges(); s.addRange(r); setStatu
562   >   s("Copying was blocked. The code is selected, press Ctrl+C.", "err"); }
563   );
564 };
565
566 // ----- gallery / history
567 const exampleThumbs = {};
568 function openTake(t) {
569   const copy = JSON.parse(JSON.stringify(t)); if (t.engine === "example") copy.id = "ex-open-" + t.id;
570   setVal($("#prompt"), String(t.prompt).split(" → ")[0]);
571   showTake(t.engine === "example" ? copy : t);
572   setStatus("Opened "" + t.title + "". Edit the prompt or settings, then generate again.");
573   bringIntoView($("#monitor"));
574 }
575 function renderGallery() {
576   const host = $("#gallery"); host.innerHTML = "";
577   const list = tab === "mine" ? history : PTM_EXAMPLES;
578   document.querySelectorAll("#galleryTabs button").forEach(b => b.setAttribute("aria-pressed", b.dataset.tab === tab));
579   $("#clearBtn").hidden = tab !== "mine" || !history.length;
580   if (!list.length) {
581     const d = mk("div", "col-span-full rounded-2xl border border-dashed border-line p-8 text-center text-mute");
582     d.append(mk("p", "font-semibold text-ink", "Nothing here yet"), mk("p", "text-sm", "Animations you generate are saved here so you can reopen and edit them."));
583     host.append(d); return;
584   }
585   for (const t of list) {
586     const b = mk("div", "take"); b.tabIndex = 0; b.setAttribute("role", "button"); b.setAttribute("aria-label", "Open " + t.title);
587     b.setAttribute("aria-current", !(cur && (cur.id === t.id || cur.id === "ex-open-" + t.id)));
588     const [W, H] = ASPECTS[t.params.aspect]; const th = mk("div", "thumb"); th.style.aspectRatio = W + " / " + H;
589     const src = t.thumb || exampleThumbs[t.id];
590     if (src) { const im = mk("img"); im.src = src; im.alt = ""; th.append(im); } else { th.classList.add("shimmer"); th.append(mk("span", "text-xs text-mute", "Preview loading..."));

```

```

> }
588   const row = mk("div", "flex items-start justify-between gap-2");
589   const tx = mk("div", "min-w-0"); tx.append(mk("p", "truncate text-sm font-semibold", t.title), mk("p", "truncate text-xs text-mute", metaLine(t) + (t.created ? " · " + ago(t.cre
ated) : " · example"))));
590   row.append(tx);
591   if (tab === "mine") {
592     const del = mk("button", "btn btn-ghost btn-icon shrink-0", "x"); del.type = "button"; del.title = "Delete"; del.setAttribute("aria-label", "Delete " + t.title);
593     del.style.cssText = "width:30px;height:30px;padding:0;font-size:18px;line-height:1";
594     del.onclick = ev => { ev.stopPropagation(); history = history.filter(x => x.id !== t.id); saveHistory(); renderGallery(); };
595     row.append(del);
596   }
597   b.append(th, row);
598   b.onclick = () => openTake(t);
599   b.onkeydown = ev => { if (ev.key === "Enter") openTake(t); };
600   host.append(b);
601 }
602 }
603 document.querySelectorAll("#galleryTabs button").forEach(b => (b.onclick = () => { tab = b.dataset.tab; renderGallery(); }));
604 let clearArmed = 0;
605 $("#clearBtn").onclick = () => {
606   if (!clearArmed) { $("#clearBtn").textContent = "Click again to clear all"; clearArmed = setTimeout(() => { clearArmed = 0; $("#clearBtn").textContent = "Clear history"; }, 3000);
607   return; }
608   clearTimeout(clearArmed); clearArmed = 0; $("#clearBtn").textContent = "Clear history"; history = []; saveHistory(); renderGallery();
609 };
610 // Thumbnails for the built-in examples, drawn one at a time by a hidden player.
611 async function buildExampleThumbs() {
612   const host = mk("div"); host.style.cssText = "position:fixed;left:-9999px;top:0;width:320px"; document.body.append(host);
613   const p = new Player(host, { loop: false });
614   for (const ex of PTM_EXAMPLES) {
615     const r = await p.load(ex, { autoplay: false });
616     if (r.ok) { exampleThumbs[ex.id] = await p.thumb(ex.params.duration * 0.4); renderGallery(); }
617   }
618   let changed = false;
619   for (const t of rebuilt) { // saved animations that were just rebuilt: draw their thumbnails again
620     const r = await p.load(t, { autoplay: false });
621     if (r.ok) { const u = await p.thumb(t.params.duration * 0.4); if (u) { t.thumb = u; changed = true; } }
622   }
623   if (changed) { saveHistory(); renderGallery(); }
624   p.destroy(); host.remove();
625 }
626
627 // ----- hero player
628 const heroPlayer = new Player($("#heroMonitor"), { onTime: m => { $("#heroTc").textContent = fmtTime(m.t) + " · frame " + (m.frame + 1); } });
629 const heroOrb = FX.orb($("#heroOrb"), heroBusy = $("#heroBusy");
630 let heroWorking = false, heroVisible = true, stopHeroType = () => {};
631 new IntersectionObserver(es => { heroVisible = es[0].isIntersecting; }, { threshold: 0.2 }).observe($("#heroCard"));
632 // As each example prompt finishes typing, the preview "generates" it with the real engine: orb, then iris open.
633 async function heroShow(prompt) {
634   const entry = HERO_SHOW.find(h => h.prompt === prompt);
635   if (!entry || heroWorking || FX.reduce || !heroVisible || document.hidden) return;
636   heroWorking = true;
637   try {
638     const m = PTM_mock(prompt, entry.style);
639     const take = { id: "hero-" + Date.now(), title: m.title, note: m.note, prompt, code: m.code, engine: "mock", params: normParams({ style: entry.style, duration: 6, aspect: "16:9"
, fps: 30, camera: entry.camera, light: entry.light, blur: 1, detail: "high" }) };
640     heroBusy.hidden = false; heroBusy.style.cssText = "";
641     FX.play(heroBusy, { opacity: [0, 1] }, { duration: 0.3 });
642     if (heroPlayer.frame) FX.play(heroPlayer.frame, { filter: ["blur(0px)", "blur(8px)"], scale: [1, 1.04] }, { duration: 0.4, keep: true });
643     heroOrb.start(); stopHeroType(); stopHeroType = FX.typeInto($("#heroEcho"), "" + prompt + "", 70);
644     await Promise.all([heroPlayer.load(take), sleep(1300)]);
645     $("#heroTitle").textContent = take.title;
646     await irisReveal(heroBusy, heroPlayer.frame, heroOrb);
647   } finally { heroWorking = false; stopHeroType(); heroBusy.hidden = true; heroOrb.stop(); }
648 }
649 window.PTM_onHeroTyped = heroShow;
650
651
652 const heroTake = () => PTM_EXAMPLES.find(e => e.id === "ex-runner") || PTM_EXAMPLES[0];
653
654 // ----- boot

```

```
655   PTM_EXAMPLES.forEach(e => (e.params = normParams(e.params)));
656   renderControls();
657   cur = JSON.parse(JSON.stringify(history[0] || heroTake()));
658   if (!history[0]) cur.id = "ex-open-" + cur.id;
659   setVal($("#prompt"), String(cur.prompt).split(" → ")[0]);
660   setEngine("mock", true);
661   renderGallery();
662   showTake(cur).then(() => { setTimeout(buildExampleThumbs, 600); });
663   heroPlayer.load(heroTake());
664   window.PTM_IDEAS = IDEAS;
665   FX.boot && FX.boot();
666   if (!history.length) tab = "examples", renderGallery();
667 }());
```

```

1  #!/usr/bin/env python3
2  """Bundle PromptToMotion into one self-contained HTML file (dist/prompttomotion.html).
3
4  The single file loads Tailwind from its CDN (or from --tailwind-css if you pass a compiled stylesheet)
5  and inlines app.css and every script. Run it from anywhere: python3 prompttomotion/build.py
6  """
7  import argparse, pathlib, re
8
9  ROOT = pathlib.Path(__file__).parent
10
11 def inline_js(name):
12     # A literal "</script" inside inlined JS would end the tag early, so escape it (valid in strings, templates and regexes).
13     return "<script>" + (ROOT / "js" / name).read_text().replace("</script", "<\\ /script") + "</script>"
14
15 def main():
16     ap = argparse.ArgumentParser()
17     ap.add_argument("--tailwind-css", help="path to a compiled Tailwind stylesheet to inline instead of the CDN script")
18     ap.add_argument("--no-fonts", action="store_true", help="drop the Google Fonts link (offline builds)")
19     ap.add_argument("--full-document", action="store_true", help="wrap in <!doctype html><html><head>...<body> (needed outside the Artifact tool)")
20     ap.add_argument("-o", "--out", default=str(ROOT / "dist" / "prompttomotion.html"))
21     a = ap.parse_args()
22
23     html = (ROOT / "index.html").read_text()
24     html = html.replace('<link rel="stylesheet" href="css/app.css">', "<style>" + (ROOT / "css" / "app.css").read_text() + "</style>")
25     cdn = '<script src="https://cdn.tailwindcss.com/3.4.16"></script>\n<script src="js/tailwind.config.js"></script>'
26     if a.tailwind_css:
27         html = html.replace(cdn, "<style>" + pathlib.Path(a.tailwind_css).read_text() + "</style>")
28     else:
29         html = html.replace('<script src="js/tailwind.config.js"></script>', inline_js("tailwind.config.js"))
30     if a.no_fonts:
31         html = re.sub(r'<link rel="(?:stylesheet|preconnect)" href="https://fonts[^\>]*>\n?', "", html)
32     for name in ("fx.js", "runtime.js", "scenes.js", "sound.js", "app.js"):
33         html = html.replace('<script src="js/%s"></script>' % name, inline_js(name))
34     if a.full_document:
35         html = '<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover"></head><body>' + html
36         l + "</body></html>"
37     out = pathlib.Path(a.out); out.parent.mkdir(parents=True, exist_ok=True); out.write_text(html)
38     print("wrote", out, len(html), "bytes")
39 main()

```